はじめに

本資料は、Scala 初学者向けの学習テキストです。本資料を読み進めることで、

- プログラミング言語 Scala を用いたアプリケーションを開発できるようになること
- 『Scala スケーラブルプログラミング』（第 4 版）（通称コップ本）を通読して理解できるようになること
- 『Scala 関数型デザイン&プログラミング―Scalaz コントリビューターによる関数型徹底ガイド』（通称 FP in Scala）を通読して理解できるようになること

が主な目的です。

『Scala スケーラブルプログラミング』は、Scala の言語設計者である Odersky さんらにより書かれた解説書で、Scala の言語機能について詳細に書かれており、Scala プログラマにとってはバイブルと言える本です。この本は決して読みにくい本ではないのですが、本の分量が多いのと、関数型など他の言語でのプログラミング経験がないとわかりにくい箇所があります。本テキストを通じて Scala に入門することによって、この『コップ本』も読みやすくなるでしょう。

『Scala 関数型デザイン&プログラミング』は、Scalaz[*1]コントリビューターである Paul Chiusano さん、Rúnar Bjarnason さんらにより書かれた関数型プログラミングについての本です。あくまで関数型プログラミングの本なので Scala や Scalaz についての解説はあまりありません。ただし Scala についての最低限の文法の説明は随時行われています。「本文中で一から自作」というスタンスが徹底されており、型クラスのみならず、List、Stream、Future など、Scala 標準ライブラリに存在するものまで、一から自作しています。そのため「単にパターンを覚えて使う」のではなく、それぞれの関数型の色々な型クラスやパターンが「なぜそういう仕組になっているのか？」という根本的な考え方が分かるようになっています。細かい Scala 自体の言語仕様やテクニック、（標準ライブラリと外部のライブラリ含めた）既存の実在するライブラリの使い方は一切説明せず、とにかく「考え方や概念」のみを重点的に説明しているので、この本を読んで身につけた知識は古くなりにくいという点でおすすめできる一冊です。

読者層としては、

[*1] 関数型プログラミングを行うための純粋なデータ構造や、ファンクターやモナドといった型クラスとそのインスタンスを提供するライブラリのこと。

- 大学の情報学部卒相当である
- 小さなものでも、何かアプリケーションを作ったことがある
- 趣味でプログラミングを行っている

を仮定しています。上記の指標はあくまで目安であり、大学の情報学部卒でなければ理解できないといったことはありません。ただし、本資料は 1 つ以上のプログラミング言語でアプリケーションを作れることを前提にしていますので、その点留意ください。

フォーマット

このテキストは下記のフォーマットで提供されています。

- HTML 版：https://scala-text.github.io/scala_text/
- PDF 版：https://scala-text.github.io/scala_text_pdf/scala_text.pdf
- EPUB 版：https://scala-text.github.io/scala_text/scala_text.epub

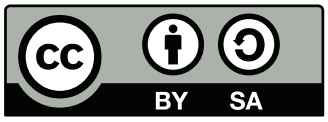
フィードバック

- 誤字・脱字の指摘や修正、および明確な技術的誤りの指摘や修正：
  - scala_text の issue 欄および pull request へ
- それ以外の改善要望や感想：
  - 専用 discussions へ

よろしくお願いいたします。

ライセンス

本文書は、CC BY-SA 3.0 の元で配布されています。



謝辞

本資料は、ドワンゴ社が作成した新卒エンジニア向けの Scala 研修資料が日本の Scala コミュニティに寄贈されたものです。

目次

はじめに i
フォーマット . . . ii
フィードバック . . . ii
ライセンス . . . ii
謝辞 . . . ii

第 1 章 Scala とは 1
1.1 なぜ開発言語として Scala を選ぶのか . . . 1
1.2 オブジェクト指向プログラミングと関数型プログラミングの統合 . . . 1
1.3 Java との互換性 . . . 3
1.4 非同期プログラミング、並行・分散プログラミング . . . 3

第 2 章 sbt をインストールする 4
2.1 SDKMAN!のインストール . . . 4
2.2 Java のインストール . . . 4
2.3 Mac OS, Linux, WSL（Windows）の場合 . . . 5
2.4 Windows の場合（WSL 以外） . . . 5
2.5 REPL と sbt . . . 6
2.6 sbt のバージョンについて . . . 7

第 3 章 Scala の基本 8
3.1 Scala のインストール . . . 8
3.2 REPL で Scala を対話的に試してみよう . . . 8
3.3 Hello, World! . . . 9
3.4 簡単な計算 . . . 9
3.5 変数の基本 . . . 11

第 4 章 sbt でプログラムをコンパイル・実行する 14

第 5 章 IDE (Integrated Development Environment) 17
5.1 sbt プロジェクトのインポート 24
5.2 プログラムを実行する 27

第 6 章 記法 29
6.1 アルファベットなどの並び 29
6.2 アルファベットなどの並びをクォートで囲んだもの 29
6.3 (と) で囲まれた要素 29
6.4 < と > で囲まれた要素 30
6.5 何らかの要素のあとに * が付加されたもの 30
6.6 何らかの要素のあとに + が付加されたもの 30
6.7 何らかの要素のあとに ? が付加されたもの 30
6.8 2 つの要素の間に | が付加されたもの 31
6.9 何らかの要素の後に ... が続くもの 31
6.10 if 式の構文 31

第 7 章 制御構文 32
7.1 「構文」と「式」と「文」という用語について 32
7.2 ブロック式 33
7.3 if 式 33
7.4 while 式 35
7.5 return 式 36
7.6 for 式 37
7.7 match 式 39

第 8 章 クラス 46
8.1 クラス定義 46
8.2 メソッド定義 47
8.3 複数の引数リストを持つメソッド 48
8.4 フィールド定義 49
8.5 抽象メンバー 49
8.6 継承 50

第 9 章 オブジェクト 54
9.1 コンパニオンオブジェクト 55

第 10 章 トレイト 57

10.1 トレイト定義 57
10.2 トレイトの基本 57
10.3 トレイトの様々な機能 60

第 11 章 型パラメータ（type parameter） 66
11.1 変位指定（variance） 68
11.2 型パラメータの境界（bounds） 72

第 12 章 Scala の関数 74
12.1 無名関数 74
12.2 関数の型 75
12.3 関数のカリー化 75
12.4 メソッドと関数の違い 76
12.5 高階関数 77

第 13 章 コレクションライブラリ（immutable と mutable） 80
13.1 Array 81
13.2 Map 95
13.3 Set 95

第 14 章 ケースクラスとパターンマッチング 97
14.1 変数宣言におけるパターンマッチング 100
14.2 ケースクラスによって自動生成されるもの 100
14.3 練習問題 102
14.4 練習問題 102
14.5 部分関数 105

第 15 章 エラー処理 107
15.1 エラーとは 107
15.2 エラー処理で実現しなければならないこと 108
15.3 Java におけるエラー処理 109
15.4 例外の問題点 110
15.5 エラーを表現するデータ型を使った処理 111
15.6 Option の例外処理を Either でリファクタする実例 122

第 16 章 implicit キーワード 126
16.1 Implicit conversion 126
16.2 Enrich my library 127
16.3 Implicit parameter（文脈引き渡し） 129

16.4 Implicit parameter（型クラス） 130

第 17 章 型クラスへの誘い 136
17.1 average メソッド 136
17.2 max メソッドと min メソッド 139
17.3 median メソッド 139
17.4 オブジェクトのシリアライズ 140
17.5 まとめ 143

第 18 章 Future/Promise について 144
18.1 Future とは 144
18.2 Promise とは 149

第 19 章 テスト 153
19.1 テストの分類 153
19.2 ユニットテスト 155
19.3 リファクタリング 155
19.4 テスティングフレームワーク 156
19.5 テストができる sbt プロジェクトの作成 156
19.6 Calc クラスとそのテストを実際に作る 157
19.7 モック 162
19.8 コードカバレッジの測定 163
19.9 コードスタイルチェック 164
19.10 テストを書こう 165

第 20 章 Java との相互運用 167
20.1 Scala と Java 167

第 21 章 S99 の案内 174
21.1 S-99: Ninety-Nine Scala Problems 174

第 22 章 トレイトの応用編：依存性の注入によるリファクタリング 175
22.1 サンプルプログラム 175
22.2 リファクタリング前のプログラムの紹介 175
22.3 リファクタリング：公開する機能を制限する 177
22.4 依存性の注入によるリファクタリング 181

第 23 章 付録：様々な型クラスの紹介 185
23.1 Functor 185

23.2 Applicative Functor 186
23.3 Monad 188
23.4 Monoid 189

第 1 章

Scala とは

Scala は 2003 年にスイス連邦工科大学ローザンヌ校（EPFL）の Martin Odersky 教授によって開発されたプログラミング言語です。Scala ではオブジェクト指向と関数型プログラミングの両方を行えるところに特徴があります。また、処理系は JVM 上で動作するため[*1]、Java 言語のライブラリのほとんどをシームレスに利用することができます。ただし、Scala はただの better Java ではないので、Scala で効率的にプログラミングするためには Scala の作法を知る必要があります。この文書がその一助になれば幸いです。

1.1 なぜ開発言語として Scala を選ぶのか

なぜ Scala を開発言語として選択するのでしょうか。ここでは Scala の優れた点を見ていきたいと思います。

1.2 オブジェクト指向プログラミングと関数型プログラミングの統合

Scala という言語の基本的なコンセプトはオブジェクト指向と関数型の統合ですが、それは Java をベースとしたオブジェクト指向言語の上に、関数型の機能を表現することで実現しています。

関数型プログラミングの第一の特徴は、関数を引数に渡したり、返り値にできたりする点ですが[*2]、Scala の世界では関数はオブジェクトです。一見メソッドを引数に渡しているように見えるところでも、そのメソッドを元に関数オブジェクトが生成されて渡されています。もちろんオブジェクト指向の世界でオブジェクトが第一級であることは自然なことです。Scala では、関数をオブジェクトやメソッドと別の概念として導入するのではなく、関数を表現するオブジェクトとして導入することで「統合」しているわけです。

[*1] https://www.scala-native.org や https://www.scala-js.org/ といった、JVM 以外の環境で動くものも存在します

[*2] この特徴を関数が「第一級（first-class）」であると言います。

他にも、たとえば Scala の「case class」は、オブジェクト指向の視点では Value Object パターンに近いものと考えられますが、関数型の視点では代数的データ型として考えることができます。また「implicit parameter」はオブジェクト指向の視点では暗黙的に型で解決される引数に見えますが、関数型の視点では Haskell の型クラスに近いものと見ることができます。

このように Scala という言語はオブジェクト指向言語に関数型の機能を足すのではなく、オブジェクト指向の概念で関数型の機能を解釈し取り込んでいます。それにより必要以上に言語仕様を肥大化させることなく多様な関数型の機能を実現しています。1 つのプログラムをオブジェクト指向の視点と関数型の視点の両方で考えることはプログラミングに柔軟性と広い視野をもたらします。

1.2.1　関数型プログラミング

最近は関数リテラルを記述できるようにするなど関数型プログラミングの機能を取り込んだプログラミング言語が増えてきていますが、その中でも Scala の関数型プログラミングはかなり高機能であると言えるでしょう。

- case class による代数的データ型
- 静的に網羅性がチェックされるパターンマッチ
- 型クラス
- for によるモナド構文
- モナドの型クラスの定義などに不可欠な高カインド型

以上のように Scala では単に関数が第一級であるだけに留まらず、本格的な関数型プログラミングをするための様々な機能があります。

また、Scala はオブジェクトの不変性（immutability）を意識している言語です。変数宣言は変更可能な var と変更不可能な val が分かれており、コレクションライブラリも mutable と immutable でパッケージがわかれています[*3]。case class もデフォルトでは immutable です。

不変性・参照透過性・純粋性は関数型プログラミングにおいて最も重要な概念と言われていますが、近年、並行・並列プログラミングにおける利便性や性能特性の面で、不変性は関数型に限らず注目を集めており、研究も進んでいます。その知見を応用できるのは Scala の大きな利点と言えるでしょう。

1.2.2　オブジェクト指向プログラミング

Scala が優れているのは関数型プログラミングの機能だけではありません。オブジェクト指向プログラミングにおいても様々な進化を遂げています。

- trait による mixin
- 構造的部分型
- 型パラメータの変位（variance）指定

[*3] https://docs.scala-lang.org/ja/overviews/collections/overview.html

- self type annotation による静的な依存性の注入
- implicit class あるいは extension による既存クラスの拡張
- Java のプリミティブ型がラップされて、全ての値がオブジェクトとして扱える

以上のように Scala ではより柔軟なオブジェクト指向プログラミングが可能になっています。Scala のプログラミングでは特に trait を使った mixin によって、プログラムに高いモジュール性と、新しい設計の視点が得られるでしょう。

1.3　Java との互換性

Scala は Java との互換性を第一に考えられた言語です。Scala の型やメソッド呼び出しは Java と互換性があり、Java のライブラリは普通に Scala から使うことができます。大量にある既存の Java ライブラリを使うことができるのは大きな利点です。

また Scala のプログラムは基本的に Java と同じようにバイトコードに変換され実行されるので、Java と同等に高速で、性能予測が容易です。コンパイルされた class ファイルを javap コマンドを使ってどのようにコンパイルされたかを確認することもできます。

運用においても JVM 系のノウハウをそのまま使えることが多いです。実績があり、広く使われている JVM で運用できるのは利点になるでしょう。

1.4　非同期プログラミング、並行・分散プログラミング

Scala では非同期の計算を表現する Future が標準ライブラリに含まれており、様々なライブラリで使われています。非同期プログラミングにより、スレッド数を超えるようなクライアントの大量同時のアクセスに対応することができます。

また、他のシステムに問い合わせなければならない場合などにも、スレッドを占有することなく他のシステムの返答を待つことができます。内部に多数のシステムがあり、外からの大量アクセスが見込まれる場合、Scala の非同期プログラミングのサポートは大きなプラスになります。

第 2 章

sbt をインストールする

現実の Scala アプリケーションでは、Scala プログラムを手動でコンパイル[*1]することは非常に稀で、標準的なビルドツールである sbt というツールを用いることになります。ここでは、sbt のインストールについて説明します。

2.1 SDKMAN!のインストール

ここでは、sbt と Java のいずれも SDKMAN!を利用したインストール方法を紹介します。Mac OS や Linux の場合、SDKMAN!は以下のコマンドでインストールできます。

```
$ curl -s "https://get.sdkman.io" | bash      # SDKMAN!のインストール
$ source "$HOME/.sdkman/bin/sdkman-init.sh"   # SDKMAN!の初期化（shellの再起動でも可）
$ sdk version                                 # パスが通っているかの確認
```

Windows の場合も WSL から同様の方法でインストールすることが可能です。

2.2 Java のインストール

Scala 2.12 以降は Java 8 以降が必須なので、もし Java がインストールされていなければ、まず Java をインストールしましょう。最新の Scala 3 や sbt 2 では JDK 17 以降が必須になるので、古い version の Scala を扱う予定がなければ、17 以降の LTS が推奨です。Scala と Java のそれぞれのバージョンの互換性の詳細に関しては、以下の Scala 公式サイトのページを見てください。

https://docs.scala-lang.org/overviews/jdk-compatibility/overview.html

ここでは、先ほどインストールした SDKMAN!を利用する方法を紹介します。

```
$ sdk list java                 # インストールできるJavaの一覧を確認
# sdk install java <Identifier> # <Identifier> の部分は上記のコマンドで確認した中からインストールしたいものを入れる
```

[*1] ここで言う”手動で”とは、scalac コマンドを直接呼び出すという意味です

```
$ sdk install java 17.0.19-tem  # 例
$ java -version                 # Javaがインストールされているかの確認（Java8以前の場
    合）
$ java --version                # Javaがインストールされているかの確認（Java11以降の場
    合）
```

2.3　Mac OS, Linux, WSL（Windows）の場合

最初にインストールした SDKMAN!を利用して sbt をインストールします。

```
$ sdk install sbt # sbtのインストール
$ which sbt       # sbtがインストールされているかの確認
```

とすれば、sbt がインストールできます。

2.3.1　Homebrew を利用する方法（Mac OS）

Homebrew を用いる方法でも可能です。

```
$ brew install sbt
```

でインストールでき、楽ですが、新しすぎる JDK がインストールされてしまうという問題があります。https://github.com/scala-text/scala_text/issues/566

2.4　Windows の場合（WSL 以外）

Windows で WSL を利用しない場合は、以下の方法で sbt をインストールすることができます。

Windows 公式の winget コマンド、あるいは chocolatey コマンドを使ってインストールすると楽です。

`winget` を使う場合は Windows Powershell を開いてください。`winget search` コマンドで最新のバージョンを確認できます。

```
winget search sbt
sbt  sbt.sbt <latest version>      winget
```

あとは `winget install sbt -v <version>` コマンドで指定したバージョンの sbt をインストールできます。

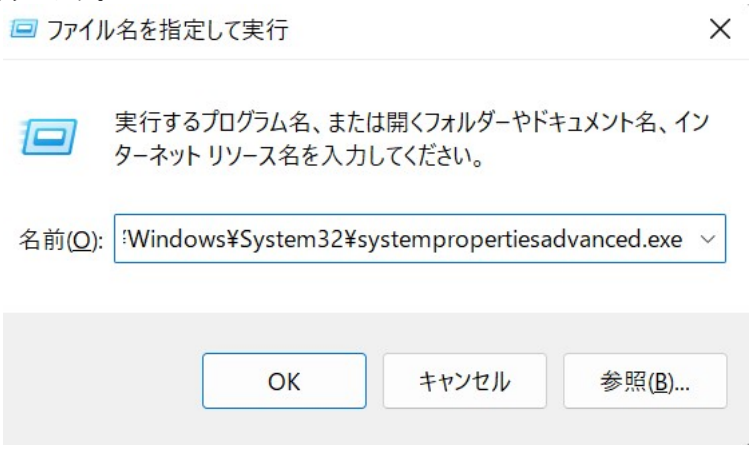
chocolatey は Windows 用のパッケージマネージャで活発に開発が行われてます。chocolatey のパッケージには sbt のものもあるので、

```
> choco install sbt
```

とすれば Windows に sbt がインストールされます。

Windows/Mac OS の場合で、シェル環境で sbt と入力するとバイナリのダウンロードが始まればインストールの成功です。sbt がないと言われる場合、環境変数へ sbt への PATH が通っていないだけですので追加しましょう。Windows の環境変数は「システムのプロパティ」から編集できます。

Windows キーと r キーを同時に押して `C:\Windows\System32\systempropertiesadvanced.exe` を入力します。



これが上手くいかない場合は、Windows キーと r キーを同時に押し、`sysdm.cpl` を入力して「システムのプロパティ」画面を開きます。

「システムのプロパティ」の「詳細設定」のタブを開き、ウィンドウの下の方にある「環境変数」ボタンを押して環境変数の設定画面を開きます。

環境変数に `PATH` が存在する場合は、`PATH` を編集して sbt のインストール先（例えば `C:\sbt\bin`）を追加します。環境変数に `PATH` が存在しない場合は新しく `PATH` 環境変数を追加して同じく sbt のインストール先を指定します。

2.5　REPL と sbt

これからしばらく、REPL（Read Eval Print Loop）機能と呼ばれる対話的な機能を用いて Scala プログラムを試していきますが、それは常に `sbt console` コマンドを経由して行います。

sbt console を起動するには、Windows でも Mac でも

```
$ sbt console
```

と入力すれば OK です。成功すれば、

```
[info] Loading global plugins from /Users/.../.sbt/1.0/plugins
[info] Set current project to sandbox (in build file:/Users/.../sandbox/)
[info] Updating {file:/Users/.../sandbox/}sandbox...
[info] Resolving org.fusesource.jansi#jansi;1.4 ...
[info] Done updating.
[info] Starting scala interpreter...
[info]
Welcome to Scala 3.8.4 (21.0.8, Java OpenJDK 64-Bit Server VM).
```

```
Type in expressions for evaluation. Or try :help.

scala>
```

のように表示されます。sbt console を終了したい場合は、

```
scala> :quit
```

と入力します。なお、sbt console を立ち上げる箇所には仮のディレクトリを作成しておくことをお勧めします。sbt はカレントディレクトリの下に *target* ディレクトリを生成してディレクトリ空間を汚してしまうからです。

ちなみに、このとき起動される Scala の REPL のバージョンは現在使っている sbt が依存している Scala のバージョンになってしまいます。しかし、この暗黙的に Scala の version が設定される機能は最新の sbt では非推奨で警告が出るので、必ず同じディレクトリに *build.sbt* というファイルを作成し、

```
1 scalaVersion := "3.8.4"
```

などとして明示的に Scala の version を指定しましょう。この_*.sbt が *sbt* のビルド定義ファイルになるのですが、今は *REPL* に慣れてもらう段階なので、この.sbt_ファイルの細かい書き方についての説明は省略します。

2.6　sbt のバージョンについて

この “sbt のバージョンについて” は、最新版を正常にインストールできた場合は、読み飛ばしていただいて構いません。

sbt は `sbt --version` もしくは `sbt --launcher-version` とすると version が表示されます[*2]。このテキストでは基本的に sbt 1.x[*3]がインストールされている前提で説明していきます。1.x 系であれば基本的には問題ないはずですが、無用なトラブルを避けるため、もし過去に少し古いバージョンの sbt をインストールしたことがある場合は、できるだけ最新版を入れておいたほうがいいでしょう。また、もし 0.13 系以前の version（0.13.16 など）が入っている場合は、色々と動作が異なり不都合が生じるので、その場合は必ず 1.x 系の最新版を入れるようにしてください。

[*2] ハイフンは 1 つではなく 2 つなので注意。version の詳細について知りたい場合は、こちらも参照。https://github.com/scala-text/scala_text/issues/122

[*3] 具体的にはこれを書いている 2026 年 6 月時点の最新版である sbt 1.12.13。

第 3 章

Scala の基本

この節では Scala の基本について、REPL を使った対話形式から始めて順を追って説明していきます。ユーザは Mac OS 環境であることを前提に説明していきますが、Mac OS 依存の部分はそれほど多くないので Windows 環境でもほとんどの場合同様に動作します。

3.1 Scala のインストール

これまでで sbt をインストールしたはずですので、特に必要ありません。sbt が適当なバージョンの Scala をダウンロードしてくれます。

3.2 REPL で Scala を対話的に試してみよう

Scala プログラムは明示的にコンパイルしてから実行することが多いですが、REPL（Read Eval Print Loop）によって対話的に Scala プログラムを実行することもできます。ここでは、まず REPL で Scala に触れてみることにしましょう。

先ほどのように、対話シェル（Windows の場合はコマンドプロンプト）から

```
$ sbt console
```

とコマンドを打ってみましょう。

```
Welcome to Scala 3.8.4 (21.0.8, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala>
```

のように表示されるはずです。これで Scala の世界に入る準備ができました。なお、$ の部分はシェルプロンプトなので入力しなくて良いです[*1]。

[*1] Windows の場合は、$ を> に読み替えてください。

3.3　Hello, World!

まずは定番の Hello, World!を表示させてみましょう。

```
println("Hello, World!")
// Hello, World!
```

無事、Hello, World!が表示されるはずです。次にちょっと入力を変えて、`println()` をはずしてみましょう。さて、どうなるでしょうか。

```
"Hello, World!"
// res1: String = "Hello, World!"
```

`res1: String = Hello, World` と、先ほどとは違う表示がされましたね。これは、`"Hello, World"` という式の型が `String` であり、その値が`"Hello, World"` であることを示しています。これまで説明していませんでしたが、Scala は静的な型を持つ言語で、実行される前に型が合っていることがチェックされます。

3.3.1　練習問題

様々なリテラルを REPL で出力してみましょう。

- `0xff`
- `1e308`
- `9223372036854775807L`
- `9223372036854775808L`
- `9223372036854775807`
- `922337203685477580.7`
- `1.00000000000000000001 == 1`
- `"\u3042"`
- `"\ud842\udf9f"`

どのように表示されるでしょうか。

3.4　簡単な計算

次に簡単な足し算を実行してみましょう。

```
scala> 1 + 2
res1: Int = 3
```

3 が表示され、その型が `Int` である事がわかりますね。なお、REPL の表示に出てくる `resN` というのは、REPL に何かの式（値を返すもの）を入力したときに、その式の値に REPL が勝手に名前をつけたものです。この名前は次のように、その後も使うことができます。

```
scala> res1
res2: Int = 3
```

この機能は REPL で少し長いプログラムを入力するときに便利ですので、活用していきましょう。Int 型には他にも `+`,`-`,`*`,`/` といった演算子が用意されており、次のようにして使うことができます。

```
1 + 2
// res2: Int = 3

2 * 2
// res3: Int = 4

4 / 2
// res4: Int = 2

4 % 3
// res5: Int = 1
```

浮動小数点数の演算のためにも、ほぼ同じ演算子が用意されています。ただし、浮動小数点数には誤差があるためその点には注意が必要です。見ればわかるように、`Double` という `Int` と異なる型が用意されています。`dbl.asInstanceOf[Int]` のようにキャストして型を変換することができますが、その場合、浮動小数点数の小数の部分が切り捨てられることに注意してください。

```
1.0 + 2.0
// res6: Double = 3.0

2.2 * 2
// res7: Double = 4.4

4.5 / 2
// res8: Double = 2.25

4 % 3
// res9: Int = 1

4.0 % 2.0
// res10: Double = 0.0

4.0 % 3.0
// res11: Double = 1.0

(4.5).asInstanceOf[Int]
// res12: Int = 4

(4.0).asInstanceOf[Int]
// res13: Int = 4
```

3.4.1　練習問題

これまで出てきた、+ ,- ,* ,/ ,% の演算子を使って好きなように数式を打ち込んでみましょう。

- `2147483647 + 1`
- `9223372036854775807L + 1`
- `1e308 + 1`
- `1 + 0.0000000000000000001`
- `1 - 1`
- `1 - 0.1`
- `0.1 - 1`
- `0.1 - 0.1`
- `0.0000000000000000001 - 1`
- `0.1 * 0.1`
- `20 * 0.1`
- `1 / 3`
- `1.0 / 3`
- `1 / 3.0`
- `3.0 / 3.0`
- `1.0 / 10 * 1 / 10`
- `1 / 10 * 1 / 10.0`

どのような値になるでしょうか。また、その型は何になるでしょうか。

3.5　変数の基本

ここまでは変数を使わずに REPL に式をそのまま打ち込んでいましたが、長い式を入力するときにそれでは不便ですね。Scala にも他の言語同様に変数があり、計算結果を格納することができます。注意しなければいけないのは Scala の変数には `val` と `var` の 2 種類があり、前者は一度変数に値を格納したら変更不可能で、後者は通常の変数のように変更可能だということです。

それでは、変数を使ってみることにします。Scala では基本的に、`var` はあまり使わず `val` のみでプログラミングします。これは **Scala でプログラミングをする上で大変重要なことなので**忘れないようにしてください。

```
1 val x = 3 * 2
2 // x: Int = 6
```

これは、変数 x を定義し、それに `3 * 2` の計算結果を格納しています。特筆すべきは、`x` の型を宣言していないということです。`3 * 2` の結果が `Int` なので、そこから `x` の型も `Int` に違いないとコン

パイラが推論できるため、変数の型を宣言する必要がないのです。Scala のこの機能を（ローカル）型推論と呼びます。定義した変数の型は Int と推論されたので、その後、x を別の型、たとえば String 型として扱おうとしても、コンパイル時のエラーになります。変数の型を宣言していなくてもちゃんと型が決まっているということです。

val を使った変数には値を再代入できず、型推論の効果がわかりにくいので、var を使った変数で実験してみます。var を使った変数宣言の方法は基本的に val と同じです。

```
scala> var x = 3 * 3
x: Int = 9

scala> x = "Hello, World!"
<console>:8: error: type mismatch;
 found   : String("Hello, World!")
 required: Int
       x = "Hello, World!"
           ^

scala> x = 3 * 4
x: Int = 12
```

ポイントは、

- var を使って宣言した場合も型推論がはたらく（3 * 3 → Int ）
- var を使った場合、変数の値を変更することができる（9 → 12 ）

というところです。

ここまでは変数の型を宣言せずに型推論にまかせて来ましたが、明示的に変数の型を宣言することもできます。変数の型を宣言するには

```
（'val'|'var'）<変数名> : <型名> = <式>
```

のようにします。

```
1 val x: Int = 3 * 3
2 // x: Int = 9
```

ちゃんと変数の型を宣言できていますね。

3.5.1　練習問題

これまで出てきた変数と演算子を用いて、より複雑な数式を入力してみましょう。

- Q. ¥3,950,000 を年利率 2.3 %の単利で 8 か月間借り入れた場合の利息はいくらか（円未満切り捨て）

A. ¥60,566

- Q. 定価¥1,980,000 の商品を値引きして販売したところ、原価 1.6 %にあたる¥26,400 の損失となった。割引額は定価の何パーセントであったか

A. 18 %

以上 2 つを、実際に変数を使って Scala のコードで解いてみましょう。

第4章

sbtでプログラムをコンパイル・実行する

前節まででは、REPLを使ってScalaのプログラムを気軽に実行してみました。この節ではScalaのプログラムをsbtでコンパイルして実行する方法を学びましょう。まずはREPLの時と同様にHello, World!を表示するプログラムを作ってみましょう。その前に、REPLを抜けましょう。REPLを抜けるには、REPLから以下のように入力します[*1]。

```
>:quit
```

Scala 2.10までは`exit`、Scala 2.11以降は`sys.exit`で終了することができますが、これらはREPL専用のコマンドではなく、今のプロセス自体を終了させる汎用的なメソッドなのでREPLを終了させる時には使用しないようにしましょう。

```
1 object HelloWorld {
2   def main(args: Array[String]): Unit = {
3     println("Hello, World!")
4   }
5 }
```

`object`や`def`といった新しいキーワードが出てきましたね。これらの詳しい意味はあとで説明するので、ここでは、`scalac`でコンパイルするプログラムはこのような形で定義するものと思ってください。`{}`で囲まれている部分はREPLの場合と同じ、`println("Hello, World!")`ですね。これを*HelloWorld.scala*という名前のファイルに保存します。

上記のプログラムはsbtでコンパイルし、実行することができます。ここでは*sandbox*というディレクトリを作成し、そこにプログラムを置くことにしましょう。

```
sandbox
```

[*1] `:quit`ではなく`:q`のみでも終了できます。

```
├── HelloWorld.scala
└── build.sbt
```

以上のようにファイルを置きます。

今回の *build.sbt* には Scala のバージョンと一緒に scalac の警告オプションも有効にしてみましょう。

```
// build.sbt
scalaVersion := "3.8.4"

scalacOptions ++= Seq("-deprecation", "-feature", "-unchecked")
```

この記述を加えることで scalac が

- 今後廃止の予定の API を利用している（-deprecation ）
- 明示的に使用を宣言しないといけない実験的な機能や注意しなければならない機能を利用している（-feature ）
- 型消去などでパターンマッチが有効に機能しない場合（-unchecked ）

などの警告の情報を詳しく出してくれるようになります。コンパイラのメッセージが親切になるので付けるようにしましょう。

さて、このようにファイルを配置したら *sandbox* ディレクトリに入り、sbt を起動します。sbt を起動するには対話シェルから以下のようにコマンドを打ちます。

```
$ sbt
```

すると sbt のプロンプトが出て、sbt のコマンドが入力できるようになります。今回は HelloWorld のプログラムを実行するために run コマンドを入力してみましょう。

```
> run
[info] Compiling 1 Scala source to ...
[info] Running HelloWorld
Hello, World!
[success] Total time: 1 s, completed 2015/02/09 15:44:44
```

HelloWorld プログラムがコンパイルされ、さらに実行されて Hello, World! と表示されました。run コマンドでは main メソッドを持っているオブジェクトを探して実行してくれます。

また sbt の管理下の Scala プログラムは console コマンドで REPL から呼び出せるようになります。*HelloWorld.scala* と同じ場所に *User.scala* というファイルを作ってみましょう

```
// User.scala
class User(val name: String, val age: Int)

object User {
  def printUser(user: User) = println(user.name + " " + user.age)
}
```

この *User.scala* には `User` クラスと `User` オブジェクトがあり、`User` オブジェクトには `User` の情報を表示する `printUser` メソッドがあります（クラスやオブジェクトの詳細についてはこの後の節で説明します）。

```
sandbox
├── HelloWorld.scala
├── User.scala
└── build.sbt
```

この状態で `sbt console` で REPL を起動すると、REPL で `User` クラスや `User` オブジェクトを利用することができます。

```
scala> val u = new User("dwango", 13)
u: User = User@20daebd4

scala> User.printUser(u)
dwango 13
```

今後の節では様々なサンプルコードが出てきますが、このように sbt を使うと簡単に自分で試してみることができるので、活用してみてください。

第5章

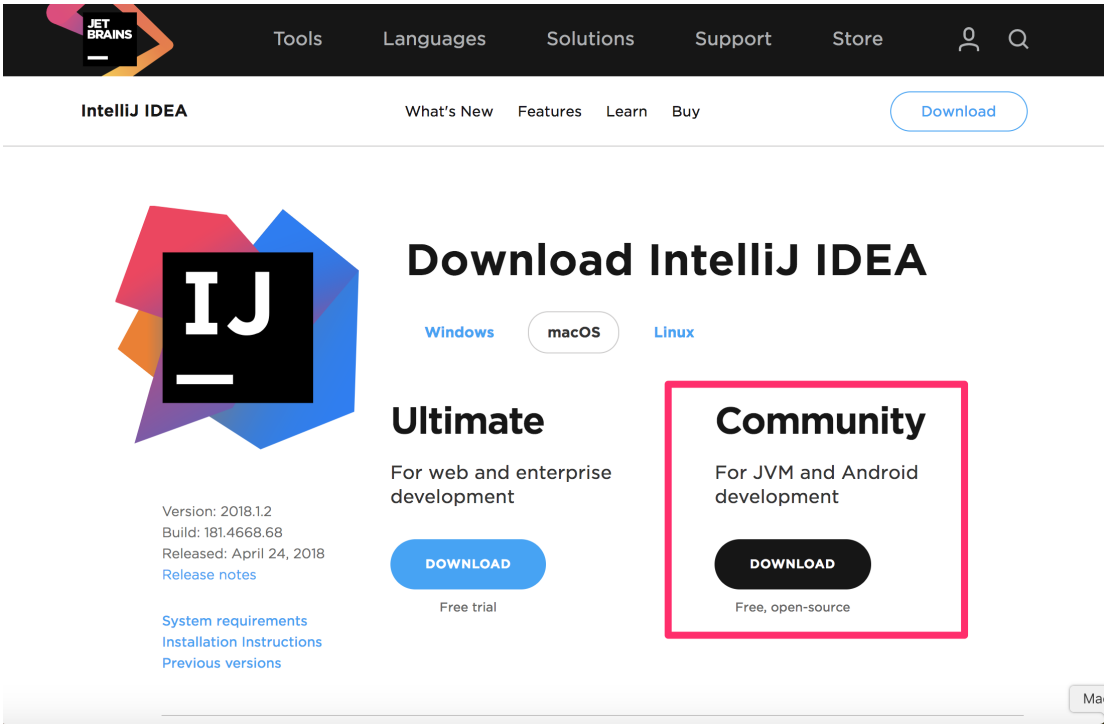
IDE (Integrated Development Environment)

Scala で本格的にコーディングする際は、エディタ（Emacs, Vim, Sublime Text）を使っても構いませんが、IDE を使うとより開発が楽になります。ここでは IntelliJ IDEA + Scala Plugin のインストール方法と基本的な操作について説明します。

補足：なお、本節で IDE をインストール後も説明は **REPL を使って行います**が、IDE ベースで学習したい場合は適宜コードを IDE のエディタに貼り付けて実行するなどしてください。

IntelliJ IDEA は JetBrains 社が開発している IDE で、Java 用 IDE として根強い人気を誇っています。有料ですが多機能の Ultimate Edition やオープンソース で無料の Community Edition があります。Community Edition でも、Scala プラグインをインストールすることで Scala 開発をすることができるので、本稿では Community Edition をインストール してもらいます。

まず、IntelliJ IDEA の Download ページに移動します。Windows, Mac OS X, Linux の 3 つのタブがあるので、それぞれの OS に応じたタブを選択して、「Download Community」ボタンをクリックしてください。以降、IDEA のスクリーンショットがでてきますが、都合上、Mac OS X 上でのスクリーンショットである ことをご承知ください。



ボタンをクリックすると、`ideaIC-${version}.dmg`（`${version}` はその時点での IDEA のバージョン）というファイルのダウンロードが始まるので（Windows の場合、インストーラの exe ファイル)、ダウンロードが終わるのを待ちます。ダウンロードが終了したら、ファイルをダブルクリックして、指示にしたがってインストールします。



インストールが終わったら起動します。スプラッシュスクリーンが現れてしばらく待つと、

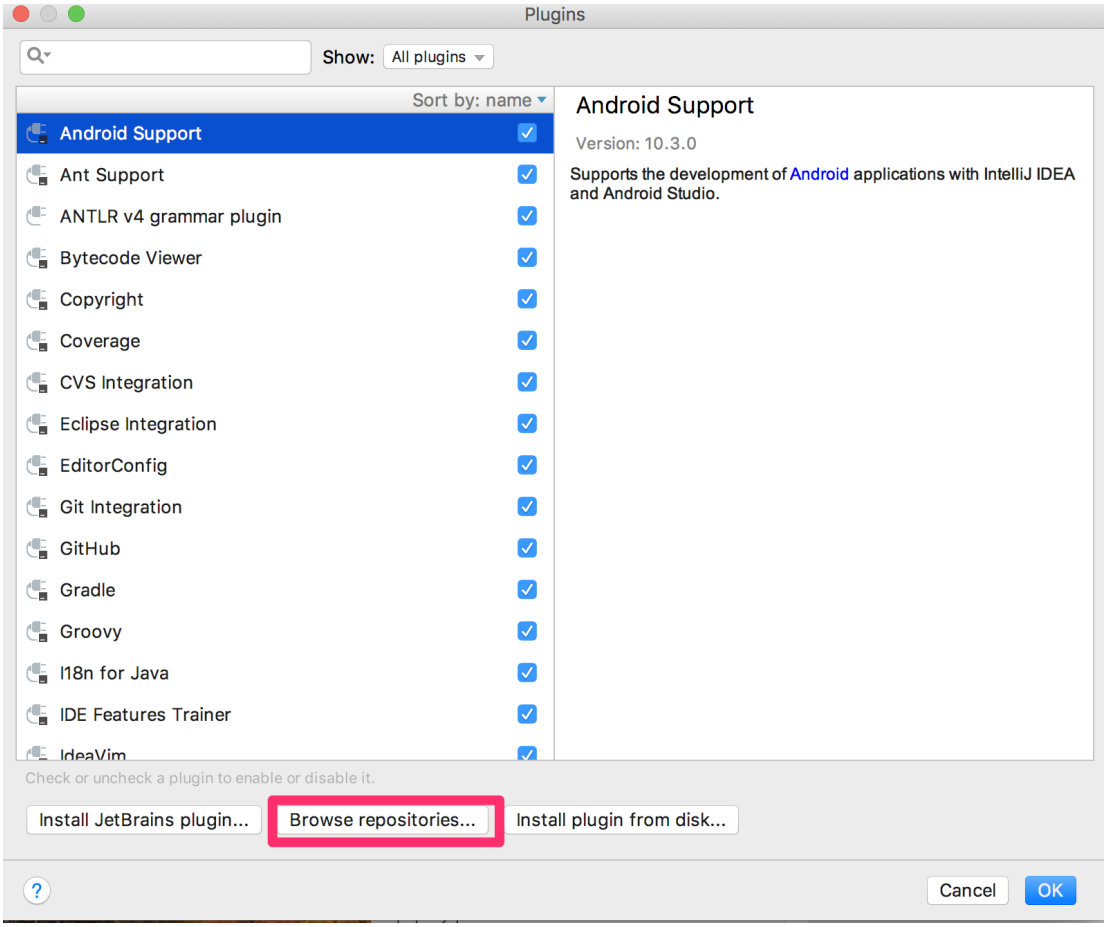


のような画面が表示されるはずです。ここまでで、IDEA のインストールは完了です。次に、IDEA

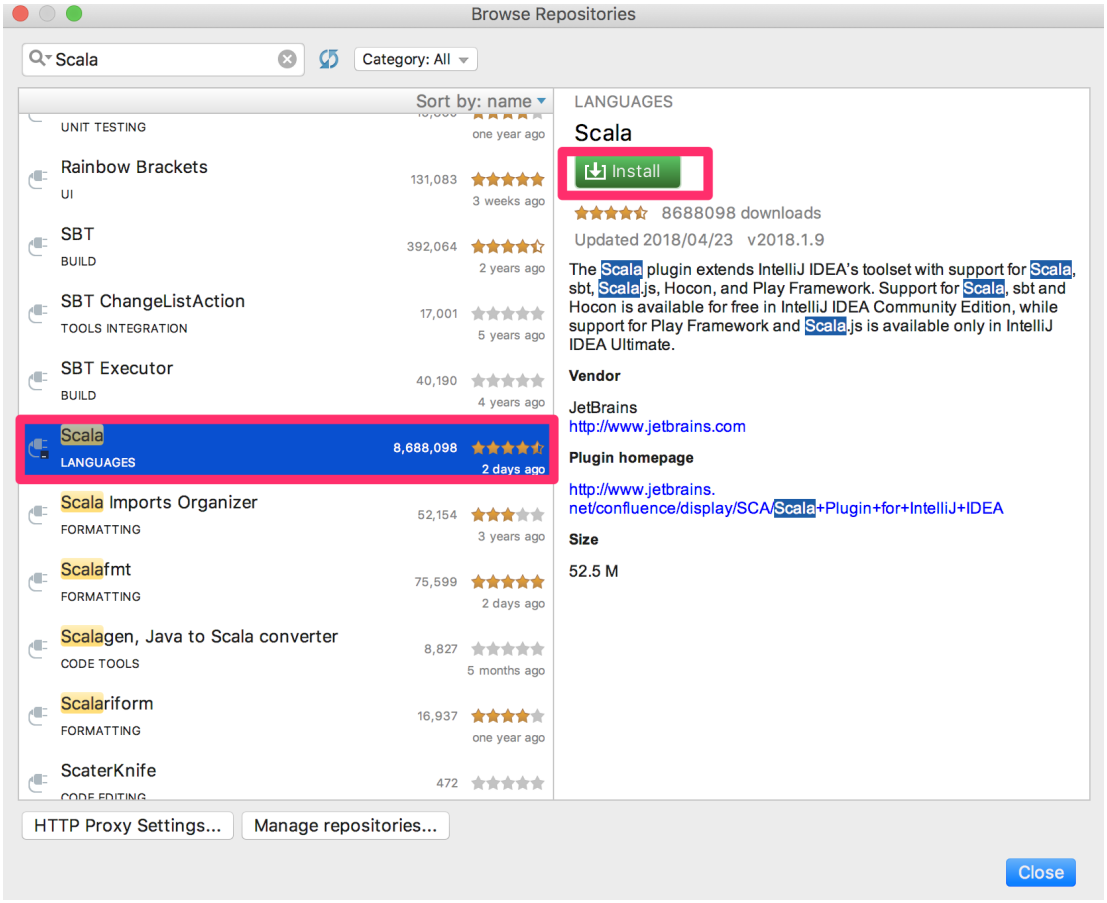
の Scala プラグインをインストールする必要があります。起動画面の `Configure->Plugins` をクリックしてください。



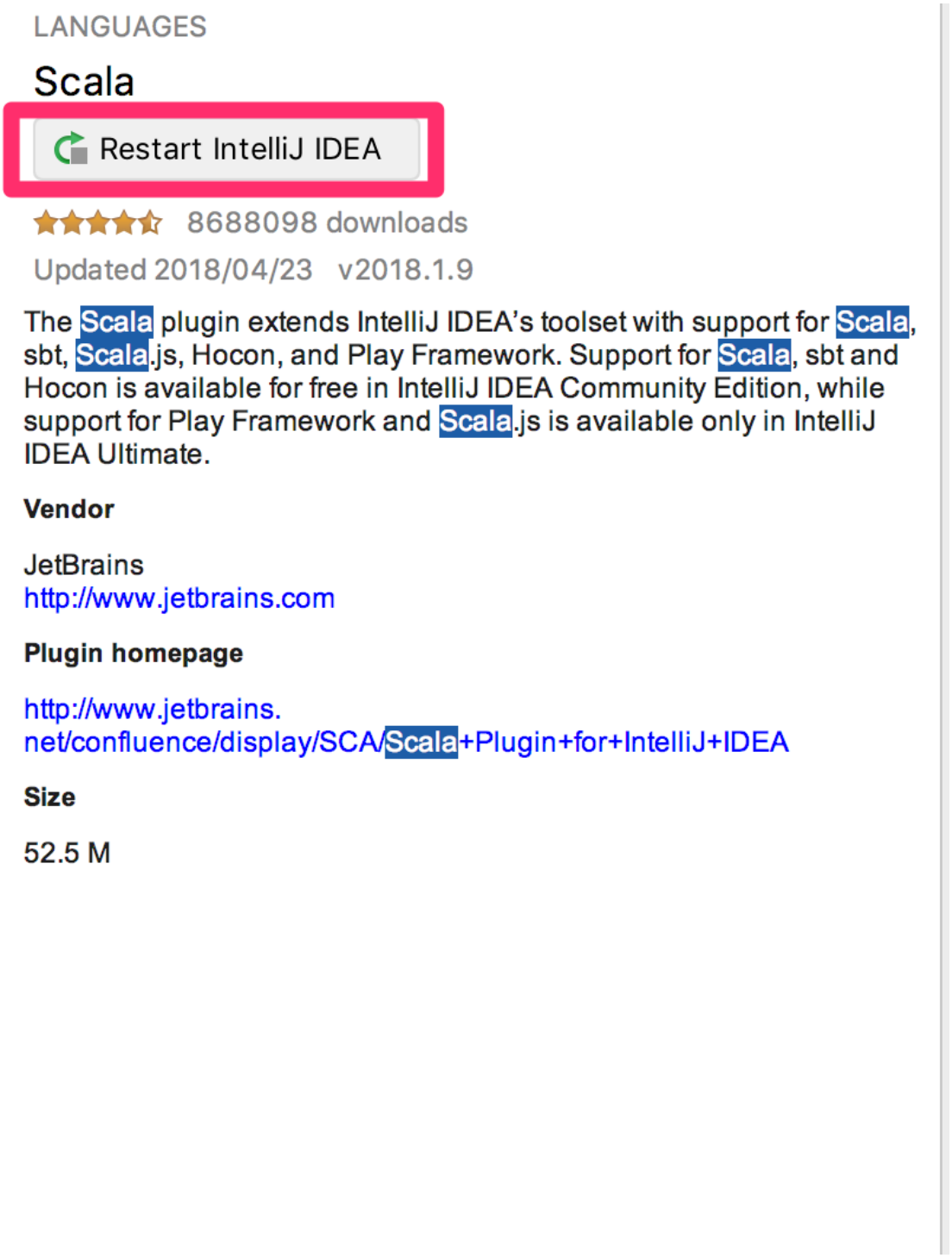
次のような画面が現れるので、その中の「Browse repositories」をクリックします。



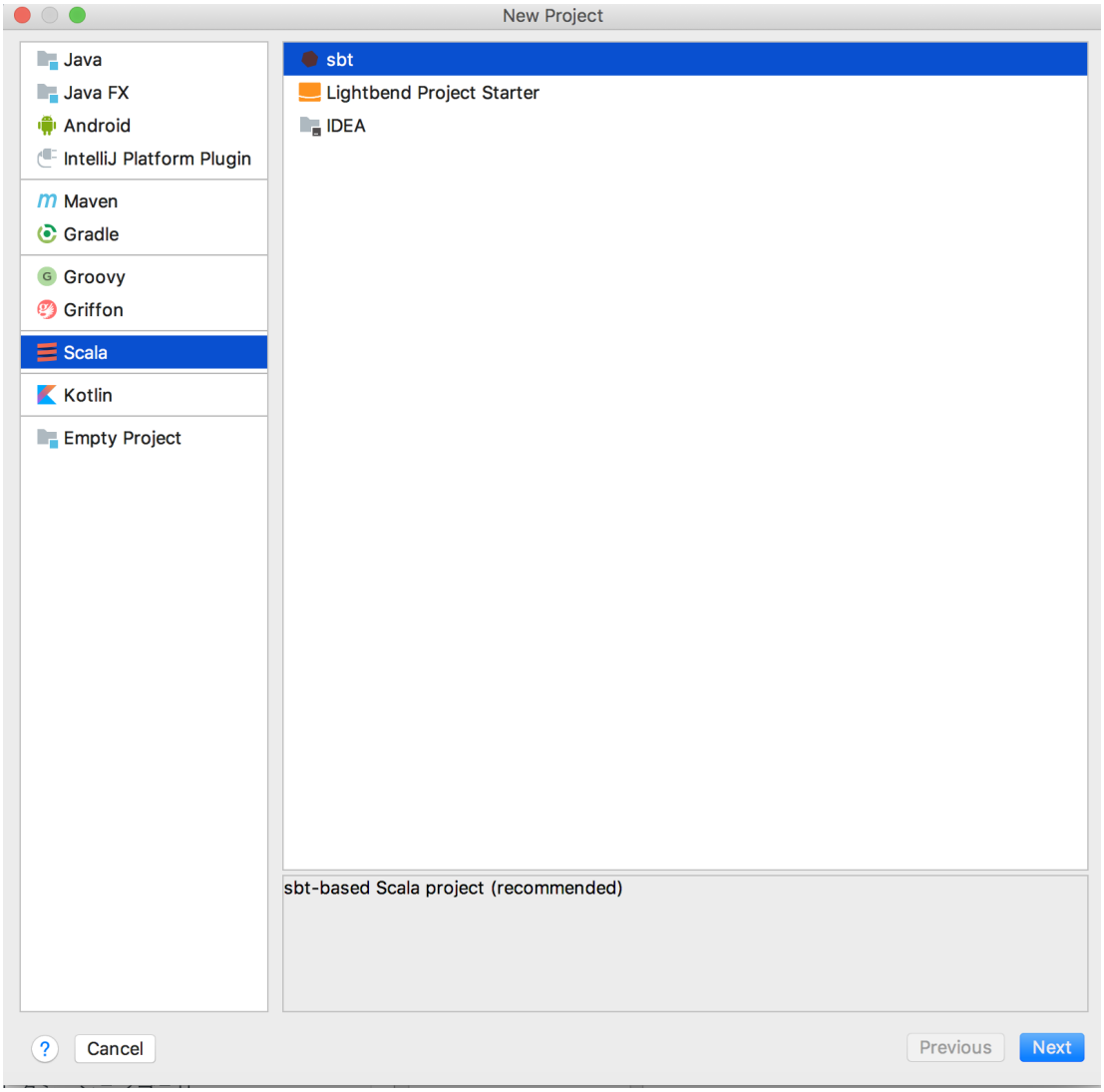
インストール可能なプラグインの一覧が現れるので、検索窓から `scala` で絞り込みをかけて、「Install」でインストールします（スクリーンショット参照）



Scala プラグインのダウンロード・インストールには若干時間がかかるのでしばらく待ちます。ダウンロード・インストールが完了したら、次のような画面が現れるので、「Restart IntelliJ IDEA」をクリックします。



再起動後、起動画面で「Create New Project」をクリックし、次の画面のようになっていればインストールは成功です。



5.1　sbt プロジェクトのインポート

次に、単純な sbt プロジェクトを IntelliJ IDEA にインポートしてみます。今回は、あらかじめ用意しておいた `scala-sandbox` プロジェクトを使います。`scala-sandbox` プロジェクトはここから `git clone` で clone します。`scala-sandbox` プロジェクトは次のような構成になっているはずです。

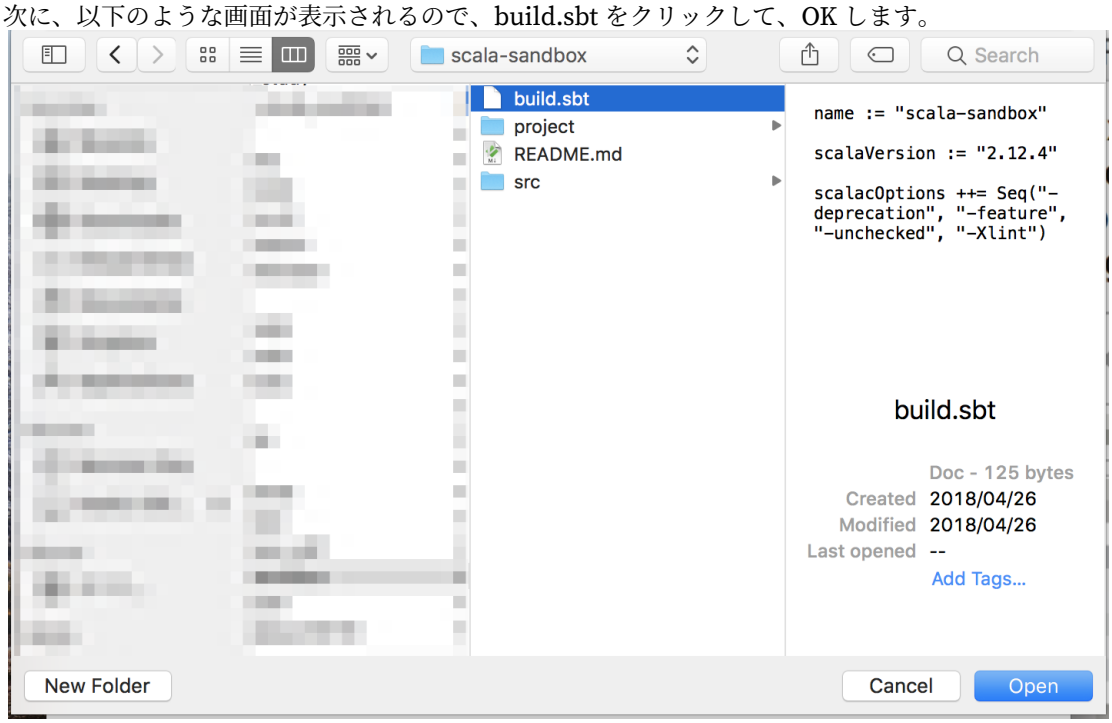
```
scala-sandbox
├── src/main/scala/HelloWorld.scala
├── project/build.properties
└── build.sbt
```

この `scala-sandbox` ディレクトリをプロジェクトとして IntelliJ IDEA に取り込みます。

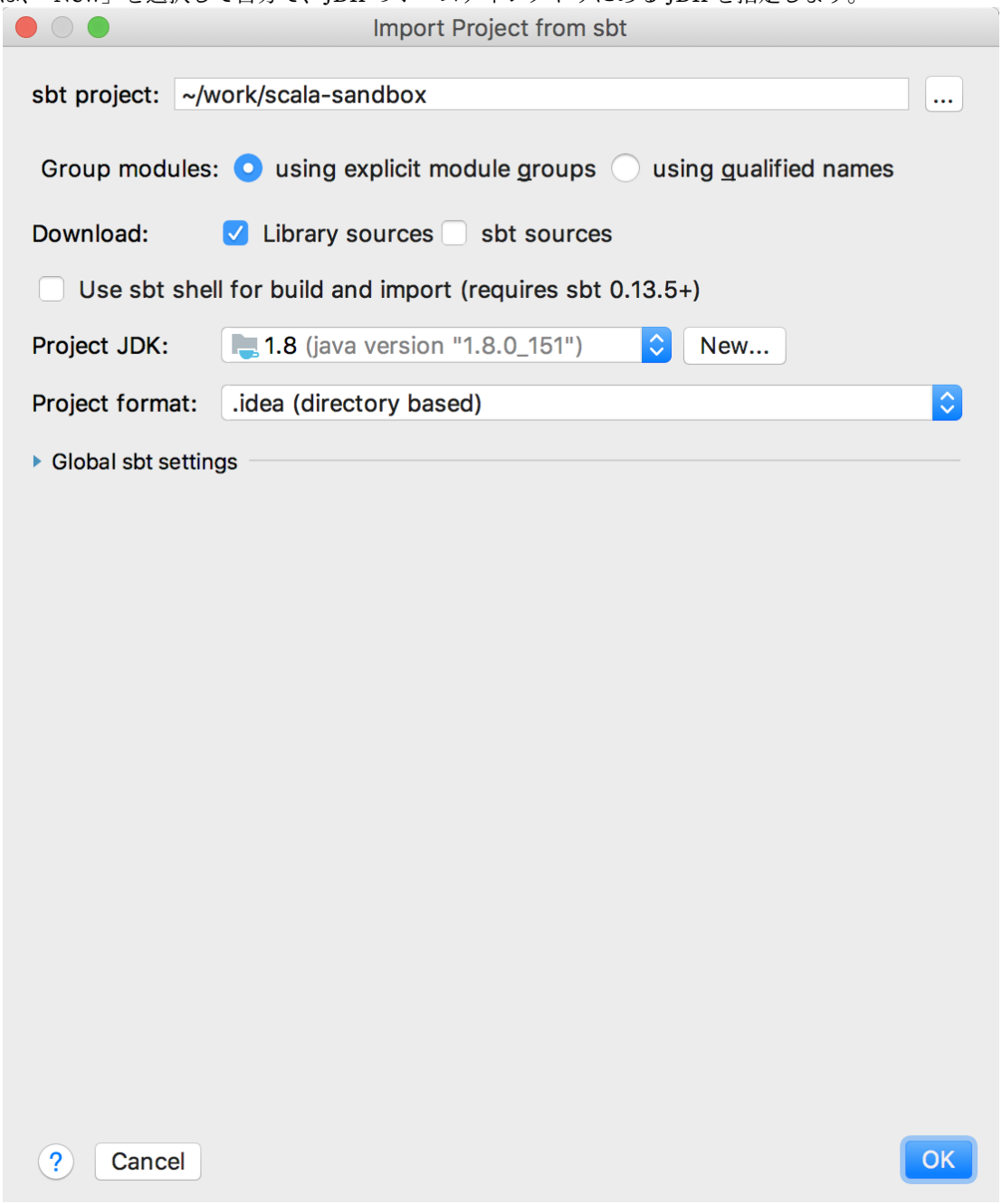
まず、IntelliJ IDEA の起動画面から、「Import Project」を選択します。



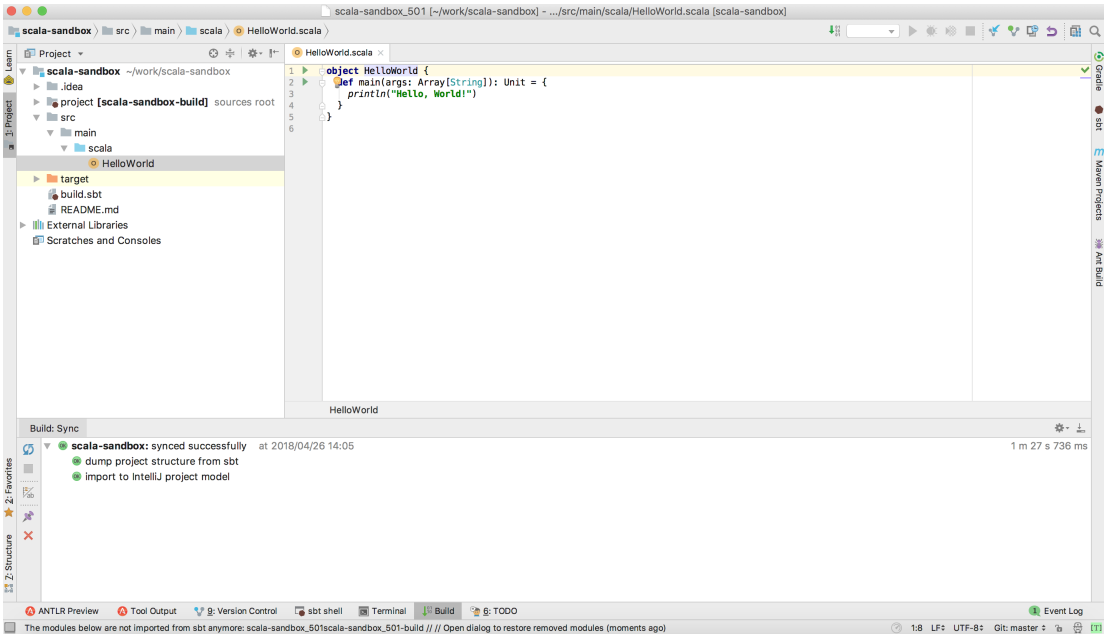
次に、以下のような画面が表示されるので、build.sbt をクリックして、OK します。



すると、さらに次のような画面が表示されます。「Project JDK」が空の場合がありますが、その場合は、「New」を選択して自分で、JDK のホームディレクトリにある JDK を指定します。



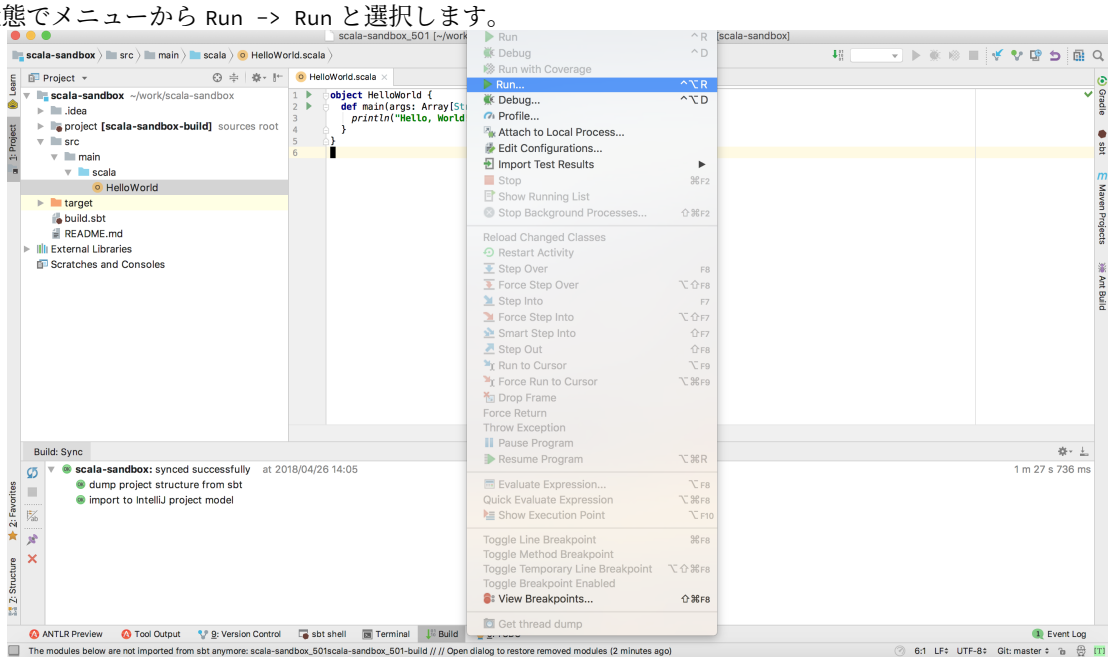
最後に、OK します。最初は、sbt 自体のソースを取得などするため時間がかかりますが、しばらく待つとプロジェクト画面が開きます。そこから、*HelloWorld.scala* を選んでダブルクリックして、以下のように表示されれば成功です。



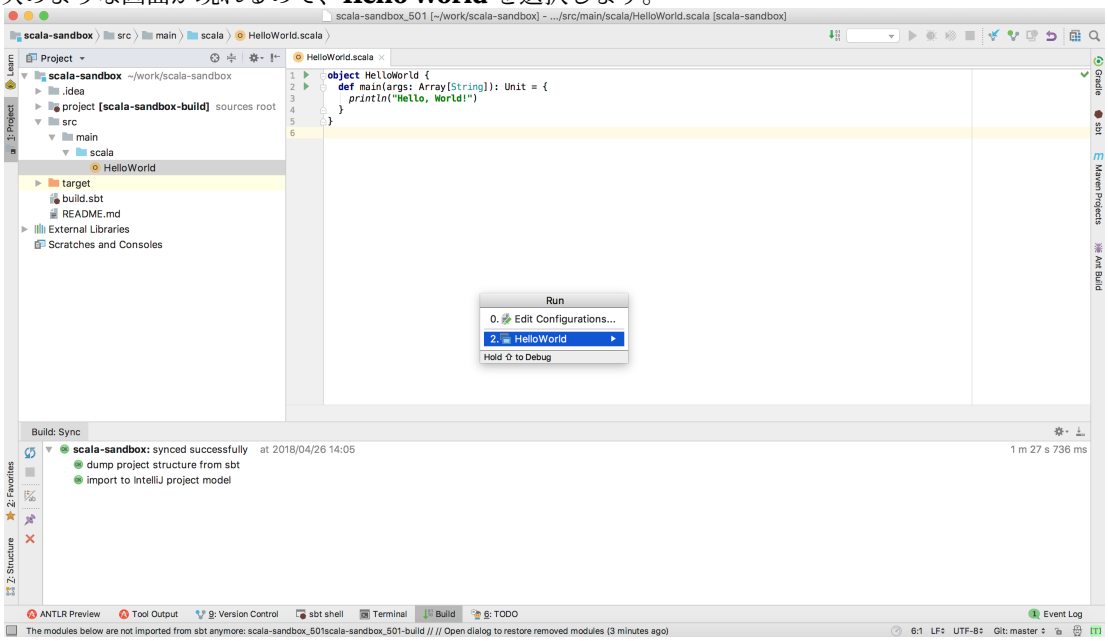
このように、IntelliJ IDEA では、sbt のプロジェクトをインポートして IDE 内で編集することができます。

5.2　プログラムを実行する

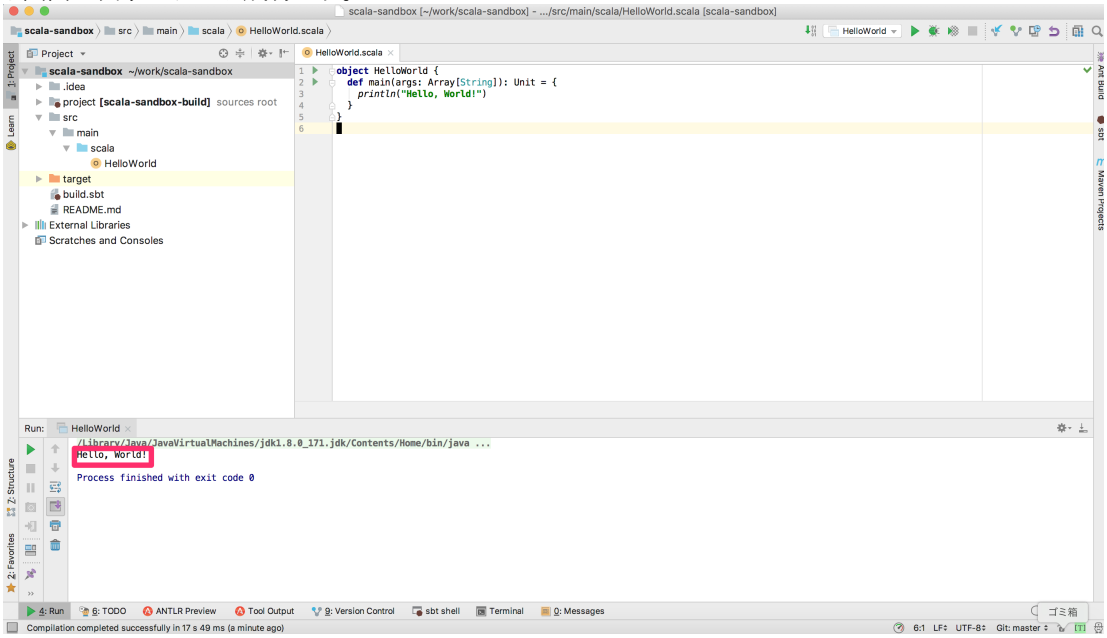
最後に、この HelloWorld プログラムを実行します。*HelloWorld.scala* を IDE のエディタで開いた状態でメニューから `Run -> Run` と選択します。



次のような画面が現れるので、**Hello World** を選択します。



すると、Hello World プログラムの最初のコンパイルが始まるので少し待ちます。その後、次のような画面が表示されたら成功です。



第 6 章

記法

このテキストは Scala の基本について学ぶものですので、これ以降、Scala のある構文に関する説明がしばしば出てきます。ここでは、構文を表すための記法について簡単に説明します。

6.1 アルファベットなどの並び

まず、以下のように、アルファベットや記号のならびがそのまま現れた場合、その文字列そのものを表します。ここでは、`if` という文字列そのものを表しているわけです。

```
if
```

6.2 アルファベットなどの並びをクォートで囲んだもの

次に、アルファベットや記号の並びをクオートで囲んだものも、同様に扱います。これは後述するように、一部の文字に特別な意味をもたせるため、それとの混同を避けるために使います。以下は、先程と同じ意味です。

```
'if'
```

6.3 （と）で囲まれた要素

なんらかの要素で、`()` で囲まれたものは、グルーピングを表現します。以下は、`(` で始まる何らかの文字列ではなく、`if()` という文字の並びをひとまとめにしたものを表します。これは、後述する繰り返しの表現などをひとまとめにするために使います。

```
('if' '(' ')')
```

明示的に `'('` や `')'` としない限り、グルーピングが優先されます。

6.4　< と > で囲まれた要素

<式> のように < と> で囲んで名前をつけたものは、何らかの構文要素を表します。<式> と書くことで、式 という概念を表現する何らかの構文要素であることを示します。以下では、Java の `if` 文の構文の一部を表現しています。<条件式> が何かについては触れていませんが、Java の言語で `boolean` が返ってくる式を想定しています。

```
if '(' <条件式> ')'
```

6.5　何らかの要素のあとに * が付加されたもの

何らかの要素に続いて、$\star$ が付加されたものは、その要素が 0 回以上現れることを意味します。以下は、<式> のあとに ; が来るような要素が 0 回以上現れることを意味します。

```
(<式> ;)*
```

ここで、a$\star$ は a のあとに $\star$ という文字が来るのか、a の 0 回以上の繰り返しなのかが曖昧です。曖昧さを解決するために、明示的に '$\star$' としない限り、繰り返しが優先されるものとします。

6.6　何らかの要素のあとに + が付加されたもの

何らかの要素に続いて、+ が付加されたものは、その要素が 1 回以上現れることを意味します。以下は、<式> のあとに ; が来るような要素が 1 回以上現れることを意味します。

```
(<式> ;)+
```

ここで、a+ は a のあとに + という文字が来るのか、a の 1 回以上の繰り返しなのかが曖昧です。曖昧さを解決するために、明示的に '+' としない限り、繰り返しが優先されるものとします。

6.7　何らかの要素のあとに ? が付加されたもの

何らかの要素に続いて、? が付加されたものは、その要素が 0 回または 1 回現れることを意味します。言い換えると、その要素はオプショナルであるということになります。以下は、

`else` で始まり、その次に<式> が来るような要素が 0 回または 1 回現れることを意味します。

```
(else <式>)?
```

ここで、a? は a のあとに ? という文字が来るのか、a の 0 回または 1 回の出現なのかが曖昧です。曖昧さを解決するために、明示的に '?' としない限り、オプショナルが優先されるものとします。

6.8　2 つの要素の間に | が付加されたもの

何らかの 2 つの要素 A と B の間に | が付加されたものは、A と B のどちらでも良いということを意味します。以下は、val または var のどちらでも良いことを意味します。

```
('val'|'var')
```

ここで、a|b は a|b という 3 文字なのか、a または b なのかが曖昧です。曖昧さを解決するために、明示的に '|' としない限り、a or b という解釈が優先されます。

6.9　何らかの要素の後に . . . が続くもの

任意個の要素が来るときに、最初の数個を例示し、後は同じパターンで出現することを明示するために利用します。以下は、[と] で囲まれ、式が任意個続くパターンを表しています。

```
'[' <式1>, <式2>, ... ']'
```

6.10　if 式の構文

以上を踏まえて、Scala の if 式の構文を表現すると、次のようになります。

```
if '(' <条件式> ')' <式> ( else <式> )?
```

Scala の if 式については、あとで詳しく解説します。

第 7 章

制御構文

この節では、Scala の制御構文について学びます。通常のプログラミング言語とくらべてそれほど突飛なものが出てくるわけではないので心配は要りません。

7.1 「構文」と「式」と「文」という用語について

この節では「構文」と「式」と「文」という用語が入り乱れて使われて少々わかりづらいかもしれないので、先にこの 3 つの用語の解説をしたいと思います。

まず「構文（Syntax）」は、そのプログラミング言語内でプログラムが構造を持つためのルールです。多くの場合、プログラミング言語内で特別扱いされるキーワード、たとえば `class` や `val` 、`if` などが含まれ、そして正しいプログラムを構成するためのルールがあります。`class` の場合であれば、`class` の後にはクラス名が続き、クラスの中身は`{` と`}` で括られる、などです。この節は Scala の制御構文を説明するので、処理の流れを制御するようなプログラムを作るためのルールが説明されるわけです。

次に「式（Expression）」は、プログラムを構成する部分のうち、評価が成功すると値になるものです。たとえば `1` や `1 + 2` 、`"hoge"` などです。これらは評価することにより、数値や文字列の値になります。評価が成功、という表現を使いましたが、評価の結果として例外が投げられた場合等が、評価が失敗した場合に当たります。

最後に「文（Statement）」ですが、式とは対照的にプログラムを構成する部分のうち、評価しても値にならないものです。たとえば変数の定義である `val i = 1` は評価しても変数 `i` が定義され、`i` の値が `1` になりますが、この定義全体としては値を持ちません。よって、これは文です。

Scala は C や Java などの手続き型の言語に比べて、文よりも式になる構文が多いです。Scala では文よりも式を多く利用する構文が採用されています。これにより変数などの状態を出来るだけ排除した分かりやすいコードが書きやすくなっています。

このような言葉の使われ方に注意し、以下の説明を読んでみてください。

7.2　ブロック式

Scala では {} で複数の式の並びを囲むと、それ全体が式になりますが、便宜上それをブロック式と呼ぶことにします。

ブロック式の一般形は

```
{ <式1>(;|<改行>) <式2>(;|<改行>) ... }
```

となります。式 の並びは、順番に評価される個々の式を表します。式が改行で区切られていればセミコロンは省略できます。{} 式は式1 , 式2 ... と式の並びを順番に評価し、最後の 式 を評価した値を返します。

次の式では

```
{ println("A"); println("B"); 1 + 2; }
// A
// B
// res0: Int = 3
```

A と B が出力され、最後の式である 1 + 2 の結果である 3 が{} 式の値になっていることがわかります。

このことは、後ほど記述するメソッド定義などにおいて重要になってきます。Scala では、

```
def foo(): String = {
  "foo" + "foo"
}
```

のような形でメソッド定義をすることが一般的ですが（後述します）、ここで{} は単に{} 式であって、メソッド定義の構文に{} が含まれているわけではありません。ただし、クラス定義構文などにおける{} は構文の一部です。

7.3　if 式

if 式は Java の if 文とほとんど同じ使い方をします。if 式の構文は次のようになります。

```
if '('<条件式>')' <then式> (else <else式>)?
```

Scala 3 では then キーワードを使用して以下のように書くこともできます。

```
if <条件式> then <then式> (else <else式>)?
```

条件式 は Boolean 型である必要があります。else <else式> は省略することができます。then式 は 条件式 が true のときに評価される式で、else式 は条件式 が false のときに評価される式です。

早速 if 式を使ってみましょう。

```
var age = 17
// age: Int = 17

if(age < 18) {
  "18?????"
} else {
  "18?????"
}
// res1: String = "18?????"

age = 18

if(age < 18) {
  "18?????"
} else {
  "18?????"
}
// res3: String = "18?????"
```

変更可能な変数 age が 18 より小さいかどうかで別の文字列を返すようにしています。

if 式に限らず、Scala の制御構文は全て式です。つまり必ず何らかの値を返します。Java などの言語で三項演算子?: を見たことがある人もいるかもしれませんが、Scala では同じように値が必要な場面で if 式を使います。

なお、else が省略可能だと書きましたが、その場合は、以下のように Unit 型の値 () が補われたのと同じ値が返ってきます。

```
if '(' <条件式> ')' <then式> else ()
```

ただし、Unit が補われたのと同等になるのは Scala 2 までの仕様であって、Scala 3 からは微妙に異なります。

Unit 型は Java では void に相当するもので、返すべき値がないときに使われ、唯一の値 () を持ちます。

7.3.1　練習問題

var age: Int = 5 という年齢を定義する変数と var isSchoolStarted: Boolean = false という就学を開始しているかどうかという変数を利用して、1 歳から 6 歳までの就学以前の子どもの場合に “幼児です” と出力し、それ以外の場合は “幼児ではありません” と出力するコードを書いてみましょう。

```
var age: Int = 5
// age: Int = 5
var isSchoolStarted: Boolean = false
// isSchoolStarted: Boolean = false
if(1 <= age && age <= 6 && !isSchoolStarted) {
  println("????")
```

```
} else {
  println("?????????")
}
// ????
```

7.4　while 式

while 式の構文は Java のものとほぼ同じです。

```
while '(' <条件式> ')' <本体式>
```

Scala 3 では do キーワードを使用して以下のように書くこともできます。

```
while <条件式> do <本体式>
```

条件式 は Boolean 型である必要があります。while 式は、条件式 が true の間、本体式 を評価し続けます。なお、while 式も式なので値を返しますが、while 式には適切な返すべき値がないので Unit 型の値 () を返します。

さて、while 式を使って 1 から 10 までの値を出力してみましょう。

```
var i = 1
// i: Int = 1

while(i <= 10) {
  println("i = " + i)
  i = i + 1
}
// i = 1
// i = 2
// i = 3
// i = 4
// i = 5
// i = 6
// i = 7
// i = 8
// i = 9
// i = 10
```

Java で while 文を使った場合と同様です。do while 式もありますが、Java と同様、かつ Scala 3 からは無くなったので説明は省略します。なお、Java の break 文や continue 文に相当する言語機能はありません。しかし、後ほど説明する高階関数を適切に利用すれば、ほとんどの場合、break や continue は必要ありません。

7.4.1　練習問題

while を利用して、0 から数え上げて 9 まで出力して 10 になったらループを終了するメソッド loopFrom0To9 を書いてみましょう。loopFrom0To9 は次のような形になります。??? の部分を埋めてください。

```
def loopFrom0To9(): Unit = {
  var i = ???
  while(???){
    ???
  }
}
```

```
def loopFrom0To9(): Unit = {
  var i = 0
  while(i < 10){
    println(i)
    i += 1
  }
}
```

```
loopFrom0To9()
```

7.5　return 式

return 式はメソッドから、途中で脱出してメソッドの呼び出し元に返り値を返すための制御構文です。

Scala では、メソッド定義の = の右は式であり、それを評価した値が返り値になるため、他の多くの言語と違い、return 式は必須ではありません。

一方で、特に手続き的にコードを書くときに return 式が便利なこともあります。以下は配列から、指定された要素を見つけてその添字を返すメソッドです。

```
def indexOf(array: Array[String], target: String): Int = {
  var index = -1
  var found = false
  var i = 0
  while(i < array.length && !found) {
    if(array(i) == target) {
      index = i
      found = true
    }
    i += 1
  }
  index
```

```
}
```

このメソッドでは、既に要素が見つかったかを found という変数で管理していますが、そのためにコードが冗長になっています。return 式を使えば、このコードは以下のように書き換えることができます。

```
def indexOf(array: Array[String], target: String): Int = {
  var i = 0
  while(i < array.length) {
    if(array(i) == target) return i
    i += 1
  }
  -1
}
```

見ての通り、不必要な変数が無くなって見通しがよくなりました。return 式を使えばコードの見通しがよくなることもある、ということを覚えておくと良いでしょう。

一方、従来の手続き型言語に親しんでいる人は、Scala では **return 式は必須ではない**（脱出の必要がなければ書かない）ということを念頭においてください。

7.6　for 式

Scala には for 式という制御構文があります。これは、Java の拡張 for 文と似た使い方ができるものの、ループ以外にも様々な応用範囲を持った制御構文です。for 式の本当の力を理解するには、flatMap , map , withFilter , foreach というメソッドについて知る必要がありますが、ここでは基本的な for 式の使い方のみを説明します。

for 式の基本的な構文は次のようになります。

```
for '(' (<ジェネレータ>;)+ ')' <本体式>
# <ジェネレータ> = x <- <式> (if <条件式>)?
```

Scala3 では do キーワードを使用して以下のように書くこともできます。

```
for (<ジェネレータ>;)+ do <本体式>
# <ジェネレータ> = x <- <式> (if <条件式>)?
```

各 ジェネレータ の変数 x に相当する部分は、好きな名前のループ変数を使うことができます。式 には色々な式が書けます。ただ、現状では全てを説明しきれないため、何かの数の範囲を表す式を使えると覚えておいてください。たとえば、1 to 10 は 1 から 10 まで（10 を含む）の範囲で、1 until 10 は 1 から 10 まで（10 を含まない）の範囲です。

それでは、早速 for 式を使ってみましょう。

```
for(x <- 1 to 5; y <- 1 until 5){
  println("x = " + x + " y = " + y)
}
```

```
// x = 1 y = 1
// x = 1 y = 2
// x = 1 y = 3
// x = 1 y = 4
// x = 2 y = 1
// x = 2 y = 2
// x = 2 y = 3
// x = 2 y = 4
// x = 3 y = 1
// x = 3 y = 2
// x = 3 y = 3
// x = 3 y = 4
// x = 4 y = 1
// x = 4 y = 2
// x = 4 y = 3
// x = 4 y = 4
// x = 5 y = 1
// x = 5 y = 2
// x = 5 y = 3
// x = 5 y = 4
```

x を 1 から 5 までループして、y を 1 から 4 までループして x , y の値を出力しています。ここでは、ジェネレータを 2 つだけにしましたが、数を増やせば何重にもループを行うことができます。

for 式の力はこれだけではありません。ループ変数の中から条件にあったものだけを絞り込むこともできます。until の後で if x != y と書いていますが、これは、x と y が異なる値の場合のみを抽出したものです。

```
for(x <- 1 to 5; y <- 1 until 5 if x != y){
  println("x = " + x + " y = " + y)
}
// x = 1 y = 2
// x = 1 y = 3
// x = 1 y = 4
// x = 2 y = 1
// x = 2 y = 3
// x = 2 y = 4
// x = 3 y = 1
// x = 3 y = 2
// x = 3 y = 4
// x = 4 y = 1
// x = 4 y = 2
// x = 4 y = 3
// x = 5 y = 1
// x = 5 y = 2
// x = 5 y = 3
// x = 5 y = 4
```

for 式はコレクションの要素を 1 つ 1 つたどって何かの処理を行うことにも利用することができます。"A" , "B" , "C" , "D" , "E" の 5 つの要素からなるリストをたどって全てを出力する処理を書いてみましょう。

```
for(e <- List("A", "B", "C", "D", "E")) println(e)
// A
// B
// C
// D
// E
```

さらに、for 式はたどった要素を加工して新しいコレクションを作ることもできます。先ほどのリストの要素全てに Pre という文字列を付加してみましょう。

```
for(e <- List("A", "B", "C", "D", "E")) yield {
  "Pre" + e
}
// res9: List[String] = List("PreA", "PreB", "PreC", "PreD", "PreE")
```

ここでポイントとなるのは、yield というキーワードです。実は、for 構文は yield キーワードを使うことで、コレクションの要素を加工して返すという全く異なる用途に使うことができます。特に yield キーワードを使った for 式を特別に for-comprehension と呼ぶことがあります。

7.6.1　練習問題

1 から 1000 までの 3 つの整数 a, b, c について、三辺からなる三角形が直角三角形になるような a , b , c の組み合わせを全て出力してください。直角三角形の条件にはピタゴラスの定理を利用してください。ピタゴラスの定理とは三平方の定理とも呼ばれ、a ^ 2 == b ^ 2 + c ^ 2 を満たす、a , b , c の長さの三辺を持つ 三角形は、直角三角形になるというものです。

```
for(a <- 1 to 1000; b <- 1 to 1000; c <- 1 to 1000 if a * a == b * b + c * c) {
  println((a, b, c))
}
```

7.7　match 式

match 式は Java の switch のように、複数の分岐を表現できる制御構造ですが、switch より様々なことができます。match 式の基本構文は

```
<対象式> match {
  (case <パターン> (if <ガード>)? '=>'
    (<式> (;|<改行>))*
  )+
}
```

のようになりますが、この「パターン」に書ける内容が非常に多岐に渡るためです。まず、Java の switch-case のような使い方をしてみます。たとえば、

```
val taro = "Taro"
// taro: String = "Taro"

taro match {
  case "Taro" => "Male"
  case "Jiro" => "Male"
  case "Hanako" => "Female"
}
// res10: String = "Male"
```

のようにして使うことができます。ここで、taro には文字列"Taro" が入っており、これは case "Taro" にマッチするため、"Male" が返されます。なお、ここで気づいた人もいるかと思いますが、match 式も値を返します。match 式の値は、マッチしたパターンの=> の右辺の式を評価したものになります。

パターンは文字列だけでなく数値など多様な値を扱うことができます。

```
val one = 1
// one: Int = 1

one match {
  case 1 => "one"
  case 2 => "two"
  case _ => "other"
}
// res11: String = "one"
```

ここで、パターンの箇所に_ が出てきましたが、これは switch-case の default に相当するもので、あらゆるものにマッチするパターンです。このパターンを**ワイルドカードパターン**と呼びます。match 式を使うときは、漏れがないようにするために、ワイルドカードパターンを使うことが多いです。

7.7.1　パターンをまとめる

Java や C などの言語で switch-case 文を学んだ方には、Scala のパターンマッチがいわゆるフォールスルー（fall through）の動作をしないことに違和感があるかもしれません。

```
"abc" match {
  case "abc" => println("first")   // ここで処理が終了
  case "def" => println("second") // こっちは表示されない
}
```

C 言語の switch-case 文のフォールスルー動作は利点よりバグを生み出すことが多いということで有名なものでした。Java が C 言語のフォールスルー動作を引き継いだことはしばしば非難されます。それで Scala のパターンマッチにはフォールスルー動作がないわけですが、複数のパターンをまとめたいときのために| があります

```
"abc" match {
```

```
  case "abc" | "def" =>
    println("first")
    println("second")
}
```

7.7.2　パターンマッチによる値の取り出し

switch-case 以外の使い方としては、コレクションの要素の一部にマッチさせる使い方があります。次のプログラムを見てみましょう。

```
val lst = List("A", "B", "C")
// lst: List[String] = List("A", "B", "C")

lst match {
  case List("A", b, c) =>
    println("b = " + b)
    println("c = " + c)
  case _ =>
    println("nothing")
}
// b = B
// c = C
```

ここでは、List の先頭要素が"A" で 3 要素のパターンにマッチすると、残りの b , c が List の 2 番目以降の要素に束縛されて、=> の右辺の式が評価されることになります。match 式では、特にコレクションの要素にマッチさせる使い方が頻出します。

パターンマッチではガード式を用いて、パターンにマッチして、かつ、ガード式（Boolean 型でなければならない）にもマッチしなければ右辺の式が評価されないような使い方もできます。

```
val lst = List("A", "B", "C")
// lst: List[String] = List("A", "B", "C")

lst match {
  case List("A", b, c) if b != "B" =>
    println("b = " + b)
    println("c = " + c)
  case _ =>
    println("nothing")
}
// nothing
```

ここでは、パターンマッチのガード条件に、List の 2 番目の要素が"B" でないこと、という条件を指定したため、最初の条件にマッチせず_ にマッチしたのです。

また、パターンマッチのパターンはネストが可能です。先ほどのプログラムを少し改変して、先頭が List("A") であるような List にマッチさせてみましょう。

```
val lst = List(List("A"), List("B", "C"))
```

```
// lst: List[List[String]] = List(List("A"), List("B", "C"))

lst match {
  case List(a@List("A"), x) =>
    println(a)
    println(x)
  case _ => println("nothing")
}
// List(A)
// List(B, C)
```

lst は List("A") と List("B", "C") の 2 要素からなる List です。ここで、match 式を使うことで、先頭が List("A") であるというネストしたパターンを記述できていることがわかります。また、パターンの前に@ がついているのは as パターンと呼ばれるもので、@ の後に続くパターンにマッチする式を@ の前の変数（ここでは a ）に束縛します。as パターンはパターンが複雑なときにパターンの一部だけを切り取りたい時に便利です。ただし| を使ったパターンマッチの場合は値を取り出すことができない点に注意してください。下記のように| のパターンマッチで変数を使った場合はコンパイルエラーになります。

```
(List("a"): Any) match {
  case List(a) | Some(a) =>
    println(a)
}
```

値を取り出さないパターンマッチは可能です。

```
(List("a"): Any) match {
  case List(_) | Some(_) =>
    println("ok")
}
```

7.7.3　中置パターンを使った値の取り出し

先の節で書いたようなパターンマッチを別の記法で書くことができます。たとえば、

```
val lst = List("A", "B", "C")
// lst: List[String] = List("A", "B", "C")

lst match {
  case List("A", b, c) =>
    println("b = " + b)
    println("c = " + c)
  case _ =>
    println("nothing")
}
// b = B
// c = C
```

というコードは、以下のように書き換えることができます。

```
val lst = List("A", "B", "C")
// lst: List[String] = List("A", "B", "C")

lst match {
  case "A" :: b :: c :: _ =>
    println("b = " + b)
    println("c = " + c)
  case _ =>
    println("nothing")
}
// b = B
// c = C
```

ここで、"A" :: b :: c :: _ のように、リストの要素の間にパターン名（::）が現れるようなものを中置パターンと呼びます。中置パターン（::）によってパターンマッチを行った場合、:: の前の要素がリストの最初の要素を、後ろの要素がリストの**残り全て**を指すことになります。リストの末尾を無視する場合、上記のようにパターンの最後に_ を挿入するといったことが必要になります。リストの中置パターンは Scala プログラミングでは頻出するので、このような機能があるのだということは念頭に置いてください。

7.7.4　型によるパターンマッチ

パターンとしては値が特定の型に所属する場合にのみマッチするパターンも使うことができます。値が特定の型に所属する場合にのみマッチするパターンは、名前:マッチする型 の形で使います。たとえば、以下のようにして使うことができます。なお、AnyRef 型は、Java の Object 型に相当する型で、あらゆる参照型の値を AnyRef 型の変数に格納することができます。

```
import java.util.Locale

val obj: AnyRef = "String Literal"
// obj: Object = "String Literal"

obj match {
  case v:java.lang.Integer =>
    println("Integer!")
  case v:String =>
    println(v.toUpperCase(Locale.ENGLISH))
}
// STRING LITERAL
```

java.lang.Integer にはマッチせず、String にマッチしていることがわかります。このパターンは例外処理や equals の定義などで使うことがあります。型でマッチした値は、その型にキャストしたのと同じように扱うことができます。

たとえば、上記の式で String 型にマッチした v は String 型のメソッドである toUpperCase を呼

びだすことができます。しばしば Scala ではキャストの代わりにパターンマッチが用いられるので覚えておくとよいでしょう。

7.7.5　JVM の制約による型のパターンマッチの落とし穴

型のパターンマッチで注意しなければならないことが 1 つあります。Scala を実行する JVM の制約により、型変数を使った場合、正しくパターンマッチがおこなわれません。

たとえば、以下の様なパターンマッチを REPL で実行しようとすると、警告が出てしまいます。

```
val obj: Any = List("a")
// obj: Any = List("a")
obj match {
  case v: List[Int]    => println("List[Int]")
  case v: List[String] => println("List[String]")
}
// List[Int]
```

型としては `List[Int]` と `List[String]` は違う型なのですが、パターンマッチではこれを区別できません。

最初の 2 つの警告の意味は Scala コンパイラの「型消去」という動作により `List[Int]` の `Int` の部分が消されてしまうのでチェックされないということです。

結果的に 2 つのパターンは区別できないものになり、パターンマッチは上から順番に実行されていくので、2 番目のパターンは到達しないコードになります。3 番目の警告はこれを意味しています。

型変数を含む型のパターンマッチは、以下のようにワイルドカードパターンを使うと良いでしょう。

```
obj match {
  case v: List[?] => println("List[?]")
}
```

7.7.6　練習問題

```
new scala.util.Random(new java.security.SecureRandom()).alphanumeric.take(5).toList
```

以上のコードを利用して、最初と最後の文字が同じ英数字であるランダムな 5 文字の文字列を 1000 回出力してください。`new scala.util.Random(new java.security.SecureRandom()).alphanumeric.take(5).toList` という値は、呼びだす度にランダムな 5 個の文字（`Char` 型）のリストを与えます。なお、以上のコードで生成されたリストの一部分を利用するだけでよく、最初と最後の文字が同じ英数字であるリストになるまで試行を続ける必要はありません。これは、`List(a, b, d, e, f)` が得られた場合に、`List(a, b, d, e, a)` のようにしても良いということです。

```
for(i <- 1 to 1000) {
  val s = new scala.util.Random(new java.security.SecureRandom()).alphanumeric.take(5).
    toList match {
```

```
3     case List(a,b,c,d,_) => List(a,b,c,d,a).mkString
4   }
5   println(s)
6 }
```

match 式は switch-case に比べてかなり強力であることがわかると思います。ですが、match 式の力はそれにとどまりません。後述しますが、パターンには自分で作ったクラス（のオブジェクト）を指定することでさらに強力になります。

第 8 章

クラス

これから Scala におけるクラス定義に関して説明します。他のオブジェクト指向言語のクラスをある程度知っていることを前提としていますが、ご了承ください。

8.1 クラス定義

Scala のクラス定義は次のような形を取ります。

```
class <クラス名> '(' (<引数名1> : <引数型1>, <引数名2>: <引数型2> ...)? ')' {
  (<フィールド定義> | <メソッド定義> )*
}
```

たとえば、点を表すクラス Point を定義したいとします。Point は x 座標を表すフィールド x（Int 型）とフィールド y（Int 型）からなるとします。このクラス Point を Scala で書くと次のようになります。

```
class Point(_x: Int, _y: Int) {
  val x = _x
  val y = _y
}
```

コンストラクタの引数と同名のフィールドを定義し、それを公開する場合は、以下のように短く書くこともできます。

```
class Point(val x: Int, val y: Int)
```

- クラス名の直後にコンストラクタ引数の定義がある
- val/var によって、コンストラクタ引数をフィールドとして公開することができる

点に注目してください。最初の点ですが、Scala では 1 クラスに付き、基本的には 1 つのコンストラクタしか使いません。このコンストラクタを、Scala では**プライマリコンストラクタ**として特別に

扱っています。文法上は複数のコンストラクタを定義できるようになっていますが、実際に使うことはほとんどありません。複数のオブジェクトの生成方法を提供したい場合、object の apply メソッドとして定義することが多いです。

2 番目の点ですが、プライマリコンストラクタの引数に val/var をつけるとそのフィールドは公開され、外部からアクセスできるようになります。なお、プライマリコンストラクタの引数のスコープはクラス定義全体におよびます。そのため、以下のようにメソッド定義の中から直接コンストラクタ引数を参照できます。

```
class Point(val x: Int, val y: Int) {
  def +(p: Point): Point = {
    new Point(x + p.x, y + p.y)
  }
  override def toString(): String = "(" + x + ", " + y + ")"
}
```

8.2　メソッド定義

先ほど既にメソッド定義の例として + メソッドの定義が出てきましたが、一般的には、次のような形をとります。

```
(private([<パッケージ名など>])? | protected([<パッケージ名など>])?)? def <メソッド名> '
    ('
  (<引数名> : 引数型 (, 引数名 : <引数型>)*)?
')': <返り値型> = <本体>
```

実際にはブロック式を使った以下の形式を取ることが多いでしょう。

```
(private([<パッケージ名など>])? | protected([<パッケージ名など>])?)? def <メソッド名> '
    ('
  (<引数名> : 引数型 (, 引数名 : <引数型>)*)?
')': <返り値型> = {
  (<式> (; | <改行>)?)*
}
```

単にメソッド本体がブロック式からなる場合にこうなるというだけであって、メソッド定義を{} で囲む専用の構文があるわけではありません。

返り値型は省略しても特別な場合以外型推論してくれますが、読みやすさのために、返り値の型は明記する習慣を付けるようにしましょう。private を付けるとそのクラス内だけから、protected を付けると派生クラスからのみアクセスできるメソッドになります。private[this] をつけると、同じオブジェクトからのみアクセス可能、という機能が Scala 2 にはありましたが、Scala 3 からは [this] の記法は非推奨になりました。private[パッケージ名] を付けると同一パッケージに所属しているものからのみ、protected[パッケージ名] をつけると、派生クラスに加えて追加で同じパッケージに所属しているもの全てからアクセスできるようになります。private も protected も付けない場合、そのメソッドは public とみなされます。

先ほど定義した Point クラスを REPL から使ってみましょう。

```
class Point(val x: Int, val y: Int) {
  def +(p: Point): Point = {
    new Point(x + p.x, y + p.y)
  }
  override def toString(): String = "(" + x + ", " + y + ")"
}

val p1 = new Point(1, 1)
// p1: Point = (1, 1)

val p2 = new Point(2, 2)
// p2: Point = (2, 2)

p1 + p2
// res0: Point = (3, 3)
```

8.3　複数の引数リストを持つメソッド

メソッドは以下のように複数の引数リストを持つように定義することができます。

```
(private([<パッケージ名など>])? | protected([<パッケージ名>])?)? def <メソッド名> '('
  (<引数名> : 引数型 (, 引数名 : <引数型>)*)?
')'( '('
  (<引数名> : 引数型 (, 引数名 : <引数型>)*)?
')' )* : <返り値型> = <本体式>
```

複数の引数リストを持つメソッドには、Scala の糖衣構文と組み合わせて流暢な API を作ったり、後述する implicit parameter のために必要になったり、型推論を補助するために使われたりといった用途があります。複数の引数リストを持つ加算メソッドを定義してみましょう。

```
class Adder {
  def add(x: Int)(y: Int): Int = x + y
}

val adder = new Adder()
// adder: Adder = repl.MdocSession$MdocApp$Adder$1@3ba332f6

adder.add(2)(3)
// res1: Int = 5

// ??Scala??????????????????? _ ????????
// ???Scala?????????? _ ?????????
val funOldSyntax: Int => Int = adder.add(2) _
// funOldSyntax: Function1[Int, Int] = repl.MdocSession$MdocApp$$Lambda$13766/0
    x0000743a8df39c00@781266

val fun: Int => Int = adder.add(2)
```

```
// fun: Function1[Int, Int] = repl.MdocSession$MdocApp$$Lambda$13767/0x0000743a8df3a000@35b2418c
fun(3)
// res2: Int = 5
```

複数の引数リストを持つメソッドは obj.method(x, y) の形式でなく obj.method(x)(y) の形式で呼びだすことになります。また、一番下の例のように最初の引数だけを適用して新しい関数を作る（部分適用）こともできます。

次のように、複数の引数リストを使わずに単に複数の引数を持つメソッドも作ることができます。

```
class Adder {
  def add(x: Int, y: Int): Int = x + y
}

val adder = new Adder()
// adder: Adder = repl.MdocSession$MdocApp$Adder$2@7b2064f8

adder.add(2, 3)
// res3: Int = 5

val fun: Int => Int = adder.add(2, _)
// fun: Function1[Int, Int] = repl.MdocSession$MdocApp$$Lambda$13768/0x0000743a8df3ae80@7028674a
fun(3)
// res4: Int = 5
```

8.4　フィールド定義

フィールド定義は

```
(private([<パッケージ名など>])? | protected([<パッケージ名など>])?)? (val | var) <フィールド名>: <フィールド型> = <初期化式>
```

という形を取ります。val の場合は変更不能、var の場合は変更可能なフィールドになります。また、private を付けるとそのクラス内だけから、protected を付けるとそのクラスの派生クラスからのみアクセスできるフィールドになります。private[this] を付加すると、同じオブジェクトからのみアクセス可能になる機能が Scala 2 にはありましたが、Scala 3 からはこの記法は非推奨になりました。さらに、private[<パッケージ名>] を付けると同一パッケージからのみ、protected[パッケージ名] をつけると、派生クラスに加えて同じパッケージに所属しているもの全てからアクセスできるようになります。private も protected も付けない場合、そのフィールドは public とみなされます。

8.5　抽象メンバー

その時点では実装を書くことができず、後述する継承の際に、メソッドやフィールドの実装を与えたいということがあります。このような場合に対応するため、Scala では抽象メンバーを定義するこ

とができます。抽象メンバーは、メソッドの場合とフィールドの場合があり、メソッドの場合は次のようになります。

```
(protected([<パッケージ名>])?)? def <メソッド名> '('
  (<引数名> : 引数型 (, 引数名 : <引数型>)*)?
')': <返り値型>
```

フィールドの定義は次のようになります。

```
(protected([<パッケージ名>])?)? (val | var) <フィールド名>: <フィールド型>
```

メソッドやフィールドの中身がない以外は、通常のメソッドやフィールド定義と同じです。また、抽象メソッドを一個以上持つクラスは、抽象クラスとして宣言する必要があります。たとえば、x 座標と y 座標を持つ、抽象クラス XY は次のようにして定義します。クラスの前に abstract 修飾子をつける必要があるのがポイントです。

```
abstract class XY {
  def x: Int
  def y: Int
}
```

8.6　継承

Scala のクラスは、他の多くのオブジェクト指向言語と同様に、継承することができます。継承には 2 つの目的があります。1 つは継承によりスーパークラスの実装をサブクラスでも使うことで実装を再利用することです。もう 1 つは複数のサブクラスが共通のスーパークラスのインタフェースを継承することで処理を共通化することです[*1]。実装の継承には複数の継承によりメソッドやフィールドの名前が衝突する場合の振舞いなどに問題があることが知られており、例えば、Java では実装継承が 1 つだけに限定されています。Scala ではトレイトという仕組みで複数の実装の継承を実現していますが、トレイトについては別の節で説明します。

ここでは通常の Scala のクラスの継承について説明します。Scala でのクラスの継承は次のような構文になります。

```
class <クラス名> <クラス引数> (extends <スーパークラス>)? (with <トレイト名>)* {
  (<フィールド定義> | <メソッド定義>)*
}
```

※「トレイト名」はここでは使われませんが、後で出てくるトレイトの節で説明を行います。

Scala では、既に実装があるメソッドをオーバーライドするときは override キーワードを使わなければなりません。たとえば、次のように書くことは可能ですが

[*1] このように継承などにより型に親子関係を作り、複数の型に共通のインタフェースを持たせることをサブタイピング・ポリモーフィズムと呼びます。Scala では他にも構造的部分型というサブタイピング・ポリモーフィズムの機能がありますが、実際に使われることが少ないため、このテキストでは説明を省略しています。

```
class APrinter() {
  def print(): Unit = {
    println("A")
  }
}

class BPrinter() extends APrinter {
  override def print(): Unit = {
    println("B")
  }
}

new APrinter().print()
// A

new BPrinter().print()
// B
```

ここで BPrinter#print() の override キーワードをはずすと、次のようにメッセージを出力して、**コンパイルエラー**になります。

```
class BPrinter() extends APrinter {
  def print(): Unit = {
    println("B")
  }
}
```

```
[error] .../Printer.scala:8:7: `override` modifier required to override concrete member
    :
[error] def print(): Unit (defined in class APrinter)
[error]   def print(): Unit = {
[error]       ^
[error] one error found
```

このような仕組みのない言語ではしばしば、気付かずに既存のメソッドをオーバーライドするつもりで新しいメソッドを定義してしまうというミスがありますが、Scala では override キーワードを使って言語レベルでこの問題に対処しているのです。

8.6.1　練習問題

全てが Int 型の x 、y 、z という名前を持った、3 次元座標を表す Point3D クラスを定義してください。Point3D クラスは次のようにして使うことができなければいけません。

```
val p = new Point3D(10, 20, 30)
println(p.x) // 10
println(p.y) // 20
println(p.z) // 30
```

```
class Point3D(val x: Int, val y: Int, val z: Int)
```

8.6.1.1　解説

プライマリコンストラクタの引数として座標の値を渡し、それをそのまま取り出しているので、プライマリコンストラクタの引数に val を付けるのが最も簡単です。別解として、以下のように別途 val でフィールドを定義することも可能ですが、今回あえてそうする意味は少ないでしょう。

```
class Point3D(x_ : Int, y_ : Int, z_ : Int) {
  val x: Int = x_
  val y: Int = y_
  val z: Int = z_
}
```

8.6.2　練習問題

次の抽象クラス Shape を継承して、Rectangle クラス（長方形クラス）と Circle クラス（円クラス）を定義してください。また、area メソッドをオーバーライドして、ただしく面積が計算できるように定義してください。なお、長方形の面積は幅を w 、高さを h とすると、w * h で求めることができます。円の面積は、半径を r とすると、r * r * math.Pi で求めることができます。

```
abstract class Shape {
  def area: Double
}
/*
 * RectangleとCircleの定義
 */
var shape: Shape = new Rectangle(10.0, 20.0)
println(shape.area)
shape = new Circle(2.0)
println(shape.area)
```

```
abstract class Shape {
  def area: Double
}
class Rectangle(val width: Double, val height: Double) extends Shape {
  override def area: Double = width * height
}
class Circle(val radius: Double) extends Shape {
  override def area: Double = radius * radius * math.Pi
}
```

8.6.2.1　解説

`Rectangle` と `Circle` クラスが `Shape` クラスを継承して、それぞれで `area: Double` メソッドをオーバーライドしています。この場合、`Shape` の `area` は抽象メソッドなので `override` は必須ではありませんが、つけた方が良いでしょう。

第 9 章

オブジェクト

Scala では、全ての値がオブジェクトです。また、全てのメソッドは何らかのオブジェクトに所属しています。そのため、Java のようにクラスに属する static フィールドや static メソッドといったものを作成することができません。その代わりに、`object` キーワードによって、同じ名前のシングルトンオブジェクトを現在の名前空間の下に 1 つ定義することができます。`object` キーワードによって定義したシングルトンオブジェクトには、そのオブジェクト固有のメソッドやフィールドを定義することができます。

`object` 構文の主な用途としては、

- ユーティリティメソッドやグローバルな状態の置き場所（Java で言う static メソッドやフィールド）
- 同名クラスのオブジェクトのファクトリメソッド

が挙げられます。

object の基本構文はクラスとおおむね同じで、

```
object <オブジェクト名> extends <クラス名> (with <トレイト名>)* {
  (<フィールド定義> | <メソッド定義>)*
}
```

となります。Scala では標準で `Predef` という `object` が定義・インポートされており、これは最初の使い方に当てはまります。`println("Hello")` となにげなく使っていたメソッドも実は `Predef` のメソッドなのです。`extends` でクラスを継承、`with` でトレイトを mix-in 可能になっているのは、オブジェクト名を既存のクラスのサブクラス等として振る舞わせたい場合があるからです。Scala の標準ライブラリでは、`Nil` という `object` がありますが、これは `List` の一種として振る舞わせたいため、`List` を継承しています。一方、`object` がトレイトを mix-in する事はあまり多くありませんが、クラスやトレイトとの構文の互換性のためにそうなっていると思われます。

一方、2 番めの使い方について考えてみます。点を表す `Point` クラスのファクトリを `object` で作

ろうとすると、次のようになります。apply という名前のメソッドは Scala 処理系によって特別に扱われ、Point(x) のような記述があった場合で、Point object に apply という名前のメソッドが定義されていた場合、Point.apply(x) と解釈されます。これを利用して Point object の apply メソッドでオブジェクトを生成するようにすることで、Point(3, 5) のような記述でオブジェクトを生成できるようになります。

```
class Point(val x:Int, val y:Int)

object Point {
  def apply(x: Int, y: Int): Point = new Point(x, y)
}
```

これは、new Point() で直接 Point オブジェクトを生成するのに比べて、

- クラス（Point）の実装詳細を内部に隠しておける（インタフェースのみを外部に公開する）
- Point ではなく、そのサブクラスのインスタンスを返すことができる

といったメリットがあります。なお、上記の記述はケースクラスを用いてもっと簡単に

```
case class Point(x: Int, y: Int)
```

と書けます。ケースクラスは後述するパターンマッチのところでも出てきますが、ここではその使い方については触れません。簡単に言うとケースクラスは、それをつけたクラスのプライマリコンストラクタ全てのフィールドを公開し、equals()・hashCode()・toString() などのオブジェクトの基本的なメソッドをオーバーライドしたクラスを生成し、また、そのクラスのインスタンスを生成するためのファクトリメソッドを生成するものです。たとえば、case class Point(x: Int, y: Int) で定義した Point クラスは equals() メソッドを明示的に定義してはいませんが、

```
Point(1, 2).equals(Point(1, 2))
```

を評価した値は true になります。

9.1　コンパニオンオブジェクト

クラスと同じファイル内、同じ名前で定義されたシングルトンオブジェクトは、コンパニオンオブジェクトと呼ばれます。コンパニオンオブジェクトは対応するクラスに対して特権的なアクセス権を持っています。たとえば、weight を private にした場合、

```
class Person(name: String, age: Int, private val weight: Int)

object Hoge {
  def printWeight(): Unit = {
    val taro = new Person("Taro", 20, 70)
    println(taro.weight)
  }
```

```
}
```

は NG ですが、

```
class Person(name: String, age: Int, private val weight: Int)

object Person {
  def printWeight(): Unit = {
    val taro = new Person("Taro", 20, 70)
    println(taro.weight)
  }
}
```

は OK です。private とした場合、コンパニオンオブジェクトからはアクセス可能です。

上記のような、コンパニオンオブジェクトを使ったコードを REPL で試す場合は、REPL の:paste コマンドを使って、クラスとコンパニオンオブジェクトを一緒にペーストするようにしてください。クラスとコンパニオンオブジェクトは同一ファイル中に置かれていなければならないのですが、REPL で両者を別々に入力した場合、コンパニオン関係を REPL が正しく認識できないのです。

```
scala> :paste
// Entering paste mode (ctrl-D to finish)

class Person(name: String, age: Int, private val weight: Int)

object Person {
  def printWeight(): Unit = {
    val taro = new Person("Taro", 20, 70)
    println(taro.weight)
  }
}

// Exiting paste mode, now interpreting.

defined class Person
defined object Person
```

9.1.1　練習問題

クラスを定義して、そのクラスのコンパニオンオブジェクトを定義してみましょう。コンパニオンオブジェクトが同名のクラスに対する特権的なアクセス権を持っていることを、クラスのフィールドを private にして、そのフィールドへアクセスできることを通じて確認してみましょう。

第 10 章 トレイト

私たちの作るプログラムはしばしば数万行、多くなると数十万行やそれ以上に及ぶことがあります。その全てを一度に把握することは難しいので、プログラムを意味のあるわかりやすい単位で分割しなければなりません。さらに、その分割された部品はなるべく柔軟に組み立てられ、大きなプログラムを作れると良いでしょう。

プログラムの分割（モジュール化）と組み立て（合成）は、オブジェクト指向プログラミングでも関数型プログラミングにおいても重要な設計の概念になります。そして、Scala のオブジェクト指向プログラミングにおけるモジュール化の中心的な概念になるのがトレイトです。

この節では Scala のトレイトの機能を一通り見ていきましょう。

10.1 トレイト定義

Scala 2 のトレイトは、クラスからコンストラクタを定義する機能を抜いたようなもので、おおまかには次のように定義することができます。

```
1 trait <トレイト名> {
2   (<フィールド定義> | <メソッド定義>)*
3 }
```

フィールド定義とメソッド定義は本体がなくても構いません。トレイト名で指定した名前がトレイトとして定義されます。

10.2 トレイトの基本

Scala のトレイトはクラスに比べて以下のような特徴があります。

- 複数のトレイトを 1 つのクラスやトレイトにミックスインできる
- 直接インスタンス化できない

- クラスパラメータ（コンストラクタの引数）を取ることができない

以下、それぞれの特徴の紹介をしていきます。

10.2.1　複数のトレイトを 1 つのクラスやトレイトにミックスインできる

Scala のトレイトはクラスとは違い、複数のトレイトを 1 つのクラスやトレイトにミックスインすることができます。

```
trait TraitA

trait TraitB

class ClassA

class ClassB

// コンパイルできる
class ClassC extends ClassA with TraitA with TraitB
```

```
// コンパイルエラー！
class ClassD extends ClassA with ClassB
```

上記の例では ClassA と TraitA と TraitB を継承した ClassC を作ることはできますが ClassA と ClassB を継承した ClassD は作ることができません。「class ClassB needs to be a trait to be mixed in」というエラーメッセージが出ますが、これは「ClassB をミックスインさせるためにはトレイトにする必要がある」という意味です。複数のクラスを継承させたい場合はクラスをトレイトにしましょう。

10.2.2　直接インスタンス化できない

Scala のトレイトはクラスと違い、直接インスタンス化できません。

```
trait TraitA
```

```
object ObjectA {
  // コンパイルエラー！
  val a = new TraitA
}
```

これは、トレイトが単体で使われることをそもそも想定していないための制限です。トレイトを使うときは、通常、それを継承したクラスを作ります。

```
trait TraitA

class ClassA extends TraitA
```

```
4
5  object ObjectA {
6    // クラスにすればインスタンス化できる
7    val a = new ClassA
8
9  }
```

なお、new Trait {} という記法を使うと、一見トレイトをインスタンス化できているように見えます。しかしこれは、Trait を継承した無名のクラスを作って、そのインスタンスを生成する構文なので、トレイトそのものをインスタンス化できているわけではありません。

10.2.3　クラスパラメータ（コンストラクタの引数）を取ることができない

Scala 2 のトレイトはクラスと違いパラメータ（コンストラクタの引数）を取ることができないという制限があります[*1]。

```
1  // 正しいプログラム
2  class ClassA(name: String) {
3    def printName() = println(name)
4  }
```

```
1  // コンパイルエラー！
2  trait TraitA(name: String)
```

これもあまり問題になることはありません。トレイトに抽象メンバーを持たせることで値を渡すことができます。インスタンス化できない問題のときと同じようにクラスに継承させたり、インスタンス化のときに抽象メンバーを実装をすることでトレイトに値を渡すことができます。

```
1  trait TraitA {
2    val name: String
3    def printName(): Unit = println(name)
4  }
5
6  // クラスにして name を上書きする
7  class ClassA(val name: String) extends TraitA
8
9  object ObjectA {
10   val a = new ClassA("dwango")
11
12   // name を上書きするような実装を与えてもよい
13   val a2 = new TraitA { val name = "kadokawa" }
14 }
```

以上のようにトレイトの制限は実用上ほとんど問題にならないようなものであり、その他の点ではクラスと同じように使うことができます。つまり実質的に多重継承と同じようなことができるわけで

[*1] Scala 3 では、トレイトがパラメータを取ることができます。

す。そしてトレイトのミックスインはモジュラリティに大きな恩恵をもたらします。是非使いこなせるようになりましょう。

10.2.4　「トレイト」という用語について

この節では、トレイトやミックスインなどオブジェクトの指向の用語が用いられますが、他の言語などで用いられる用語とは少し違う意味を持つかもしれないので、注意が必要です。

トレイトは Schärli らによる 2003 年の ECOOP に採択された論文『Traits: Composable Units of Behaviour』がオリジナルとされていますが、この論文中のトレイトの定義と Scala のトレイトの仕様は、合成時の動作や、状態変数の取り扱いなどについて、異なっているように見えます。

しかし、トレイトやミックスインという用語は言語によって異なるものであり、我々が参照している Scala の公式ドキュメントや『Scala スケーラブルプログラミング』でも「トレイトをミックスインする」という表現が使われていますので、ここではそれに倣いたいと思います。

10.3　トレイトの様々な機能

10.3.1　菱形継承問題

以上見てきたようにトレイトはクラスに近い機能を持ちながら実質的な多重継承が可能であるという便利なものなのですが、1 つ考えなければならないことがあります。多重継承を持つプログラミング言語が直面する「菱形継承問題」というものです。

以下のような継承関係を考えてみましょう。`greet` メソッドを定義した `TraitA` と、`greet` を実装した `TraitB` と `TraitC`、そして `TraitB` と `TraitC` のどちらも継承した `ClassA` です。

```
trait TraitA {
  def greet(): Unit
}

trait TraitB extends TraitA {
  def greet(): Unit = println("Good morning!")
}

trait TraitC extends TraitA {
  def greet(): Unit = println("Good evening!")
}
```

```
class ClassA extends TraitB with TraitC
```

`TraitB` と `TraitC` の `greet` メソッドの実装が衝突しています。この場合 `ClassA` の `greet` はどのような動作をすべきなのでしょうか？ `TraitB` の `greet` メソッドを実行すべきなのか、`TraitC` の `greet` メソッドを実行すべきなのか。多重継承をサポートする言語はどれもこのようなあいまいさの問題を抱えており、対処が求められます。

ちなみに、上記の例を Scala でコンパイルすると以下のようなエラーが出ます。

```
scala> class ClassA extends TraitB with TraitC
<console>:13: error: class ClassA inherits conflicting members:
  method greet in trait TraitB of type ()Unit  and
  method greet in trait TraitC of type ()Unit
(Note: this can be resolved by declaring an override in class ClassA.)
       class ClassA extends TraitB with TraitC
             ^
```

Scala では override 指定なしの場合メソッド定義の衝突はエラーになります。

この場合の 1 つの解法は、コンパイルエラーに「Note: this can be resolved by declaring an override in class ClassA.」とあるように `ClassA` で `greet` を override することです。

```
class ClassA extends TraitB with TraitC {
  override def greet(): Unit = println("How are you?")
}
```

このとき `ClassA` で `super` に型を指定してメソッドを呼びだすことで、`TraitB` や `TraitC` のメソッドを指定して使うこともできます。

```
class ClassB extends TraitB with TraitC {
  override def greet(): Unit = super[TraitB].greet()
}
```

実行結果は以下にようになります。

```
(new ClassA).greet()
// How are you?

(new ClassB).greet()
// Good morning!
```

では、`TraitB` と `TraitC` の両方のメソッドを呼び出したい場合はどうでしょうか？ 1 つの方法は上記と同じように `TraitB` と `TraitC` の両方のクラスを明示して呼びだすことです。

```
class ClassA extends TraitB with TraitC {
  override def greet(): Unit = {
    super[TraitB].greet()
    super[TraitC].greet()
  }
}
```

しかし、継承関係が複雑になった場合にすべてを明示的に呼ぶのは大変です。また、コンストラクタのように必ず呼び出されるメソッドもあります。

Scala のトレイトにはこの問題を解決するために「線形化（linearization）」という機能があります。

10.3.2　線形化（linearization）

Scala のトレイトの線形化機能とは、トレイトがミックスインされた順番をトレイトの継承順番と見做すことです。

次に以下の例を考えてみます。先程の例との違いは TraitB と TraitC の greet メソッド定義に override 修飾子が付いていることです。

```
trait TraitA {
  def greet(): Unit
}

trait TraitB extends TraitA {
  override def greet(): Unit = println("Good morning!")
}

trait TraitC extends TraitA {
  override def greet(): Unit = println("Good evening!")
}

class ClassA extends TraitB with TraitC
```

この場合はコンパイルエラーにはなりません。では ClassA の greet メソッドを呼び出した場合、いったい何が表示されるのでしょうか？ 実際に実行してみましょう。

```
(new ClassA).greet()
// Good evening!
```

ClassA の greet メソッドの呼び出しで、TraitC の greet メソッドが実行されました。これはトレイトの継承順番が線形化されて、後からミックスインした TraitC が優先されているためです。つまりトレイトのミックスインの順番を逆にすると TraitB が優先されるようになります。以下のようにミックスインの順番を変えてみます。

```
class ClassB extends TraitC with TraitB
```

すると ClassB の greet メソッドの呼び出しで、今度は TraitB の greet メソッドが実行されます。

```
scala> (new ClassB).greet()
Good morning!
```

super を使うことで線形化された親トレイトを使うこともできます

```
trait TraitA {
  def greet(): Unit = println("Hello!")
}

trait TraitB extends TraitA {
  override def greet(): Unit = {
    super.greet()
```

```
    println("My name is Terebi-chan.")
  }
}

trait TraitC extends TraitA {
  override def greet(): Unit = {
    super.greet()
    println("I like niconico.")
  }
}

class ClassA extends TraitB with TraitC
class ClassB extends TraitC with TraitB
```

この greet メソッドの結果もまた継承された順番で変わります。

```
(new ClassA).greet()
// Hello!
// My name is Terebi-chan.
// I like niconico.

(new ClassB).greet()
// Hello!
// I like niconico.
// My name is Terebi-chan.
```

線形化の機能によりミックスインされたすべてのトレイトの処理を簡単に呼び出せるようになりました。このような線形化によるトレイトの積み重ねの処理を Scala の用語では積み重ね可能なトレイト（Stackable Trait）と呼ぶことがあります。

この線形化が Scala の菱形継承問題に対する対処法になるわけです。

10.3.3　落とし穴：トレイトの初期化順序

Scala のトレイトの val の初期化順序はトレイトを使う上で大きな落とし穴になります。以下のような例を考えてみましょう。トレイト A で変数 foo を宣言し、トレイト B が foo を使って変数 bar を作成し、クラス C で foo に値を代入してから bar を使っています。

```
trait A {
  val foo: String
}

trait B extends A {
  val bar = foo + "World"
}

class C extends B {
  val foo = "Hello"

  def printBar(): Unit = println(bar)
```

```
}
```

REPL でクラス C の printBar メソッドを呼び出してみましょう。

```
(new C).printBar()
// nullWorld
```

nullWorld と表示されてしまいました。クラス C で foo に代入した値が反映されていないようです。どうしてこのようなことが起きるかというと、Scala のクラスおよびトレイトはスーパークラスから順番に初期化されるからです。この例で言えば、クラス C はトレイト B を継承し、トレイト B はトレイト A を継承しています。つまり初期化はトレイト A が一番先におこなわれ、変数 foo が宣言され、中身は何も代入されていないので、null になります。次にトレイト B で変数 bar が宣言され null である foo と”World” という文字列から"nullWorld" という文字列が作られ、変数 bar に代入されます。先ほど表示された文字列はこれになります。

10.3.4　トレイトの val の初期化順序の回避方法

では、この罠はどうやれば回避できるのでしょうか。上記の例で言えば、使う前にちゃんと foo が初期化されるように、bar の初期化を遅延させることです。処理を遅延させるには lazy val か def を使います。

具体的なコードを見てみましょう。

```
trait A {
  val foo: String
}

trait B extends A {
  lazy val bar = foo + "World" // もしくは def bar でもよい
}

class C extends B {
  val foo = "Hello"

  def printBar(): Unit = println(bar)
}
```

先ほどの nullWorld が表示されてしまった例と違い、bar の初期化に lazy val が使われるようになりました。これにより bar の初期化が実際に使われるまで遅延されることになります。その間にクラス C で foo が初期化されることにより、初期化前の foo が使われることがなくなるわけです。

今度はクラス C の printBar メソッドを呼び出してもちゃんと HelloWorld と表示されます。

```
(new C).printBar()
// HelloWorld
```

lazy val は val に比べて若干処理が重く、複雑な呼び出しでデッドロックが発生する場合があります。val のかわりに def を使うと毎回値を計算してしまうという問題があります。しかし、両方とも大きな問題にならない場合が多いので、特に val の値を使って val の値を作り出すような場合は lazy val か def を使うことを検討しましょう。

第 11 章

型パラメータ（type parameter）

クラスの節では触れませんでしたが、クラスは 0 個以上の型をパラメータとして取ることができます。これは、クラスを作る時点では何の型か特定できない場合（たとえば、コレクションクラスの要素の型）を表したい時に役に立ちます。型パラメータを入れたクラス定義の文法は次のようになります。

```
class <クラス名>[<型パラメータ1>, <型パラメータ2>, ...](<クラス引数>) {
  (<フィールド定義>|<メソッド定義>)*
}
```

型パラメータの並びにはそれぞれ好きな名前を付け、クラス定義の中で使うことができます。Scala 言語では最初から順に、`A`、`B`、...と命名する慣習がありますので、それに合わせるのが無難です。

簡単な例として、1 個の要素を保持して、要素を入れる（put する）か取りだす（get する）操作ができるクラス `Cell` を定義してみます。`Cell` の定義は次のようになります。

```
class Cell[A](var value: A) {
  def put(newValue: A): Unit = {
    value = newValue
  }

  def get(): A = value
}
```

これを REPL で使ってみましょう。

```
scala> class Cell[A](var value: A) {
     |   def put(newValue: A): Unit = {
     |     value = newValue
     |   }
     |
     |   def get(): A = value
     | }
defined class Cell

```

```
scala> val cell = new Cell[Int](1)
cell: Cell[Int] = Cell@192aaffb

scala> cell.put(2)

scala> cell.get()
res1: Int = 2

scala> cell.put("something")
<console>:10: error: type mismatch;
 found   : String("something")
 required: Int
              cell.put("something")
                       ^
```

上記コードの

```
val cell = new Cell[Int](1)
// cell: Cell[Int] = repl.MdocSession$MdocApp$Cell$1@70fe0d97
```

の部分で、型パラメータとして Int 型を与えて、その初期値として 1 を与えています。型パラメータに Int を与えて Cell をインスタンス化したため、REPL では String を put しようとして、コンパイラにエラーとしてはじかれています。Cell は様々な型を与えてインスタンス化したいクラスであるため、クラス定義時には特定の型を与えることができません。そういった場合に、型パラメータは役に立ちます。

次に、もう少し実用的な例をみてみましょう。メソッドから複数の値を返したい、という要求はプログラミングを行う上でよく発生します。複数の値を返す言語サポート（Scheme や Go 等の言語が持っています）も型パラメータも無い言語では、

- 片方を返り値として、もう片方を引数を経由して返す
- 複数の返り値専用のクラスを必要になる度に作る

という選択肢しかありませんでした。しかし、前者は引数を返り値に使うという点で邪道ですし、後者の方法は多数の引数を返したい、あるいは解く問題上で意味のある名前の付けられるクラスであれば良いですが、ただ 2 つの値を返したいといった場合には小回りが効かず不便です。こういう場合、型パラメータを 2 つ取る Pair クラスを作ってしまいます。Pair クラスの定義は次のようになります。toString メソッドの定義は後で表示のために使うだけなので気にしないでください。

```
class Pair[A, B](val a: A, val b: B) {
  override def toString(): String = "(" + a + "," + b + ")"
}
```

このクラス Pair の利用法としては、たとえば割り算の商と余りの両方を返すメソッド divide が挙げられます。divide の定義は次のようになります。

```
def divide(m: Int, n: Int): Pair[Int, Int] = new Pair[Int, Int](m / n, m % n)
```

これらを REPL にまとめて流し込むと次のようになります。

```
class Pair[A, B](val a: A, val b: B) {
  override def toString(): String = "(" + a + "," + b + ")"
}

def divide(m: Int, n: Int): Pair[Int, Int] = new Pair[Int, Int](m / n, m % n)

divide(7, 3)
// res0: Pair[Int, Int] = (2,1)
```

7 割る 3 の商と余りが `res0` に入っていることがわかります。なお、ここでは `new Pair[Int, Int](m / n, m % n)` としましたが、引数の型から型パラメータの型を推測できる場合、省略できます。この場合、`Pair` のコンストラクタに与える引数は `Int` と `Int` なので、`new Pair(m / n, m % n)` としても同じ意味になります。この `Pair` は 2 つの異なる型（同じ型でも良い）を返り値として返したい全ての場合に使うことができます。このように、どの型でも同じ処理を行う場合を抽象化できるのが型パラメータの利点です。

ちなみに、この `Pair` のようなクラスは Scala ではよく使われるため、`Tuple1` から `Tuple22` (`Tuple` の後の数字は要素数）があらかじめ用意されています。また、インスタンス化する際も、

```
val m = 7
// m: Int = 7
val n = 3
// n: Int = 3
new Tuple2(m / n, m % n)
// res1: Tuple2[Int, Int] = (2, 1)
```

などとしなくても、

```
val m = 7
// m: Int = 7
val n = 3
// n: Int = 3
(m / n, m % n)
// res2: Tuple2[Int, Int] = (2, 1)
```

とすれば良いようになっています。

11.1　変位指定（variance）

この節では、型パラメータに関する性質である反変、共変について学びます。

変位指定は使いこなすのが難しいため、自作クラスで利用する場合には注意が必要です。まずは、変位指定が利用されたコードを読めるようになることを目指しましょう。

11.1.1　共変（covariant）

Scala では、何も指定しなかった型パラメータは通常は**非変（invariant）**になります。非変というのは、型パラメータを持ったクラス G 、型パラメータ A と B があったとき、A = B のときにのみ

```
val : G[A] = G[B]
```

というような代入が許されるという性質を表します。これは、違う型パラメータを与えたクラスは違う型になることを考えれば自然な性質です。ここであえて非変について言及したのは、Java の組み込み配列クラスは標準で非変ではなく共変であるという設計ミスを犯しているからです。

ここでまだ共変について言及していなかったので、簡単に定義を示しましょう。共変というのは、型パラメータを持ったクラス G 、型パラメータ A と B があったとき、A が B を継承しているときにのみ、

```
val : G[B] = G[A]
```

というような代入が許される性質を表します。Scala では、クラス定義時に

```
1 class G[+A]
```

のように型パラメータの前に + を付けるとその型パラメータは（あるいはそのクラスは）共変になります。

このままだと定義が抽象的でわかりづらいかもしれないので、具体的な例として配列型を挙げて説明します。配列型は Java では共変なのに対して Scala では非変であるという点において、面白い例です。まずは Java の例です。G = 配列、A = String , B = Object として読んでください。

```
Object[] objects = new String[1];
objects[0] = 100;
```

このコード断片は Java のコードとしてはコンパイルを通ります。ぱっと見でも、Object の配列を表す変数に String の配列を渡すことができるのは理にかなっているように思えます。しかし、このコードを実行すると例外 java.lang.ArrayStoreException が発生します。これは、objects に入っているのが実際には String の配列（String のみを要素として持つ）なのに、2 行目で int 型（ボクシング変換されて Integer 型）の値である 100 を渡そうとしていることによります。

一方、Scala では同様のコードの一行目に相当するコードをコンパイルしようとした時点で、次のようなコンパイルエラーが出ます（Any は全ての型のスーパークラスで、AnyRef に加え、AnyVal（値型）の値も格納できます）。

```
1 scala> val arr: Array[Any] = new Array[String](1)
2 <console>:7: error: type mismatch;
3  found   : Array[String]
4  required: Array[Any]
```

このような結果になるのは、Scala では配列は非変だからです。静的型付き言語の型安全性とは、コンパイル時により多くのプログラミングエラーを捕捉するものであるとするなら、配列の設計は Scala の方が Java より型安全であると言えます。

さて、Scala では型パラメータを共変にした時点で、安全ではない操作はコンパイラがエラーを出してくれるので安心ですが、共変をどのような場合に使えるかを知っておくのは意味があります。たとえば、先ほど作成したクラス `Pair[A, B]` について考えてみましょう。`Pair[A, B]` は一度インスタンス化したら、変更する操作ができませんから、`ArrayStoreException` のような例外は起こり得ません。実際、`Pair[A, B]` は安全に共変にできるクラスで、`class Pair[+A, +B]` のようにしても問題が起きません。

```
class Pair[+A, +B](val a: A, val b: B) {
  override def toString(): String = "(" + a + "," + b + ")"
}

val pair: Pair[AnyRef, AnyRef] = new Pair[String, String]("foo", "bar")
// pair: Pair[AnyRef, AnyRef] = (foo,bar)
```

ここで、`Pair` は作成時に値を与えたら後は変更できず、したがって `ArrayStoreException` のような例外が発生する余地がないことがわかります。一般的には、一度作成したら変更できない（immutable）などの型パラメータは共変にしても多くの場合問題がありません。

11.1.1.1　練習問題

次の *immutable* な *Stack* 型の定義（途中）があります。`???` の箇所を埋めて、*Stack* の定義を完成させなさい。なお、`E >: A` は、`E` は `A` の継承元である、という制約を表しています。

```
trait Stack[+A] {
  def push[E >: A](e: E): Stack[E]
  def top: A
  def pop: Stack[A]
  def isEmpty: Boolean
}

class NonEmptyStack[+A](private val first: A, private val rest: Stack[A]) extends Stack
    [A] {
  def push[E >: A](e: E): Stack[E] = ???
  def top: A = ???
  def pop: Stack[A] = ???
  def isEmpty: Boolean = ???
}

case object EmptyStack extends Stack[Nothing] {
  def push[E >: Nothing](e: E): Stack[E] = new NonEmptyStack[E](e, this)
  def top: Nothing = throw new IllegalArgumentException("empty stack")
  def pop: Nothing = throw new IllegalArgumentException("empty stack")
  def isEmpty: Boolean = true
}

```

```
object Stack {
  def apply(): Stack[Nothing] = EmptyStack
}
```

また、Nothing は全ての型のサブクラスであるような型を表現します。Stack[A] は共変なので、Stack[Nothing] はどんな型の Stack 変数にでも格納することができます。例えば Stack[Nothing] 型である EmptyStack は、Stack[Int] 型の変数と Stack[String] 型の変数の両方に代入することができます。

```
val intStack: Stack[Int] = Stack()
// intStack: Stack[Int] = EmptyStack
val stringStack: Stack[String] = Stack()
// stringStack: Stack[String] = EmptyStack
```

```
class NonEmptyStack[+A](private val first: A, private val rest: Stack[A]) extends Stack
    [A] {
  def push[E >: A](e: E): Stack[E] = new NonEmptyStack[E](e, this)
  def top: A = first
  def pop: Stack[A] = rest
  def isEmpty: Boolean = false
}
```

11.1.2　反変（contravariant）

次は共変とちょうど対になる性質である反変です。簡単に定義を示しましょう。反変というのは、型パラメータを持ったクラス G、型パラメータ A と B があったとき、A が B を継承しているときにのみ、

```
val : G[A] = G[B]
```

というような代入が許される性質を表します。Scala では、クラス定義時に

```
class G[-A]
```

のように型パラメータの前に- を付けるとその型パラメータは（あるいはそのクラスは）反変になります。

反変の例として最もわかりやすいものの 1 つが関数の型です。たとえば、型 A と B があったとき、

```
val x1: A => AnyRef = B => AnyRef型の値
x1(A型の値)
```

というプログラムの断片が成功するためには、A が B を継承する必要があります。その逆では駄目です。仮に、A = String , B = AnyRef として考えてみましょう。

```
val x1: String => AnyRef = AnyRef => AnyRef型の値
x1(String型の値)
```

ここで x1 に実際に入っているのは AnyRef => AnyRef 型の値であるため、引数として String 型の値を与えても、AnyRef 型の引数に String 型の値を与えるのと同様であり、問題なく成功します。A と B が逆で、A = AnyRef , B = String の場合、String 型の引数に AnyRef 型の値を与えるのと同様になってしまうので、これは x1 へ値を代入する時点でコンパイルエラーになるべきであり、実際にコンパイルエラーになります。

実際に REPL で試してみましょう。

```
scala> val x1: AnyRef => AnyRef = (x: String) => (x:AnyRef)
<console>:7: error: type mismatch;
 found   : String => AnyRef
 required: AnyRef => AnyRef
       val x1: AnyRef => AnyRef = (x: String) => (x:AnyRef)
                                                 ^

scala> val x1: String => AnyRef = (x: AnyRef) => x
x1: String => AnyRef = <function1>
```

このように、先ほど述べたような結果になっています。

11.2　型パラメータの境界（bounds）

型パラメータ T に対して何も指定しない場合、その型パラメータ T は、どんな型でも入り得ることしかわかりません。そのため、何も指定しない型パラメータ T に対して呼び出せるメソッドは Any に対するもののみになります。しかし、たとえば、順序がある要素からなるリストをソートしたい場合など、T に対して制約を書けると便利な場合があります。そのような場合に使えるのが、型パラメータの境界（bounds）です。型パラメータの境界には 2 種類あります。

11.2.1　上限境界（upper bounds）

1 つ目は、型パラメータがどのような型を継承しているかを指定する上限境界（upper bounds）です。上限境界では、型パラメータの後に、<: を記述し、それに続いて制約となる型を記述します。以下では、show によって文字列化できるクラス Show を定義したうえで、Show であるような型のみを要素として持つ ShowablePair を定義しています。

```
abstract class Show {
  def show: String
}
class ShowablePair[A <: Show, B <: Show](val a: A, val b: B) extends Show {
  override def show: String = "(" + a.show + "," + b.show + ")"
}
```

ここで、型パラメータ A 、B ともに上限境界として Show が指定されているため、a と b に対して show を呼び出すことができます。なお、上限境界を明示的に指定しなかった場合、Any が指定されたものとみなされます。

11.2.2　下限境界（lower bounds）

2 つ目は、型パラメータがどのような型のスーパータイプであるかを指定する下限境界（lower bounds）です。下限境界は、共変パラメータと共に用いることが多い機能です。実際に例を見ます。

まず、共変の練習問題であったような、イミュータブルな Stack クラスを定義します。この Stack は共変にしたいとします。

```
abstract class Stack[+A]{
  def push(element: A): Stack[A]
  def top: A
  def pop: Stack[A]
  def isEmpty: Boolean
}
```

しかし、この定義は、以下のようなコンパイルエラーになります。

```
error: covariant type A occurs in contravariant position in type A of value element
         def push(element: A): Stack[A]
                               ^
```

このコンパイルエラーは、共変な型パラメータ A が反変な位置（反変な型パラメータが出現できる箇所）に出現したということを言っています。一般に、引数の位置に共変型パラメータ E の値が来た場合、型安全性が壊れる可能性があるため、このようなエラーが出ます。しかし、この Stack は配列と違ってイミュータブルであるため、本来ならば型安全性上の問題は起きません。この問題に対処するために型パラメータの下限境界を使うことができます。型パラメータ E を push に追加し、その下限境界として、Stack の型パラメータ A を指定します。

```
abstract class Stack[+A]{
  def push[E >: A](element: E): Stack[E]
  def top: A
  def pop: Stack[A]
  def isEmpty: Boolean
}
```

このようにすることによって、コンパイラは、Stack には A の任意のスーパータイプの値が入れられる可能性があることがわかるようになります。そして、型パラメータ E は共変ではないため、どこに出現しても構いません。このようにして、下限境界を利用して、型安全な Stack と共変性を両立することができます。

第 12 章

Scala の関数

Scala の関数は、他の言語の関数と扱いが異なります。Scala の関数は単に Function0 〜Function22 までのトレイトの無名サブクラスのインスタンスなのです。

たとえば、2 つの整数を取って加算した値を返す add 関数は次のようにして定義することができます：

```
val add = new Function2[Int, Int, Int]{
  def apply(x: Int, y: Int): Int = x + y
}
// add: Function2[Int, Int, Int] = <function2>

add.apply(100, 200)
// res0: Int = 300

add(100, 200)
// res1: Int = 300
```

Function0 から Function22 までの全ての関数は引数の数に応じた apply メソッドを定義する必要があります。apply メソッドは Scala コンパイラから特別扱いされ、x.apply(y) は常に x(y) のように書くことができます。後者の方が関数の呼び方としては自然ですね。

また、関数を定義するといっても、単に Function0 から Function22 までのトレイトの無名サブクラスのインスタンスを作っているだけです。

12.1 無名関数

前項で Scala で関数を定義しましたが、これを使ってプログラミングをするとコードが冗長になり過ぎます。そのため、Scala では Function0 〜Function22 までのトレイトのインスタンスを生成する

ためのシンタックスシュガー[*1]が用意されています。たとえば、先ほどの add 関数は

```
val add = (x: Int, y: Int) => x + y
// add: Function2[Int, Int, Int] = repl.MdocSession$MdocApp$$Lambda$14142/0
    x0000743a8dfc9000@7ede794a
```

と書くことができます。ここで、add には単に関数オブジェクトが入っているだけであって、関数本体には何の名前も付いていないことに注意してください。この、add の右辺のような定義を Scala では無名関数と呼びます。無名関数は単なる FunctionN オブジェクトですから、自由に変数や引数に代入したり返り値として返すことができます。このような、関数を自由に変数や引数に代入したり返り値として返すことができる性質を指して、Scala では関数が第一級の値（First Class Object）であるといいます。

無名関数の一般的な構文は次のようになります。

```
(n1: N1, n2: N2, n3: N3, ...nn: NN) => B
```

n1 から nn までが仮引数の定義で N1 から NN までが仮引数の型です。B は無名関数の本体です。無名関数の返り値の型は通常は B の型から推論されます。先ほど述べたように、Scala 2 における関数は Function0 〜Function22 までのトレイトの無名サブクラスのインスタンスですから、引数の最大個数は 22 個になります。Scala 3 からはその制約が撤廃されて、23 以上の関数も作成可能になっています。

12.2　関数の型

このようにして定義した関数の型は、本来は FunctionN[...] のようにして記述しなければいけませんが、関数の型については特別にシンタックスシュガーが設けられています。一般に、

```
(n1: N1, n2: N2, n3: N3, ...nn: NN) => B
```

となるような関数の型は FunctionN[N1, N2, N3, ...NN, Bの型] と書く代わりに

```
(N1, N2, N3, ...NN) => Bの型
```

として記述することができます。直接 FunctionN を型として使うことは稀なので、こちらのシンタックスシュガーを覚えておくと良いでしょう。

12.3　関数のカリー化

関数型言語ではカリー化というテクニックがよく使われます。カリー化とは、たとえば (Int, Int) => Int 型の関数のように複数の引数を取る関数があったとき、これを Int => Int => Int 型の関数

[*1] Scala 2.12 からは、SAM Type と互換性がある場合には対応する SAM Type のコードが生成されるので、純粋なシンタックスシュガーとは呼べなくなりましたが、それ以外のケースについては従来と変わりありません。

のように、1 つの引数を取り、残りの引数を取る関数を返す関数のチェインで表現するというものです。試しに上記の add をカリー化してみましょう。

```
val add = (x: Int, y: Int) => x + y
// add: Function2[Int, Int, Int] = repl.MdocSession$MdocApp$$Lambda$14143/0
    x0000743a8dfc97f0@7789d0a7

val addCurried = (x: Int) => ((y: Int) => x + y)
// addCurried: Function1[Int, Function1[Int, Int]] = repl.
    MdocSession$MdocApp$$Lambda$14144/0x0000743a8dfca000@77fb3fb4

add(100, 200)
// res2: Int = 300

addCurried(100)(200)
// res3: Int = 300
```

無名関数を定義する構文をネストさせて使っているだけで、何も特別なことはしていないことがわかります。

また、Scala ではメソッドの引数リストを複数に分けた場合に、そのメソッドに対してスペースに続いて_ をつけることで引数リストそれぞれを新たな引数としたカリー化関数を得ることができます。このことを REPL を用いて確認してみましょう。

```
scala> def add(x: Int, y: Int): Int = x + y
add: (x: Int, y: Int)Int

scala> add _
res0: (Int, Int) => Int = <function2>

scala> def addMultiParameterList(x: Int)(y: Int): Int = x + y
addMultiParameterList: (x: Int)(y: Int)Int

scala> addMultiParameterList _
res1: Int => (Int => Int) = <function1>
```

引数リストを 2 つに分けた addMultiParameterList（これはメソッドであって関数ではありません）から得られた関数は 1 引数関数のチェイン（res1 ）になっていて、確かにカリー化されています。

ただ、Scala ではカリー化のテクニックを使うことは、他の関数型言語に比べてあまり多くありません。また、この _ を付与して関数を得る記法は Scala 3 では非推奨になっており、_ を付与しなくても関数に変換されるようになりました。

12.4　メソッドと関数の違い

メソッドについては既に説明しましたが、メソッドと関数の違いについては Scala を勉強する際に注意する必要があります。本来は def で始まる構文で定義されたものだけがメソッドなのですが、説明の便宜上、所属するオブジェクトの無いメソッド（今回は説明していません）や REPL で定義した

メソッドを関数と呼んだりすることがあります。書籍や Web でもこの 2 つを意図的に、あるいは無意識に混同している例が多々あるので（Scala のバイブル『Scala スケーラブルプログラミング』でも意図的なメソッドと関数の混同の例がいくつかあります）注意してください。

再度強調すると、メソッドは def で始まる構文で定義されたものであり、それを関数と呼ぶのはあくまで説明の便宜上であるということです。ここまでメソッドと関数の違いについて強調してきましたが、それは、メソッドは第一級の値ではないのに対して関数は第一級の値であるという大きな違いがあるからです。メソッドを取る引数やメソッドを返す関数、メソッドが入った変数といったものは Scala には存在しません。

12.5　高階関数

関数を引数に取ったり関数を返すメソッドや関数のことを高階関数と呼びます。先ほどメソッドと関数の違いについて説明したばかりなのに、メソッドのことも関数というのはいささか奇妙ですが、慣習的にそう呼ぶものだと思ってください。

早速高階関数の例についてみてみましょう。

```
def double(n: Int, f: Int => Int): Int = {
  f(f(n))
}
```

これは与えられた関数 f を 2 回 n に適用する関数 double です。ちなみに、高階関数に渡される関数は適切な名前が付けられないことも多く、その場合は f や g などの 1 文字の名前をよく使います。他の関数型プログラミング言語でも同様の慣習があります。呼び出しは次のようになります。

```
double(1, m => m * 2)
// res4: Int = 4

double(2, m => m * 3)
// res5: Int = 18

double(3, m => m * 4)
// res6: Int = 48
```

最初の呼び出しは 1 に対して、与えられた引数を 2 倍する関数を渡していますから、1 * 2 * 2 = 4 になります。2 番めの呼び出しは 2 に対して、与えられた引数を 3 倍する関数を渡していますから、2 * 3 * 3 = 18 になります。最後の呼び出しは、3 に対して与えられた引数を 4 倍する関数を渡していますから、3 * 4 * 4 = 48 になります。

もう少し意味のある例を出してみましょう。プログラムを書くとき、

1. 初期化
2. 何らかの処理
3. 後始末処理

というパターンは頻出します。これをメソッドにした高階関数 around を定義します。

```
def around(init: () => Unit, body: () => Any, fin: () => Unit): Any = {
  init()
  try {
    body()
  } finally {
    fin()
  }
}
```

try-finally 構文は、後の例外処理の節でも出てきますが、大体 Java のそれと同じだと思ってください。この around 関数は次のようにして使うことができます。

```
around(
  () => println("???????"),
  () => println("??????????"),
  () => println("????????")
)
// ???????
// ??????????
// ????????
// res7: Any = ()
```

around に渡した関数が順番に呼ばれていることがわかります。ここで、body の部分で例外を発生させてみます。throw は Java のそれと同じで例外を投げるための構文です。

```
around(
  () => println("???????"),
  () => throw new Exception("?????"),
  () => println("????????")
)
// java.lang.Exception: ?????
//  at repl.MdocSession$MdocApp.$init$$$anonfun$1$$anonfun$2(function.md:109)
//  at repl.MdocSession$MdocApp.around$1(function.md:84)
//  at repl.MdocSession$MdocApp.$init$$$anonfun$1(function.md:110)
```

body の部分で例外が発生しているにも関わらず、fin の部分はちゃんと実行されていることがわかります。ここで、around という高階関数を定義することで、

1. 初期化
2. 何らかの処理
3. 後始末処理

のそれぞれを部品化して、「何らかの処理」の部分で異常が発生しても必ず後始末処理を実行できています。この around メソッドは 1〜3 の手順を踏む様々な処理に流用することができます。一方、1〜3 のそれぞれは呼び出し側で自由に与えることができます。このように処理を**値**として**部品化**することは高階関数を定義する大きなメリットの 1 つです。Java 7 では後始末処理を自動化する

try-with-resources 文が言語として取り入れられましたが、高階関数のある言語では、言語に頼らず自分でそのような働きをするメソッドを定義することができます。

なお around のように高階関数を利用してリソースの後始末を行うパターンはローンパターンと呼ばれています。ローンパターンは例えば以下の withFile のような形で利用されることが多いです。

```
import scala.io.Source
def withFile[A](filename: String)(f: Source => A): A = {
  val s = Source.fromFile(filename)
  try {
    f(s)
  } finally {
    s.close()
  }
}
```

ファイル以外でも、何かリソースを取得して処理が終わったら確実に解放したいケースではローンパターンを利用することが出来ます。

12.5.1　練習問題

withFile メソッドを使って、次のようなシグネチャを持つテキストファイルの中身を一行ずつ表示する関数 printFile を実装してみましょう。

```
def printFile(filename: String): Unit = ???
```

```
def printFile(filename: String): Unit = {
  withFile(filename) { file =>
    file.getLines().foreach(println)
  }
}
```

ここでは高階関数の利用例を簡単に紹介しました。後のコレクションの節を読むことで、高階関数のメリットをさらに理解できるようになるでしょう。

第13章

コレクションライブラリ（immutable と mutable）

Scala には配列（`Array`）やリスト（`List`）、連想配列（`Map`）、集合（`Set`）を扱うための豊富なライブラリがあります。これを使いこなすことで、Scala でのプログラミングは劇的に楽になります。注意しなければならないのは、Scala では一度作成したら変更できない（immutable）なコレクションと変更できる通常のコレクション（mutable）があることです。皆さんは mutable なコレクションに馴染んでいるかと思いますが、Scala で関数型プログラミングを行うためには、immutable なコレクションを活用する必要があります。

immutable なコレクションを使うのにはいくつものメリットがあります

- 関数型プログラミングで多用する再帰との相性が良い
- 高階関数を用いて簡潔なプログラムを書くことができる
- 一度作ったコレクションが知らない箇所で変更されていない事を保証できる
- 並行に動作するプログラムの中で、安全に受け渡しすることができる

mutable なコレクションを効果的に使えばプログラムの実行速度を上げることができますが、mutable なコレクションをどのような場面で使えばいいかは難しい問題です。

この節では、Scala のコレクションライブラリに含まれる以下のものについての概要を説明します。

- `Array` (mutable)
- `List` (immutable)
- `Map` (immutable)・`Map` (mutable)
- `Set` (immutable)・`Set` (mutable)

13.1 Array

まずは大抵のプログラミング言語にある配列です。

```
val arr = Array(1, 2, 3, 4, 5)
// arr: Array[Int] = Array(1, 2, 3, 4, 5)
```

これで 1 から 5 までの要素を持った配列が arr に代入されました。Scala の配列は、他の言語のそれと同じように要素の中身を入れ替えることができます。配列の添字は 0 から始まります。なお、配列の型を指定しなくて良いのは、Array(1, 2, 3, 4, 5) の部分で、要素型が Int であるに違いないとコンパイラが型推論してくれるからです。型を省略せずに書くと

```
val arr = Array[Int](1, 2, 3, 4, 5)
// arr: Array[Int] = Array(7, 2, 3, 4, 5)
```

となります。ここで、[Int] の部分は型パラメータと呼びます。Array だけだとどの型かわからないので、[Int] を付けることでどの型の Array かを指定しているわけです。この型パラメータは型推論を補うために、色々な箇所で出てくるので覚えておいてください。しかし、この場面では、Array の要素型は Int だとわかっているので、冗長です。次に要素へのアクセスと代入です。

```
arr(0) = 7

arr
// res1: Array[Int] = Array(7, 2, 3, 4, 5)

arr(0)
// res2: Int = 7
```

他の言語だと arr[0] のようにしてアクセスすることが多いので最初は戸惑うかもしれませんが、慣れてください。配列の 0 番目の要素がちゃんと 7 に入れ替わっていますね。

配列の長さは arr.length で取得することができます。

```
arr.length
// res3: Int = 5
```

Array[Int] は Java では int[] と同じ意味です。Scala では、配列などのコレクションの要素型を表記するとき Collection[ElementType] のように一律に表記し、配列も同じように記述するのです。Java では配列型だけ特別扱いするのに比べると統一的だと言えるでしょう。

ただし、あくまでも表記上はある程度統一的に扱えますが、実装上は JVM の配列であり、**要素が同じでも equals の結果が true にならない, 生成する際に ClassTag というものが必要**などのいくつかの罠があるので、Array はパフォーマンス上必要になる場合以外はあまり積極的に使うものではありません。

13.1.1　練習問題

配列の i 番目の要素と j 番目の要素を入れ替える swapArray メソッドを定義してみましょう。swapArray メソッドの宣言は

```
def swapArray[T] (arr: Array[T])(i: Int, j: Int): Unit = ???
```

となります。i と j が配列の範囲外である場合は特に考慮しなくて良いです。

```
def swapArray[T] (arr: Array[T])(i: Int, j: Int): Unit = {
  val tmp = arr(i)
  arr(i) = arr(j)
  arr(j) = tmp
}
```

```
val arr = Array(1, 2, 3, 4, 5)
// arr: Array[Int] = Array(5, 4, 3, 2, 1)

swapArray(arr)(0, 4)

arr
// res5: Array[Int] = Array(5, 4, 3, 2, 1)

swapArray(arr)(1, 3)

arr
// res7: Array[Int] = Array(5, 4, 3, 2, 1)
```

13.1.2　Range

Range は範囲を表すオブジェクトです。Range は直接名前を指定して生成するより、to メソッドと until メソッドを用いて呼びだすことが多いです。また、toList メソッドを用いて、その範囲の数値の列を後述する List に変換することができます。では、早速 REPL で Range を使ってみましょう。

```
1 to 5
// res8: Inclusive = Range(1, 2, 3, 4, 5)

(1 to 5).toList
// res9: List[Int] = List(1, 2, 3, 4, 5)

1 until 5
// res10: Range = Range(1, 2, 3, 4)

(1 until 5).toList
// res11: List[Int] = List(1, 2, 3, 4)
```

to は右の被演算子を含む範囲を、until は右の被演算子を含まない範囲を表していることがわかります。また、Range は toList で後述する List に変換することができることもわかります。

13.1.3　List

さて、導入として大抵の言語にある Array を出しましたが、Scala では Array を使うことはそれほど多くありません。代わりに List や Vector といったデータ構造をよく使います（Vector については後述します）。List の特徴は、一度作成したら中身を変更できない（immutable）ということです。中身を変更できないデータ構造（永続データ構造とも呼びます）は Scala がサポートしている関数型プログラミングにとって重要な要素です。それでは List を使ってみましょう。

```
val lst = List(1, 2, 3, 4, 5)
// lst: List[Int] = List(1, 2, 3, 4, 5)
```

```
lst(0) = 7
```

見ればわかるように、List は一度作成したら値を更新することができません。しかし、List は値を更新することができませんが、ある List を元に新しい List を作ることができます。これが値を更新することの代わりになります。以降、List に対して組み込みで用意されている各種操作をみていくことで、List の値を更新することなく色々な操作ができることがわかるでしょう。

13.1.4　Nil：空の List

まず最初に紹介するのは Nil です。Scala で空の List を表すには Nil というものを使います。Ruby などでは nil は言語上かなり特別な意味を持ちますが、Scala ではデフォルトでスコープに入っているということ以外は特別な意味はなく単に object です。Nil は単体では意味がありませんが、次に説明する :: と合わせて用いることが多いです。

13.1.5　:: - List の先頭に要素をくっつける

::（コンスと読みます）は既にある List の先頭に要素をくっつけるメソッドです。これについては、REPL で結果をみた方が早いでしょう。

```
val a1 = 1 :: Nil
// a1: List[Int] = List(1)
val a2 = 2 :: a1
// a2: List[Int] = List(2, 1)
val a3 = 3 :: a2
// a3: List[Int] = List(3, 2, 1)
val a4 = 4 :: a3
// a4: List[Int] = List(4, 3, 2, 1)
val a5 = 5 :: a3
// a5: List[Int] = List(5, 3, 2, 1)
```

付け足したい要素を :: を挟んで List の前に書くことで List の先頭に要素がくっついていることがわかります。ここで、:: はやや特別な呼び出し方をするメソッドであることを説明しなければなり

ません。まず、Scala では 1 引数のメソッドは中置記法で書くことができます。それで、`1 :: Nil` のように書くことができるわけです。次に、メソッド名の最後が: で終わる場合、被演算子の前と後ろをひっくり返して右結合で呼び出します。たとえば、

```
1 :: 2 :: 3 :: 4 :: Nil
// res12: List[Int] = List(1, 2, 3, 4)
```

は、実際には、

```
Nil.::(4).::(3).::(2).::(1)
// res13: List[Int] = List(1, 2, 3, 4)
```

のように解釈されます。`List` の要素が演算子の前に来て、一見数値のメソッドのように見えるのに `List` のメソッドとして呼び出せるのはそのためです。

13.1.6　++：List 同士の連結

++ は List 同士を連結するメソッドです。これも REPL で見た方が早いでしょう。

```
List(1, 2) ++ List(3, 4)
// res14: List[Int] = List(1, 2, 3, 4)

List(1) ++ List(3, 4, 5)
// res15: List[Int] = List(1, 3, 4, 5)

List(3, 4, 5) ++ List(1)
// res16: List[Int] = List(3, 4, 5, 1)
```

++ は 1 引数のメソッドなので、中置記法で書いています。また、末尾が: で終わっていないので、たとえば、

```
List(1, 2) ++ List(3, 4)
// res17: List[Int] = List(1, 2, 3, 4)
```

は

```
List(1, 2).++(List(3, 4))
// res18: List[Int] = List(1, 2, 3, 4)
```

と同じ意味です。大きな `List` 同士を連結する場合、計算量が大きくなるのでその点には注意した方が良いです。

13.1.7　mkString：文字列のフォーマッティング

このメソッドは Scala で非常に**頻繁に使用されます。**皆さんも、Scala を使っていく上で使う機会が多いであろうメソッドです。このメソッドは引数によって多重定義されており、3 バージョンあるのでそれぞれを紹介します。

13.1.7.1　mkString

引数なしバージョンです。このメソッドは、単に List の各要素を左から順に繋げた文字列を返します。

```
List(1, 2, 3, 4, 5).mkString
// res19: String = "12345"
```

注意しなければならないのは、引数なしメソッドの mkString は () を付けて呼びだすことが**できない**という点です。たとえば、以下のコードは、若干分かりにくいエラーメッセージがでてコンパイルに失敗します。

```
List(1, 2, 3, 4, 5).mkString()
```

Scala の 0 引数メソッドは () なしと () を使った定義の二通りあって、前者の形式で定義されたメソッドは () を付けずに呼び出さなければいけません。同様に、() を使って定義されたメソッドは、() を付けて呼び出さなければいけません。この Scala の仕様は混乱しやすいので注意してください。

13.1.7.2　mkString(sep: String)

引数にセパレータ文字列 sep を取り、List の各要素を sep で区切って左から順に繋げた文字列を返します。

```
List(1, 2, 3, 4, 5).mkString(",")
// res20: String = "1,2,3,4,5"
```

13.1.7.3　mkString(start: String, sep: String, end: String)

mkString(sep) とほとんど同じですが、start と end に囲まれた文字列を返すところが異なります。

```
List(1, 2, 3, 4, 5).mkString("[", ",", "]")
// res21: String = "[1,2,3,4,5]"
```

13.1.7.4　練習問題

mkString を使って、最初の数 start と最後の数 end を受け取って、

```
start,(start+1),(start+2)...,end
```

となるような文字列を返すメソッド joinByComma を定義してみましょう（ヒント：Range にも mkString メソッドはあります）。

例

```
joinByComma(1,5)  // 1,2,3,4,5
```

```
def joinByComma(start: Int, end: Int): String = {
  ???
}
```

```
def joinByComma(start: Int, end: Int): String = {
  (start to end).mkString(",")
}
```

```
joinByComma(1, 10)
// res22: String = "1,2,3,4,5,6,7,8,9,10"
```

(start to end) で、start から end までの列を作って、mkString(",") を使って間に , を挟んでいます。

13.1.8　foldLeft：左からの畳み込み

foldLeft メソッドは List にとって非常に基本的なメソッドです。他の様々なメソッドを foldLeft を使って実装することができます。foldLeft の宣言を Scala の API ドキュメントから引用すると、

```
def foldLeft[B](z: B)(f: (B, A) => B): B
```

となります。z が foldLeft の結果の初期値で、リストを左からたどりながら f を適用していきます。foldLeft についてはイメージが湧きにくいと思いますので、List(1, 2, 3).foldLeft(0)((x, y) => x + y) の結果を図示します。

```
      +
     / \
    +   3
   / \
  +   2
 / \
0   1
```

この図で、

```
  +
 / \
0   1
```

は + に 0 と 1 を与えて適用するということを意味します。リストの要素を左から順に f を使って「畳み込む」（fold は英語で畳み込むという意味を持ちます）状態がイメージできるでしょうか。foldLeft は汎用性の高いメソッドで、たとえば、List の要素の合計を求めたい場合は

```
List(1, 2, 3).foldLeft(0)((x, y) => x + y)
// res23: Int = 6
```

List の要素を全て掛けあわせた結果を求めたい場合は

```
List(1, 2, 3).foldLeft(1)((x, y) => x * y)
// res24: Int = 6
```

とすることで求める結果を得ることができます[*1]。その他にも様々な処理を foldLeft を用いて実装することができます。

さて、節の最後に、実用上の補足を少ししておきます。少し恣意的ですが 1 つの例として、「リストのリスト」をリストに変換する（平らにする）処理というのを考えてみます。List(List(1), List(2 ,3)) を List(1, 2, 3) に変換するのが目標です。安直に書くとこうなるでしょうか：

```
scala> List(List(1), List(2, 3), List(4)).foldLeft(Nil)(_ ++ _)
<console>:12: error: type mismatch;
 found   : List[Int]
 required: scala.collection.immutable.Nil.type
       List(List(1), List(2, 3), List(4)).foldLeft(Nil)(_ ++ _)
                                                         ^
```

しかしコンパイルが通りません。エラーメッセージの意味としては、今回の Nil は List[Int] 型と見なされてほしいわけですが、期待したように型推論できていないようです。Nil に明示的に型注釈を付けることで、コンパイルできるようになります。

```
List(List(1), List(2, 3), List(4)).foldLeft(Nil: List[Int])(_ ++ _)
// res25: List[Int] = List(1, 2, 3, 4)
```

このように、Nil が混ざった処理はそのままだとうまくコンパイルが通ってくれないことがあります。そういう場合は型注釈を試すとよい、と頭の片隅に入れておいてください。

13.1.8.1　練習問題

foldLeft を用いて、List の要素を反転させる次のシグニチャを持ったメソッド reverse を実装してみましょう：

```
def reverse[T](list: List[T]): List[T] = ???
```

```
def reverse[T](list: List[T]): List[T] = list.foldLeft(Nil: List[T])((a, b) => b :: a)
```

```
reverse(List(1, 2, 3, 4, 5))
// res26: List[Int] = List(5, 4, 3, 2, 1)
```

foldLeft の初期値に Nil を与えて、そこから後ろにたどる毎に、「前に」要素を追加していくことで、逆順のリストを作ることができています。

[*1] ただし、これはあくまでも foldLeft の例であって、要素の和や積を求めたい場合に限って言えばもっと便利なメソッドが標準ライブラリに存在するので、実際にはこの例のような使い方はしません

13.1.9　foldRight：右からの畳み込み

foldLeft が List の左からの畳み込みだったのに対して、foldRight は右からの畳込みです。foldRight の宣言を Scala の API ドキュメントから参照すると、

```
def foldRight[B](z: B)(op: (A, B) => B): B
```

となります。foldRight に与える関数である op の引数の順序が foldLeft の場合と逆になっている事に注意してください。foldRight を List(1, 2, 3).foldRight(0)((y, x) => y + x) とした場合の様子を図示すると次のようになります。

```
   +
  / \
 1   +
    / \
   2   +
      / \
     3   0
```

ちょうど foldLeft と対称になっています。foldRight も非常に汎用性の高いメソッドで、多くの処理を foldRight を用いて実装することができます。

13.1.9.1　練習問題

List の全ての要素を足し合わせるメソッド sum を foldRight を用いて実装してみましょう。sum の宣言は次のようになります。なお、List が空のときは 0 を返してみましょう。

```
def sum(list: List[Int]): Int = ???
```

```
def sum(list: List[Int]): Int = list.foldRight(0){(x, y) => x + y}
```

```
sum(List(1, 2, 3, 4, 5))
// res28: Int = 15
```

13.1.9.2　練習問題

List の全ての要素を掛け合わせるメソッド mul を foldRight を用いて実装してみましょう。mul の宣言は次のようになります。なお、List が空のときは 1 を返してみましょう。

```
def mul(list: List[Int]): Int = ???
```

```
def mul(list: List[Int]): Int = list.foldRight(1){(x, y) => x * y}
```

```
mul(List(1, 2, 3, 4, 5))
// res29: Int = 120
```

13.1.9.3　練習問題

mkString を実装してみましょう。mkString そのものを使ってはいけませんが、foldLeft や foldRight などの List に定義されている他のメソッドは自由に使って構いません。List の API リファレンスを読めば必要なメソッドが載っています。実装する mkString の宣言は

```
def mkString[T](list: List[T])(sep: String): String = ???
```

となります。残りの 2 つのバージョンの mkString は実装しなくても構いません。

```
def mkString[T](list: List[T])(sep: String): String = list match {
  case Nil => ""
  case x::xs => xs.foldLeft(x.toString){(x, y) => x + sep + y}
}
```

13.1.10　map：各要素を加工した新しい List を返す

map メソッドは、1 引数の関数を引数に取り、各要素に関数を適用した結果できた要素からなる新たな List を返します。ためしに List(1, 2, 3, 4, 5) の各要素を 2 倍してみましょう。

```
List(1, 2, 3, 4, 5).map(x => x * 2)
// res30: List[Int] = List(2, 4, 6, 8, 10)
```

x => x * 2 の部分は既に述べたように、無名関数を定義するための構文です。メソッドの引数に与える短い関数を定義するときは、Scala では無名関数をよく使います。List の全ての要素に何らかの処理を行い、その結果を加工するという処理は頻出するため、map は Scala のコレクションのメソッドの中でも非常によく使われるものになっています。

13.1.10.1　練習問題

次のシグニチャを持つ map メソッドを foldLeft と reverse を使って実装してみましょう：

```
def map[T, U](list: List[T])(f: T => U): List[U] = ???
```

map メソッドは次のようにして使います。

```
assert(List(2, 3, 4) == map(List(1, 2, 3))(x => x + 1))
assert(List(2, 4, 6) == map(List(1, 2, 3))(x => x * 2))
assert(Nil == map(List[Int]())(x => x * x))
assert(List(0, 0, 0)  == map(List(1, 2, 3))(x => 0))
```

```
def map[T, U](list: List[T])(f: T => U): List[U] = {
  list.foldLeft(Nil:List[U]){(x, y) => f(y) :: x}.reverse
}
```

13.1.11　filter：条件に合った要素だけを抽出した新しい List を返す

filter メソッドは、Boolean 型を返す 1 引数の関数を引数に取り、各要素に関数を適用し、true になった要素のみを抽出した新たな List を返します。List(1, 2, 3, 4, 5) から奇数だけを抽出してみましょう。

```
List(1, 2, 3, 4, 5).filter(x => x % 2 == 1)
// res31: List[Int] = List(1, 3, 5)
```

13.1.11.1　練習問題

次のシグニチャを持つ filter メソッドを foldLeft と reverse を使って実装してみましょう：

```
def filter[T](list: List[T])(f: T => Boolean): List[T] = ???
```

```
assert(List(2) == filter(List(1, 2, 3))(x => x % 2 == 0))
assert(List(1, 3) == filter(List(1, 2, 3))(x => x % 2 == 1))
assert(Nil == filter(List(1, 2, 3))(x => x > 3))
assert(List(1) == filter(List(1))(x => x == 1))
assert(Nil == filter(List[Int]())(x => false))
```

```
def filter[T](list: List[T])(f: T => Boolean): List[T] = {
  list.foldLeft(Nil:List[T]){(x, y) => if(f(y)) y::x else x}.reverse
}
```

13.1.12　find：条件に合った最初の要素を返す

find メソッドは、Boolean 型を返す 1 引数の関数を引数に取り、各要素に前から順番に関数を適用し、最初に true になった要素を Some でくるんだ値を Option 型として返します。1 つの要素もマッチしなかった場合 None を Option 型として返します。List(1, 2, 3, 4, 5) から最初の奇数だけを抽出してみましょう

```
List(1, 2, 3, 4, 5).find(x => x % 2 == 1)
// res32: Option[Int] = Some(1)
```

後で説明されることになりますが、Option 型は Scala プログラミングの中で重要な要素であり頻出します。

13.1.12.1　練習問題

次のシグニチャを持つ find メソッドを foldLeft または再帰で実装してみましょう。Option[T] 型の Some[T] は Some(1) のように生成できます。また、要素がないことを表す None は None と表記できます。

```
def find[T](list: List[T])(f: T => Boolean): Option[T] = ???
```

```
assert(Some(2) == find(List(1, 2, 3))(x => x == 2))
assert(None == find(List(1, 2, 3))(x => x > 3))
assert(Some(1) == find(List(1))(x => x == 1))
assert(None == find(List(1))(x => false))
assert(None == find(List[Int]())(x => x == 1))
```

```
def find[T](list: List[T])(f: T => Boolean): Option[T] = list match {
  case x::xs if f(x) => Some(x)
  case x::xs => find(xs)(f)
  case _ => None
}
```

13.1.13　takeWhile：先頭から条件を満たしている間を抽出する

`takeWhile` メソッドは、`Boolean` 型を返す 1 引数の関数を引数に取り、前から順番に関数を適用し、結果が `true` の間のみからなる `List` を返します。`List(1, 2, 3, 4, 5)` の 5 より前の 4 要素を抽出してみます。

```
List(1, 2, 3, 4, 5).takeWhile(x => x != 5)
// res33: List[Int] = List(1, 2, 3, 4)
```

13.1.13.1　練習問題

次のシグニチャを持つ `takeWhile` メソッドをループまたは再帰を使って実装してみましょう：

```
def takeWhile[T](list: List[T])(f: T => Boolean): List[T] = ???
```

`takeWhile` メソッドは次のようにして使います。

```
assert(List(1, 2, 3) == takeWhile(List(1, 2, 3, 4, 5))(x => x <= 3))
assert(List(1) == takeWhile(List(1, 2, 3, 3, 4, 5))(x => x == 1))
assert(List(1, 2, 3, 4)  == takeWhile(List(1, 2, 3, 4, 5))(x => x < 5))
assert(Nil == takeWhile(List(1, 2, 3, 3, 2, 2))(x => false))
```

```
def takeWhile[T](list: List[T])(f: T => Boolean): List[T] = {
  list match {
    case x::xs if f(x) =>
      x::takeWhile(xs)(f)
    case _ =>
      Nil
  }
}
```

13.1.14　count：List の中で条件を満たしている要素の数を計算する

count メソッドは、Boolean 型を返す 1 引数の関数を引数に取り、全ての要素に関数を適用して、true が返ってきた要素の数を計算します。例として List(1, 2, 3, 4, 5) の中から偶数の数（2 になるはず）を計算してみます。

```
List(1, 2, 3, 4, 5).count(x => x % 2 == 0)
// res34: Int = 2
```

13.1.14.1　練習問題

次のシグニチャを持つ count メソッドを foldLeft を使って実装してみましょう：

```
def count[T](list: List[T])(f: T => Boolean): Int = ???
```

count メソッドは次のようにして使います。

```
assert(3 == count(List(1, 2, 3, 3, 2, 2))(x => x == 2))
assert(1 == count(List(1, 2, 3, 3, 2, 2))(x => x == 1))
assert(2 == count(List(1, 2, 3, 3, 2, 2))(x => x == 3))
assert(0 == count(List(1, 2, 3, 3, 2, 2))(x => x == 5))
```

```
def count[T](list: List[T])(f: T => Boolean): Int  = {
  list.foldLeft(0){(x, y) => if(f(y)) x + 1 else x}
}
```

13.1.15　flatMap：List をたいらにする

flatMap は一見少し変わったメソッドですが、後々重要になってくるメソッドなので説明しておきます。flatMap の宣言は Scala の API ドキュメントから参照すると、

```
final def flatMap[B](f: (A) => IterableOnce[B]): List[B]
```

となります。ここで、IterableOnce[B] という変わった型が出てきていますが、ここではあらゆるコレクション（要素の型は B 型である）を入れることができる型程度に考えてください。さて、flatMap の引数 f の型は (A) => IterableOnce[B] です。flatMap はこれを使って、各要素に f を適用して、結果の要素からなるコレクションを分解して List の要素にします。これについては、実際に見た方が早いでしょう。

```
List(List(1, 2, 3), List(4, 5)).flatMap{e => e.map{g => g + 1}}
// res35: List[Int] = List(2, 3, 4, 5, 6)
```

ネストした List の各要素に flatMap の中で map を適用して、List の各要素に 1 を足したものをたいらにしています。これだけだとありがたみがわかりにくいですが、ちょっと形を変えてみると非常に面白い使い方ができます：

```
List(1, 2, 3).flatMap{e => List(4, 5).map(g => e * g)}
// res36: List[Int] = List(4, 5, 8, 10, 12, 15)
```

List(1, 2, 3) と List(4, 5) の 2 つの List についてループし、各々の要素を掛けあわせた要素からなる List を抽出しています。実は、for-comprehension

```
for(x <- col1; y <- col2) yield z
```

は

```
col1.flatMap{x => col2.map{y => z}}
```

のシンタックスシュガーだったのです。すなわち、ある自分で定義したデータ型に flatMap と map を（適切に）実装すれば for 構文の中で使うことができるのです。

13.1.15.1　練習問題

次のシグニチャを持つ flatMap メソッドを再帰やループで実装してみましょう：

```
def flatMap[T, U](list: List[T])(f: T => List[U]): List[U] = ???
```

flatMap メソッドは次のようにして使います。

```
assert(List(1, 2, 3) == flatMap(List(1, 2, 3))(x => List(x)))
assert(
  List(3, 4, 6, 8) == flatMap(List(1, 2))(x =>
    map(List(3, 4))(y => x * y)
  )
)
```

```
def flatMap[T, U](list: List[T])(f: T => List[U]): List[U] = {
  list match {
    case Nil => Nil
    case x::xs => f(x) ::: flatMap(xs)(f)
  }
}
```

13.1.15.2　List の性能特性

List の性能特性として、List の先頭要素へのアクセスは高速にできる反面、要素へのランダムアクセスや末尾へのデータの追加は List の長さに比例した時間がかかってしまうということが挙げられます。List は関数型プログラミング言語で最も基本的なデータ構造で、どの関数型プログラミング

言語でもたいていは List がありますが、その性能特性には十分注意して扱う必要があります。特に他の言語のプログラマはうっかり List の末尾に要素を追加するような遅いプログラムを書いてしまうことがあるので注意する必要があります。

```
List(1, 2, 3, 4)
// res37: List[Int] = List(1, 2, 3, 4)

5 :: List(1, 2, 3, 4) // List???????????????
// res38: List[Int] = List(5, 1, 2, 3, 4)

List(1, 2, 3, 4) :+ 5 // ??????????List????????
// res39: List[Int] = List(1, 2, 3, 4, 5)
```

13.1.16　紹介したメソッドについて

mkString をはじめとした List の色々なメソッドを紹介してきましたが、実はこれらの大半は List 特有ではなく、既に紹介した Range や Array 、これから紹介する他のコレクションでも同様に使うことができます。何故ならばこれらの操作の大半は特定のコレクションではなく、コレクションのスーパータイプである共通のトレイト中に宣言されているからです。もちろん、List に要素を加える処理と Set に要素を加える処理（Set に既にある要素は加えない）のように、中で行われる処理が異なることがあるので、その点は注意する必要があります。詳しくは Scala の API ドキュメントを探索してみましょう。

13.1.17　Vector

Vector は少々変わったデータ構造です。Vector は一度データ構造を構築したら変更できない immutable なデータ構造です。要素へのランダムアクセスや長さの取得、データの挿入や削除、いずれの操作も十分に高速にできる比較的万能なデータ構造です。immutable なデータ構造を使う場合は、まず Vector を検討すると良いでしょう。

```
Vector(1, 2, 3, 4, 5) //??????????????????
// res40: Vector[Int] = Vector(1, 2, 3, 4, 5)

6 +: Vector(1, 2, 3, 4, 5)
// res41: Vector[Int] = Vector(6, 1, 2, 3, 4, 5)

Vector(1, 2, 3, 4, 5) :+ 6
// res42: Vector[Int] = Vector(1, 2, 3, 4, 5, 6)

Vector(1, 2, 3, 4, 5).updated(2, 5)
// res43: Vector[Int] = Vector(1, 2, 5, 4, 5)
```

13.2　Map

Map はキーから値へのマッピングを提供するデータ構造です。他の言語では辞書や連想配列と呼ばれたりします。Scala では Map として一度作成したら変更できない immutable な Map と変更可能な mutable な Map の 2 種類を提供しています。

13.2.1　scala.collection.immutable.Map

Scala で何も設定せずにただ Map と書いた場合、scala.collection.immutable.Map が使われます。その名の通り、一度作成すると変更することはできません。内部の実装としては主に scala.collection.immutable.HashMap と scala.collection.immutable.TreeMap の 2 種類がありますが、通常は HashMap が使われます。

```
val m = Map("A" -> 1, "B" -> 2, "C" -> 3)
// m: Map[String, Int] = Map("A" -> 1, "B" -> 2, "C" -> 3)

m.updated("B", 4) //????Map???????????
// res44: Map[String, Int] = Map("A" -> 1, "B" -> 4, "C" -> 3)

m // ??Map?????
// res45: Map[String, Int] = Map("A" -> 1, "B" -> 2, "C" -> 3)
```

13.2.2　scala.collection.mutable.Map

Scala の変更可能な Map は scala.collection.mutable.Map にあります。実装としては、scala.collection.mutable.HashMap 、scala.collection.mutable.LinkedHashMap 、リストをベースにした scala.collection.mutable.ListMap がありますが、通常は HashMap が使われます。

```
import scala.collection.mutable

val m = mutable.Map("A" -> 1, "B" -> 2, "C" -> 3)
// m: Map[String, Int] = HashMap("A" -> 1, "B" -> 5, "C" -> 3)

m("B") = 5 // B -> 5 ????????????

m // ??????????
// res47: Map[String, Int] = HashMap("A" -> 1, "B" -> 5, "C" -> 3)
```

13.3　Set

Set は値の集合を提供するデータ構造です。Set の中では同じ値が 2 つ以上存在しません。たとえば、Int の Set の中には 1 が 2 つ以上含まれていてはいけません。REPL で Set を作成するための式

を入力すると、

```
Set(1, 1, 2, 3, 4)
// res48: Set[Int] = Set(1, 2, 3, 4)
```

重複した 1 が削除されて、1 が 1 つだけになっていることがわかります。

13.3.1　scala.collection.immutable.Set

Scala で何も設定せずにただ Set と書いた場合、scala.collection.immutable.Set が使われます。immutable な Map の場合と同じく、一度作成すると変更することはできません。内部の実装としては、主に scala.collection.immutable.HashSet と scala.collection.immutable.TreeSet の 2 種類がありますが、通常は HashSet が使われます。

```
val s = Set(1, 2, 3, 4, 5)
// s: Set[Int] = Set(5, 1, 2, 3, 4)

s - 5 // 5???????
// res49: Set[Int] = Set(1, 2, 3, 4)

s // ??Set?????
// res50: Set[Int] = Set(5, 1, 2, 3, 4)
```

13.3.2　scala.collection.mutable.Set

Scala の変更可能な Set は scala.collection.mutable.Set にあります。主な実装としては、scala.collection.mutable.HashSet 、scala.collection.mutable.TreeSet がありますが、通常は HashSet が使われます。

```
import scala.collection.mutable

val s = mutable.Set(1, 2, 3, 4, 5)
// s: Set[Int] = HashSet(1, 2, 3, 4)

s -= 5 // 5 ??????
// res51: Set[Int] = HashSet(1, 2, 3, 4)

s // ????????
// res52: Set[Int] = HashSet(1, 2, 3, 4)
```

13.3.3　その他資料

さらにコレクションライブラリについて詳しく知りたい場合は、以下の公式のドキュメントなどを読みましょう https://docs.scala-lang.org/ja/overviews/collections/introduction.html

第 14 章

ケースクラスとパターンマッチング

パターンマッチングは、Scala をはじめとする関数型言語に一般的な機能です。C や Java の switch 文に似ていますが、より強力な機能です。しかし、パターンマッチングの真価を発揮するには、標準ライブラリまたはユーザが定義したケースクラス（case class）によるデータ型の定義が必要になります。

簡単なケースクラスによるデータ型を定義してみます。

```
sealed abstract class DayOfWeek
case object Sunday extends DayOfWeek
case object Monday extends DayOfWeek
case object Tuesday extends DayOfWeek
case object Wednesday extends DayOfWeek
case object Thursday extends DayOfWeek
case object Friday extends DayOfWeek
case object Saturday extends DayOfWeek
```

これは、一週間の曜日を表すデータ型です。C や Java の `enum` に似ていますね。実際、同じように使うことができます。

また、Scala 3 からは enum という新しい機能が追加されたため、ほぼ同等のものを以下のように簡潔に書くことが可能です。

```
enum DayOfWeekScala3Enum {
  case Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday
}
```

たとえば、以下のように `DayOfWeek` 型の変数に `Sunday` を代入することができます。

```
val x: DayOfWeek = Sunday
```

`object` またはその他のデータ型は、パターンマッチングのパターンを使うことができます。この例では、この `DayOfWeek` 型を継承した各 `object` をパターンマッチングのパターンを使うことができます。パターンマッチングの構文は再度書くと、

```
<対象式> match {
  (case <パターン> (if <ガード>)? '=>'
    (<式> (;|<改行>))*
  )+
}
```

のようになります。DayOfWeek の場合、次のようにして使うことができます。

```
x match {
  case Sunday => 1
  case Monday => 2
  case Tuesday => 3
  case Wednesday => 4
  case Thursday => 5
  case Friday => 6
}
// res0: Int = 1
```

これは、x が Sunday なら 1 を、Monday なら 2 を...返すパターンマッチです。Saturday に対する場合分けが記述されていませんが、パターンマッチの漏れはコンパイラが警告してくれます。

```
warning: match may not be exhaustive.
It would fail on the following input: Saturday
```

この警告は、sealed 修飾子をスーパークラス/トレイトに付けることによって、その（直接の）サブクラス/トレイトは同じファイル内にしか定義できないという性質を利用して実現されています。この用途以外で sealed はめったに使われないので、ケースクラスのスーパークラス/トレイトには sealed を付けるものだと覚えておけば良いでしょう。

これだけだと、C や Java の列挙型とあまり変わらないように見えますが、それらと異なるのは各々のデータは独立してパラメータを持つことができることです。また、パターンマッチの際はそのデータ型の種類によって分岐するだけでなく、データを分解することができることが特徴です。

例として四則演算を表す構文木を考えてみます。各ノード Exp を継承し（つまり、全てのノードは式である）、二項演算を表すノードはそれぞれの子として lhs（左辺）、rhs（右辺）を持つこととします。葉ノードとして整数リテラル（Lit ）も入れます。これは Int の値を取るものとします。また、二項演算の結果として小数が現れた場合は小数部を切り捨てることとします。これを表すデータ型を Scala で定義すると次のようになります。

```
sealed abstract class Exp

object Exp {
  case class Add(lhs: Exp, rhs: Exp) extends Exp
  case class Sub(lhs: Exp, rhs: Exp) extends Exp
  case class Mul(lhs: Exp, rhs: Exp) extends Exp
  case class Div(lhs: Exp, rhs: Exp) extends Exp
  case class Lit(value: Int) extends Exp
}
```

こちらも Scala 3 の enum を使うと以下のように簡潔に書くことが可能です。

```
enum Exp {
  case Add(lhs: Exp, rhs: Exp)
  case Sub(lhs: Exp, rhs: Exp)
  case Mul(lhs: Exp, rhs: Exp)
  case Div(lhs: Exp, rhs: Exp)
  case Lit(value: Int)
}
```

全てのデータ型に case 修飾子がついているので、これらのデータ型はパターンマッチングのパターンとして使うことができます。この定義から、1 + ((2 * 3) / 2) という式を表すノードを構築します。

```
import Exp.*

val example = Add(Lit(1), Div(Mul(Lit(2), Lit(3)), Lit(2)))
// example: Exp = Add(
//   lhs = Lit(1),
//   rhs = Div(lhs = Mul(lhs = Lit(2), rhs = Lit(3)), rhs = Lit(2))
// )
```

この example ノードを元に四則演算を定義する関数を定義してみます。関数の定義の詳細は後ほど説明しますが、ここでは雰囲気だけをつかんでください。

```
def eval(exp: Exp): Int = exp match {
  case Add(l, r) => eval(l) + eval(r)
  case Sub(l, r) => eval(l) - eval(r)
  case Mul(l, r) => eval(l) * eval(r)
  case Div(l, r) => eval(l) / eval(r)
  case Lit(v) => v
}
```

この定義を REPL に読み込ませて、eval(example) として、

```
eval(example)
// res1: Int = 4
```

のように表示されれば成功です。きちんと、1 + ((2 * 3) / 2) という式の計算結果が出ていますね。ここで注目すべきは、パターンマッチングによって、

1. ノードの種類と構造によって分岐する
2. ネストしたノードを分解する
3. ネストしたノードを分解した結果で変数を束縛する

という 3 つの動作が同時に行えていることです。これがケースクラスを使ったデータ型とパターンマッチングの組み合わせの強力さです。また、この match 式の中で、たとえば case Lit(v) => v の行を書き忘れた場合、DayOfWeek の例と同じように、

```
<console>:16: warning: match may not be exhaustive.
It would fail on the following input: Lit(_)
       def eval(exp: Exp): Int = exp match {
```

記述漏れがあることを指摘してくれますから、ミスを防ぐこともできます。

14.1　変数宣言におけるパターンマッチング

match 式中のパターンマッチングのみを扱ってきましたが、実は変数宣言でもパターンマッチングを行うことができます。

たとえば、次のようなケースクラス Point があったとします。

```
case class Point(x: Int, y: Int)
```

このケースクラス Point に対して、

```
val Point(x, y) = Point(10, 20)
// x: Int = 10
// y: Int = 20
```

とすると、x が 10 に、y が 20 に束縛されます。もしパターンにマッチしなかった場合は、例外 scala.MatchError が発生してしまうので、変数宣言におけるパターンマッチングは、それが必ず成功すると型情報から確信できる場合にのみ使うようにしましょう。

14.2　ケースクラスによって自動生成されるもの

オブジェクトの章で少し触れましたが、ケースクラスはクラスに対して、いくらか追加の自動生成を行います。このテキストでは特に重要なものについて触れます。

- プライマリコンストラクタ引数を公開します（val を付けたかのように扱われる）
- インスタンス間の同値比較が行えるように equals()・hashCode()・canEqual() が定義されます
  - つまり、クラスが同じで、プライマリコンストラクタ引数の値すべてが一致しているかどうかで同値判定するようになります
- 型とプライマリコンストラクタ引数を使った toString() が定義されます
- コンパニオンオブジェクトにプライマリコンストラクタ引数と対応する apply() が定義されます

ラフに言うと、タプルに近い感覚でクラスを操作できるようになるということです。REPL での実行例を示します。

```
scala> case class Point(x: Int, y: Int)
```

```
defined class Point

scala> val p = Point(1, 2)
p: Point = Point(1,2)

scala> println(p.x, p.y)
(1,2)

scala> Point(1, 2) == Point(1, 2)
res4: Boolean = true

scala> Point(1, 2) == Point(3, 4)
res5: Boolean = false
```

本節で紹介しているケースクラスの機能は、あくまで便利さ簡潔さのためだけのものです。クラスに実装を足すことでも同等の振る舞いを持たせることもできます。例えば、前節で定義した `Point` をケースクラスを使わずに定義するとしたら、次のようになるでしょう。

```scala
class Point(val x: Int, val y: Int) {
  override def equals(that: Any): Boolean = that match {
    case thatPoint: Point =>
      thatPoint.canEqual(this) && this.x == thatPoint.x && this.y == thatPoint.y
    case _ =>
      false
  }

  override def hashCode(): Int = x.hashCode ^ y.hashCode

  def canEqual(that: Any): Boolean = that.isInstanceOf[Point]

  override def toString(): String = "Point(" + x + ", " + y + ")"
}

object Point {
  def apply(x: Int, y: Int): Point = new Point(x, y)
}
```

比べてみると、ケースクラスによって大幅に記述量が減っていることがわかると思います。`Point` が典型例でしたが、一般に、データ構造のようなものを表すクラスをケースクラスにすると便利なことがあります。

また、比較周りの実装が想像よりずっと複雑だと思った方もいるかもしれません。比較を「期待通り」に実装するのは、比較対象との継承関係を考慮する必要があるなど、それなりに難しいことが知られています。しかも、クラスのフィールド変数が増えるなど、クラスが何か変化するたびに修正が発生しかねない箇所でもあります。そういう問題を考えなくて済む点でもケースクラスは便利です。

最後に改めて述べておきたいと思いますが、ケースクラスでは紹介したもの以外にもメソッド定義が増えます。例えば、パターンマッチの実現機構である `unapply()` や、複製のための `copy()` については触れませんでした。興味のある方はぜひ調べてみてください。

14.3　練習問題

DayOfWeek 型を使って、ある日の次の曜日を返すメソッド nextDayOfWeek

```
def nextDayOfWeek(d: DayOfWeek): DayOfWeek = ???
```

をパターンマッチを用いて定義してみましょう。

```
def nextDayOfWeek(d: DayOfWeek): DayOfWeek = d match {
  case Sunday => Monday
  case Monday => Tuesday
  case Tuesday => Wednesday
  case Wednesday => Thursday
  case Thursday => Friday
  case Friday => Saturday
  case Saturday => Sunday
}
```

```
nextDayOfWeek(Sunday)
// res2: DayOfWeek = Monday
nextDayOfWeek(Monday)
// res3: DayOfWeek = Tuesday
nextDayOfWeek(Saturday)
// res4: DayOfWeek = Sunday
```

14.4　練習問題

二分木（子の数が最大で 2 つであるような木構造）を表す型 Tree と Branch , Empty を考えます：

```
sealed abstract class Tree
case class Branch(value: Int, left: Tree, right: Tree) extends Tree
case object Empty extends Tree
```

子が 2 つで左の子の値が 2 、右の子の値が 3 、自分自身の値が 1 の木構造はたとえば次のようにして定義することができます。

```
val tree: Tree = Branch(1, Branch(2, Empty, Empty), Branch(3, Empty, Empty))
// tree: Tree = Branch(
//   value = 1,
//   left = Branch(value = 2, left = Empty, right = Empty),
//   right = Branch(value = 3, left = Empty, right = Empty)
// )
```

このような木構造に対して、

1. 最大値を求める max メソッド：

2. 最小値を求める min メソッド：
3. 深さを求める depth メソッド：

```
def max(tree: Tree): Int = ???
def min(tree: Tree): Int = ???
def depth(tree: Tree): Int = ???
```

をそれぞれ定義してみましょう。なお、

```
depth(Empty) == 0
depth(Branch(10, Empty, Empty)) == 1
depth(Branch(10, Branch(20,
                    Empty,
                    Empty
                ), Empty)) == 2
// 右のBranchの方が、左のBranchよりも深い
depth(Branch(10, Branch(20,
                    Empty,
                    Empty
                ), Branch(30,
                    Branch(40,
                        Empty,
                        Empty
                    ),
                Empty))) == 3
```

です。

余裕があれば木構造を、

```
左の子孫の全ての値 <= 自分自身の値 < 右の子孫の全部の値
```

となるような木構造に変換する sort メソッド：

```
def sort(tree: Tree): Tree = ???
```

を定義してみましょう。なお、sort メソッドは、葉ノードでないノードの個数と値が同じであれば元の構造と同じでなくても良いものとします。

```
object BinaryTree {
  sealed abstract class Tree
  case class Branch(value: Int, left: Tree, right: Tree) extends Tree
  case object Empty extends Tree

  def max(t: Tree): Int = t match {
    case Branch(v, Empty, Empty) =>
      v
    case Branch(v, Empty, r) =>
      val x = max(r)
      if(v > x) v else x
    case Branch(v, l, Empty) =>
      val x = max(l)
```

```
14        if(v > x) v else x
15      case Branch(v, l, r) =>
16        val x = max(l)
17        val y = max(r)
18        if(v > x) {
19          if(v > y) v else y
20        } else {
21          if(x > y) x else y
22        }
23      case Empty =>
24        throw new RuntimeException
25    }
26
27
28    def min(t: Tree): Int = t match {
29      case Branch(v, Empty, Empty) =>
30        v
31      case Branch(v, Empty, r) =>
32        val x = min(r)
33        if(v < x) v else x
34      case Branch(v, l, Empty) =>
35        val x = min(l)
36        if(v < x) v else x
37      case Branch(v, l, r) =>
38        val x = min(l)
39        val y = min(r)
40        if(v < x) {
41          if(v < y) v else y
42        } else {
43          if(x < y) x else y
44        }
45      case Empty =>
46        throw new RuntimeException
47    }
48
49    def depth(t: Tree): Int = t match {
50      case Empty => 0
51      case Branch(_, l, r) =>
52        val ldepth = depth(l)
53        val rdepth = depth(r)
54        (if(ldepth < rdepth) rdepth else ldepth) + 1
55    }
56
57    def toList(tree: Tree): List[Int] = tree match {
58      case Empty => Nil
59      case Branch(v, l, r) => toList(l) ++ List(v) ++ toList(r)
60    }
61
62    def sort(t: Tree): Tree = {
63      def fromList(list: List[Int]): Tree = {
64        def insert(value: Int, t: Tree): Tree = t match {
65          case Empty => Branch(value, Empty, Empty)
66          case Branch(v, l, r) =>
```

```
67            if(value <= v) Branch(v, insert(value, l), r)
68            else Branch(v, l, insert(value, r))
69        }
70        list.foldLeft(Empty:Tree){ case (t, v) => insert(v, t) }
71      }
72      fromList(toList(t))
73    }
74
75  }
```

14.5　部分関数

これまでの説明の中で、無名関数とパターンマッチングについて説明してきましたが、この 2 つの機能を組み合わせた部分関数（PartialFunction）が Scala には存在します。説明の前に、まず、具体的なユースケースを挙げます：

```
1  List(1, 2, 3, 4, 5).collect { case i if i % 2 == 1 => i * 2 }
2  // res7: List[Int] = List(2, 6, 10)
```

ここで、collect メソッドは pf: PartialFunction[A, B] を引数に取り、pf.isDefinedAt(i) が true になる要素のみを残し、さらに、pf.apply(i) の結果の値を元にした新しいコレクションを返します。

isDefinedAt は

```
1  i % 2 == 1
```

の部分から自動的に生成され、パターンがマッチするときのみ真になるように定義されます。collect はこの isDefinedAt メソッドを使うことで、filter と map に相当する処理を同時に行うことができています。

PartialFunction は、自分でクラスを継承して作ることも可能ですが、一般的には、

```
1  {
2    case pat1 => exp1
3    case pat2 => exp2
4    ...
5    case patn => expn
6  }
```

という形の式から、自動的に生成されます。isDefinedAt が真になる条件は、pat1 ... patn のいずれかの条件にマッチすることです。

PartialFunction を使う機会は実用的にはそれほど多いわけではありませんが、collect やサードパーティのライブラリで使うことがしばしばあるので、覚えておくと良いでしょう。

注意点として、`{ case .... }` という形式は、あくまで `PartialFunction` 型が要求されているときにのみ `PartialFunction` が生成されるのであって、通常の `FunctionN` 型が要求されたときには、違う意味を持つということです。たとえば、以下のような定義があったとします。

```
val even: Int => Boolean = {
  case i if i % 2 == 0 => true
  case _ => false
}
// even: Function1[Int, Boolean] = repl.MdocSession$MdocApp$$Lambda$13686/0
    x0000743a8df16410@132deca0
```

このとき、この定義は、無名関数とパターンマッチを組み合わせたものと同じ意味になります。この点にだけ注意してください。

```
val even: Int => Boolean = (x => x match {
  case i if i % 2 == 0 => true
  case _ => false
})
// even: Function1[Int, Boolean] = repl.MdocSession$MdocApp$$Lambda$13687/0
    x0000743a8df16a20@47f9626a
```

第 15 章

エラー処理

ここでは Scala におけるエラー処理の基本を学びます。Scala でのエラー処理は例外を使う方法と、Option や Either や Try などのデータ型を使う方法があります。この 2 つの方法はどちらか一方だけを使うわけではなく、状況に応じて使いわけることになります。

まずは私たちが扱わなければならないエラーとエラー処理の性質について確認しましょう。

15.1 エラーとは

プログラムにとって、エラーというものにはどういったものがあるのか考えてみます。

15.1.1 ユーザーからの入力

1 つはユーザーから受け取る不正な入力です。たとえば以下のようなものが考えられます。

- 文字数が長すぎる
- 電話番号に文字列を使うなど、正しいフォーマットではない
- 既に登録されているユーザー名を使おうとしている

など色々な問題が考えられます。また悪意のある攻撃者から攻撃を受けることもあります。

- アクセスを制限しているデータを見ようとしている
- ログインセッションの Cookie を改変する
- 大量にアクセスをおこない、システムを利用不能にしようとする

基本的に外から受け取るデータはすべてエラーの原因となりえるので注意が必要です。

15.1.2　外部サービスのエラー

自分たちのプログラムが利用する外部サービスのエラーも考えられます。

- Twitter や Facebook に投稿しようとしても繋がらない
- iPhone や Android と通信しようとしても回線の都合で切れてしまう
- ユーザーにメールを送信しようとしても失敗する

以上のように外部のサービスを使わなければならないような処理はすべて失敗することを想定したほうがいいでしょう。

15.1.3　内部のエラー

外的な要因だけではなく、内部の要因でエラーが発生することもあります。

- ライブラリのバグや自分たちのバグにより、プログラム全体が終了してしまう
- MySQL や Redis などの内部で利用しているサーバーが終了してしまう
- メモリやディスク容量が足りない
- 処理に非常に大きな時間がかかってしまう

内部のエラーは扱うことが難しい場合が多いですが、起こりうることは念頭に置くべきです。

15.2　エラー処理で実現しなければならないこと

以上のようなエラーに対して、私たちが行わなければいけないことを挙げてみます。

15.2.1　例外安全性

エラー処理の中の 1 つの例外処理には「例外安全性」という概念があります。例外が発生してもシステムがダウンしたり、データの不整合などの問題が起きない場合、例外安全と言います。

この概念はエラー処理全般にもあてはまります。私たちが作るプログラムを継続的に動作させたいと考えた場合、ユーザーの入力や外部サービスの問題により、システムダウンやデータの不整合が起きてはなりません。これがエラー処理の第一の目的になります。

15.2.2　強い例外安全性

例外安全性にはさらに強い概念として「強い例外安全性」というものがあります。これは例外が発生した場合、すべての状態が例外発生前に戻らなければならないという制約です。一般的にはこの制約を満たすことは難しいのですが、たとえばユーザーがサービスに課金して、何らかのエラーが生じた場合、確実にエラーを検出し、課金処理を取り消さなければなりません。どのような処理に強い例

外安全性が求められるか判断し、どのように実現するかを考える必要があります。

15.3　Java におけるエラー処理

Java のエラー処理の方法は Scala にも適用できるものが多いです。ここでは Java のエラー処理の注意点についていくつか復習しましょう。

15.3.1　null を返すことでエラーを表現する場合の注意点

Java では、変数が未初期化である場合や、コレクションライブラリが空なのに要素を取得しようとした場合など、null でエラーを表現することがあります。Java はプリミティブ型以外の参照型はすべて null にすることができます。この性質はエラー値を他に用意する必要がないという点では便利なのですが、しばしば返り値を null かどうかチェックするのを忘れて実行時エラーの NullPointerException（通称：ぬるぽ・NPE）を発生させてしまいます。（「ぬるぽ」と「ガッ」というやりとりをする 2ch の文化の語源でもあります）

参照型がすべて `null` になりうるということは、メソッドが `null` が返されるかどうかはメソッドの型からはわからないので、Java のメソッドで `null` を返す場合はドキュメントに書くようにしましょう。そして、`null` をエラー値に使うエラー処理は暗黙的なエラー状態をシステムのいたるところに持ち込むことになり、発見困難なバグを生む要因になります。後述しますが、Scala では Option というデータ構造を使うことでこの問題を解決します。

15.3.2　例外を投げる場合の注意点

Java のエラー処理で中心的な役割を果たすのが例外です。例外は今実行している処理を中断し、大域的に実行を移動できる便利な機能ですが、濫用することで処理の流れがわかりづらいコードにもなります。例外はエラー状態にのみ利用し、メソッドが正常な値を返す場合には使わないようにしましょう。

15.3.3　チェック例外の注意点

Java にはメソッドに `throws` 節を付けることで、メソッドを使う側に例外を処理することを強制するチェック例外という機能もあります。チェック例外は例外の発生を表現し、コンパイラにチェックさせるという点で便利な機能ですが、上げられた例外の catch 処理はわずらわしいものにもなりえます。使う側が適切に処理できない例外を上げられた場合はあまり意味のないエラー処理コードを書かざるをえません。よってチェック例外は利用側が catch して適切にエラー状態から回復できる場合にのみ利用したほうがいいでしょう。

15.3.4　例外翻訳の注意点

Java の例外は実装の変更により変化する場合があります。たとえば今まで HTTP で取得していたデータを MySQL に保存したとしましょう。その場合、今までは HTTPException が投げられていたものが、SQLException が投げられるようになるかもしれません。すると、この例外を catch する側も HTTPException ではなく SQLException を扱うようにしなければなりません。このように低レベルの実装の変更がプログラム全体に影響することがありえます。

そのような問題を防ぐために途中の層で一度例外を catch し、適切な例外で包んでもう一度投げる手法があります。このことを例外翻訳と呼びます。例外翻訳は例外に対する情報を増やし、catch する側の影響も少なくする手法です。ただし、この例外翻訳も乱用すると例外の種類が増えて例外処理が煩雑になる可能性もあるので注意が必要です。

15.3.5　例外をドキュメントに書く

例外はチェック例外でない場合、API から読み取ることができません。さらに後述しますが Scala ではチェック例外がないので、メソッドの型からどんな例外を投げるかは判別できません。そのため API ドキュメントには発生しうる例外についても書いておいたほうが良いでしょう。

15.4　例外の問題点

Java のエラー処理では例外が中心的な役割を担っていましたが、Scala でも例外は多く使われます。しかし、例外は便利な反面、様々な問題もあります。ここで例外の問題点を把握し、適切に使えるようになりましょう。

15.4.1　例外を使うと制御の流れがわかりづらくなる

先ほど述べたように例外は、適切に使えば正常系の処理とエラー処理を分離し、コードの可読性を上げ、エラー処理をまとめる効果があります。しかし、往々にして例外の catch 漏れが発生し、障害に繋がることがあります。逆に例外を catch しているところで、どこで発生した例外を catch しているのか判別できないために、コードの修正を阻害する場合もあります。

15.4.2　例外は非同期プログラミングでは扱いづらい

例外の基本メカニズムは、送出されたら catch されるまで（同一スレッドの）コールスタックを遡っていくというものです。これは、素直に考えると別スレッドで発生した例外を取り扱うことが難しいということを意味しています。別スレッドで発生した例外を取り扱うメカニズムを考えることも可能ですが、既存の例外の仕組みをそのまま使えないことは確かです。特に Scala では非同期プログラミングが多用されるので、例外をそのまま使えないことが多いです。

15.4.3　例外は型チェックできない

チェック例外を使わない限り、どんな例外が発生するのかメソッドの型としては表現されません。また catch する側でも間違った例外をキャッチしているかどうかは実行時にしかわかりません。例外に頼りすぎると静的型付き言語の利点が損われます。

15.4.4　チェック例外の問題点

チェック例外を使わないとコンパイル時に型チェックできないわけですが、Scala では Java とは違いチェック例外の機能はなくなりました。これにはチェック例外の様々な問題点が理由としてあると思います

- 高階関数でチェック例外を扱うことが難しい
- ボイラープレートが増える
- 例外によるメソッド型の変更を防ぐために例外翻訳を多用せざるをえない

特に Scala では 1 番目の問題が大きいと思います。後述しますが、Scala ではチェック例外の代替手段として、エラーを表現するデータ型を使い、エラー処理を型安全にすることもできます。それらを考えると Scala でチェック例外をなくしたのは妥当な判断と言えるでしょう。

15.5　エラーを表現するデータ型を使った処理

例外に問題があるとすれば、どのようにエラーを扱えばよいのでしょうか。その答えの 1 つはエラーを例外ではなく、メソッドの返り値で返せるような値にすることです。

ここでは正常の値とエラー値のどちらかを表現できるデータ構造の紹介を通じて、Scala の関数型のエラー処理の方法を見ていきます。

15.5.1　Option

Option は Scala でもっとも多用されるデータ型の 1 つです。前述のとおり Java の null の代替として使われることが多いデータ型です。

Option 型は簡単に言うと、値を 1 つだけいれることのできるコンテナです。ただし、Option のまま様々なデータ変換処理ができるように便利な機能を持ちあわせています。

15.5.1.1　Option の作り方と値の取得

では具体的に Option の作り方と値の取得を見てみましょう。Option 型には具体的には

- `Some`
- `None`

以上 2 つの具体的な値が存在します。Some は何かしらの値が格納されている時の Option の型、None は値が何も格納されていない時の Option の型です。

具体的な動きを見てみましょう。Option に具体的な値が入った場合は以下の様な動きをします。

```
val o: Option[String] = Option("hoge")
// o: Option[String] = Some("hoge")

o.get
// res0: String = "hoge"

o.isEmpty
// res1: Boolean = false

o.isDefined
// res2: Boolean = true
```

今度は null を Option に入れるとどうなるでしょうか。

```
val o: Option[String] = Option(null)
// o: Option[String] = None

o.isEmpty
// res3: Boolean = true

o.isDefined
// res4: Boolean = false
```

```
o.get

// java.util.NoSuchElementException: None.get
//  at scala.None$.get(Option.scala:628)
//  at scala.None$.get(Option.scala:628)
//  at repl.MdocSession$MdocApp.$init$$$anonfun$1(error-handling.md:48)
```

Option のコンパニオンオブジェクトの apply には引数が null であるかどうかのチェックが入っており、引数が null の場合、値が None になります。get メソッドを叩いた時に、java.util.NoSuchElementException という例外が起こっているので、これが NPE と同じだと思うかもしれません。しかし Option には以下の様な便利メソッドがあり、それらを回避することができます。

```
o.getOrElse("")
// res5: String = ""
```

以上は Option[String] の中身が None だった場合に、空文字を返すというコードになります。値以外にも処理を書くこともできます。

```
o.getOrElse(throw new RuntimeException("null??????????"))
// java.lang.RuntimeException: null??????????
//  at repl.MdocSession$MdocApp.$init$$$anonfun$2$$anonfun$1(error-handling.md:66)
//  at scala.Option.getOrElse(Option.scala:203)
```

```
//  at repl.MdocSession$MdocApp.$init$$$anonfun$2(error-handling.md:66)
```

このように書くこともできるのです。

15.5.1.2　Option のパターンマッチ

上記では、手続き的に Option を処理しましたが、型を持っているためパターンマッチを使って処理することもできます。

```
val s: Option[String] = Some("hoge")
// s: Option[String] = Some("hoge")

val result = s match {
  case Some(str) => str
  case None => "not matched"
}
// result: String = "hoge"
```

上記のように Some か None にパターンマッチを行い、Some にパターンマッチする場合には、その中身の値で `str` という別の変数を束縛することもできます。

中身を取りだすのではなく、中身で変数を束縛するというテクニックは、`List` のパターンマッチでも行うことができますが、全く同様のことが Option でもできます。

15.5.1.3　Option に関数を適用する

Option には、コレクションの性質があると言いましたが、関数を内容の要素に適用できるという性質もそのまま持ち合わせています。

```
Some(3).map(_ * 3)
// res6: Option[Int] = Some(9)
```

このように、`map` で関数を適用する事もできます。なお、値が `None` の場合にはどうなるでしょうか。

```
val n: Option[Int] = None
// n: Option[Int] = None

n.map(_ * 3)
// res7: Option[Int] = None
```

None のままだと型情報を持たないので一度、変数にしていますが、None に 3 をかけるという関数を適用しても None のままです。この性質はとても便利で、その値が Option の中身が Some なのか None なのかどちらであったとしても、同様の処理で記述でき、処理を分岐させる必要がないのです。

Java 風に書くならば、

```
if (n.isDefined) {
  n.get * 3
} else {
```

```
  throw new RuntimeException
}
// java.lang.RuntimeException
//  at repl.MdocSession$MdocApp.$init$$$anonfun$3(error-handling.md:113)
```

きっと上記のように書くことになっていたでしょう。ただ、よくよく考えると上記の Java 風に書いた例と map の例は異なることに気が付きます。map では、値が Some の場合は中身に関数を適用しますが、None の時には何も実行しません。上記の例では例外を投げています。そして、値も Int 型の値を返していることも異なっています。

このように、None の場合に実行し、値を返す関数を定義できるのが `fold` です。`fold` の宣言を Scala の API ドキュメントから引用すると、

```
fold[B](ifEmpty: => B)(f: (A) => B): B
```

となります。

そして関数を適用した値を最終的に取得できます。

```
n.fold(throw new RuntimeException)(_ * 3)
// java.lang.RuntimeException
//  at repl.MdocSession$MdocApp.$init$$$anonfun$4$$anonfun$1(error-handling.md:124)
//  at scala.Option.fold(Option.scala:265)
//  at repl.MdocSession$MdocApp.$init$$$anonfun$4(error-handling.md:124)
```

上記のように書くことで、None の際に実行する処理を定義し、かつ、関数を適用した中身の値を取得することができます。

```
Some(3).fold(throw new RuntimeException)(_ * 3)
// res8: Int = 9
```

`Some(3)` を与えるとこのように Int の 9 の値を返すことがわかります。

15.5.1.4　Option の入れ子を解消する

実際の複雑なアプリケーションの中では、Option の値が取得されることがよくあります。

たとえばキャッシュから情報を取得する場合は、キャッシュヒットする場合と、キャッシュミスする場合があり、それらは Scala ではよく Option 型で表現されます。

このようなキャッシュ取得が連続して繰り返された場合はどうなるでしょうか。例えば、1 つ目と 2 つ目の整数の値が Option で返ってきてそれをかけた値をもとめるような場合です。

```
val v1: Option[Int] = Some(3)
// v1: Option[Int] = Some(3)

val v2: Option[Int] = Some(5)
// v2: Option[Int] = Some(5)

v1.map(i1 => v2.map(i2 => i1 * i2))
// res9: Option[Option[Int]] = Some(Some(15))
```

map だけを使ってシンプルに実装するとこんな風になってしまいます。ウウッ...、悲しいことに `Option[Option[Int]]` のように Option が入れ子になってしまいます。

このような入れ子の option を解消するために用意されているのが、`flatten` です。

```
v1.map(i1 => v2.map(i2 => i1 * i2)).flatten
// res10: Option[Int] = Some(15)
```

最後に `flatten` を実行することで、Option の入れ子を解消することができます。なお、v2 が None である場合にも `flatten` は成立します。

```
val v1: Option[Int] = Some(3)
// v1: Option[Int] = Some(3)

val v2: Option[Int] = None
// v2: Option[Int] = None

v1.map(i1 => v2.map(i2 => i1 * i2)).flatten
// res11: Option[Int] = None
```

つまり、キャッシュミスで Some の値が取れなかった際も問題なくこの処理で動きます。

15.5.1.5　練習問題

`map` と `flatten` を利用して、`Some(2)` と `Some(3)` と `Some(5)` と `Some(7)` と `Some(11)` の値をかけて、`Some(2310)` を求めてみましょう。

```
val v1: Option[Int] = Some(2)
val v2: Option[Int] = Some(3)
val v3: Option[Int] = Some(5)
val v4: Option[Int] = Some(7)
val v5: Option[Int] = Some(11)
v1.map { i1 =>
    v2.map { i2 =>
        v3.map { i3 =>
            v4.map { i4 =>
                v5.map { i5 => i1 * i2 * i3 * i4 * i5 }
            }.flatten
        }.flatten
    }.flatten
}.flatten
```

15.5.2　flatMap

ここまでで、`map` と `flatten` を話しましたが、実際のプログラミングではこの両方を組み合わせて使うということが多々あります。そのためその 2 つを適用してくれる `flatMap` というメソッドが Option には用意されています。名前は `flatMap` なのですが、意味としては Option に `map` をかけて `flatten` を適用してくれます。

実際に先ほどの、Some(3) と Some(5) をかける例で利用してみると以下のようになります。

```
val v1: Option[Int] = Some(3)
// v1: Option[Int] = Some(3)

val v2: Option[Int] = Some(5)
// v2: Option[Int] = Some(5)

v1.flatMap(i1 => v2.map(i2 => i1 * i2))
// res13: Option[Int] = Some(15)
```

ずいぶんシンプルに書くことができるようになります。

Some(3) と Some(5) と Some(7) をかける場合はどうなるでしょうか。

```
val v1: Option[Int] = Some(3)
// v1: Option[Int] = Some(3)

val v2: Option[Int] = Some(5)
// v2: Option[Int] = Some(5)

val v3: Option[Int] = Some(7)
// v3: Option[Int] = Some(7)

v1.flatMap(i1 => v2.flatMap(i2 => v3.map(i3 => i1 * i2 * i3)))
// res14: Option[Int] = Some(105)
```

無論これは、v1 , v2 , v3 のいずれが None であった場合にも成立します。その場合には flatten の時と同様に None が最終的な答えになります。

```
val v3: Option[Int] = None
// v3: Option[Int] = None

v1.flatMap(i1 => v2.flatMap(i2 => v3.map(i3 => i1 * i2 * i3)))
// res15: Option[Int] = None
```

以上のようになります。

15.5.2.1　練習問題

flatMap と map を利用して、Some(2) と Some(3) と Some(5) と Some(7) と Some(11) の値をかけて、Some(2310) を求めてみましょう。

```
val v1: Option[Int] = Some(2)
val v2: Option[Int] = Some(3)
val v3: Option[Int] = Some(5)
val v4: Option[Int] = Some(7)
val v5: Option[Int] = Some(11)
v1.flatMap { i1 =>
    v2.flatMap { i2 =>
        v3.flatMap { i3 =>
            v4.flatMap { i4 =>
```

```
                v5.map { i5 => i1 * i2 * i3 * i4 * i5 }
            }
        }
    }
}
```

一見してわかるように、かける値が増えるとネストが増えてあまり読みやすくありません。次に説明する for 式によって、より綺麗に書けるようになります。

15.5.3　for を利用した flatMap のリファクタリング

Option はコレクションのようなものだという風に言いましたが、for を Option に使うこともできます。for 式は実際には `flatMap` と `map` 展開されて実行されるのです。

何をいっているのかわかりにくいと思いますので、先ほどの `Some(3)` と `Some(5)` と `Some(7)` を flatMap でかけるという処理を for で書いてみましょう。

```
val v1: Option[Int] = Some(3)
// v1: Option[Int] = Some(3)

val v2: Option[Int] = Some(5)
// v2: Option[Int] = Some(5)

val v3: Option[Int] = Some(7)
// v3: Option[Int] = Some(7)

for { i1 <- v1
      i2 <- v2
      i3 <- v3 } yield i1 * i2 * i3
// res17: Option[Int] = Some(105)
```

実はこの for 式は先ほどの `flatMap` と `map` で書かれたものとまったく同じ動作をします。`flatMap` と `map` を複数回使うような場合は for 式のほうがよりシンプルに書くことができていることがわかると思います。

15.5.3.1　練習問題

for を利用して、`Some(2)` と `Some(3)` と `Some(5)` と `Some(7)` と `Some(11)` の値をかけて、`Some(2310)` を求めてみましょう。

```
val v1: Option[Int] = Some(2)
val v2: Option[Int] = Some(3)
val v3: Option[Int] = Some(5)
val v4: Option[Int] = Some(7)
val v5: Option[Int] = Some(11)
for { i1 <- v1
      i2 <- v2
      i3 <- v3
      i4 <- v4
```

```
10      i5 <- v5 } yield i1 * i2 * i3 * i4 * i5
```

15.5.4 Either

Option により null を使う必要はなくなりましたが、いっぽうで Option では処理が成功したかどうかしかわからないという問題があります。None の場合、値が取得できなかったことはわかりますが、エラーの状態は取得できないので、使用できるのはエラーの種類が問題にならないような場合のみです。

そんな Option と違い、エラー時にエラーの種類まで取得できるのが Either です。Option が正常な値と何もない値のどちらかを表現するデータ型だったのに対して、Either は 2 つの値のどちらかを表現するデータ型です。具体的には、Option では `Some` と `None` の 2 つの値を持ちましたが、Either は `Right` と `Left` の 2 つの値を持ちます。

```
1  val v1: Either[String, Int] = Right(123)
2  // v1: Either[String, Int] = Right(123)
3
4  val v2: Either[String, Int] = Left("abc")
5  // v2: Either[String, Int] = Left("abc")
```

パターンマッチで値を取得できるのも Option と同じです。

```
1  v1 match {
2    case Right(i) => println(i)
3    case Left(s)  => println(s)
4  }
5  // 123
```

15.5.4.1 Either でエラー値を表現する

一般的に Either を使う場合、Left 値をエラー値、Right 値を正常な値とみなすことが多いです。英語の”right” が正しいという意味なので、それにかけているという説があります。そして Left に用いるエラー値ですが、これは代数的データ型（sealed trait または sealed abstract class と case class で構成される一連のデータと型のこと）で定義するとよいでしょう。パターンマッチの節で解説したように代数的データ型を用いることでエラーの処理が漏れているかどうかをコンパイラが検知してくれるようになります。単に `Throwable` 型をエラー型に使うのなら後述の Try で十分です。

例として Either を使ってログインのエラーを表現してみましょう。Left の値となる `LoginError` を定義します。`sealed` を使って代数的データ型として定義するのがポイントです。

```
1  sealed trait LoginError
2  // パスワードが間違っている場合のエラー
3  case object InvalidPassword extends LoginError
4  // nameで指定されたユーザーが見つからない場合のエラー
5  case object UserNotFound extends LoginError
```

```
// パスワードがロックされている場合のエラー
case object PasswordLocked extends LoginError
```

ログイン API の型は以下のようにします。

```
case class User(id: Long, name: String, password: String)

object LoginService {
  def login(name: String, password: String): Either[LoginError, User] = ???
}
```

login メソッドはユーザー名とパスワードをチェックして正しい組み合わせの場合は User オブジェクトを Either の Right の値で返し、エラーが起きた場合は LoginError を Either の Left の値で返します。

それでは、この login メソッドを使ってみましょう。

```
LoginService.login(name = "dwango", password = "password") match {
  case Right(user) => println(s"id: ${user.id}")
  case Left(InvalidPassword) => println(s"Invalid Password!")
}
```

とりあえず呼び出して、println を使って中身を表示しているだけです。ここで注目していただきたいのが、Left の値のパターンマッチです。InvalidPassword の処理はしていますが、UserNotFound の場合と PasswordLocked の場合の処理が抜けてしまっています。そのような場合でもエラー値に代数的データ型を用いているので、コンパイラがエラー処理漏れを検知してくれます。

試しに上のコードをコンパイルしてみると、

```
<console>:11: warning: match may not be exhaustive.
It would fail on the following inputs: Left(PasswordLocked), Left(UserNotFound)
              LoginService.login(name = "dwango", password = "password") match {
                                 ^
```

のようにコンパイラが Left(PasswordLocked) と Left(UserNotFound) の処理が漏れていることを warning で教えてくれます。Either を使う場合はこのテクニックを覚えておいたほうがいいでしょう。

15.5.4.2　Either の map と flatMap

以上、見てきたように格納できるデータが増えているという点で Either は Option の拡張版に近いです。Option と同様に Either も for 式を使って複数の Either を組み合わせることができます。Either には Right, Left の 2 つの値がありますが、Scala の Either では Right が正常な値になることが多いため、map や flatMap では Right の値が利用されます。[*1]

ためしに Either の map メソッドを使ってみましょう

[*1] Scala 2.11 までは、両者の値を平等に扱っていたため.right や .left を用いてどちらの値を map に渡すかを明示する必要がありました。

```
val v: Either[String, Int] = Right(123)
// v: Either[String, Int] = Right(123)

v.map(_ * 2)
// res20: Either[String, Int] = Right(246)

val v2: Either[String, Int] = Left("a")
// v2: Either[String, Int] = Left("a")
v2.map(_ * 2) // v2?Left?????????
// res21: Either[String, Int] = Left("a")
```

これで map を使って値を二倍にする関数を Right に適用できました。Either が Left の場合は何の処理もおこなわれません。これは Option で None に対して map を使った場合に何の処理もおこなわれないという動作に似ていますね。

15.5.5　名前渡しパラメータ

少し寄り道をして、名前渡しパラメータ（by-name parameter）という Scala の機能を紹介します。これから紹介する `Try` 型の実装などで使われている機能だからです。

Scala においては、メソッド実行前にはまず引数が評価され、次いでメソッド本体のコードが実行されます。次の例からも分かります。

```
def f(x: Any): Unit = println("f")
def g(): Unit = println("g")
f(g())
// g
// f
```

ごく普通の挙動だと思います。Scala に限らず、他の多くのプログラミング言語でも同様の実行順序となります。この評価順序のことを先行評価（eager evaluation）あるいは正格評価（strict evaluation）と呼びます。

さて、時折この挙動を変更したい場合があります。名前渡しパラメータを使うと、**変数が実際に使用される箇所まで評価を遅延させる** ことができます。メソッド本体のそれが使われる箇所で引数の式が計算されるということです。次のようなコードを見ると分かりやすいと思います。

```
def g(): Unit = println("g")
def f(g: => Unit): Unit = {
  println("prologue f")
  g
  println("epilogue f")
}
f(g())
// prologue f
// g
// epilogue f
```

"g" の出力が関数の内側になっていることがわかると思います。メソッド f の引数の型に注目すると、型 Unit の手前に=> が付いています。これが名前渡しパラメータの指定を表します。

名前渡しパラメータは次のような場合に使われます。

- 引数の式が例外を投げるかもしれないので、try-finally 構文の中で引数を評価したい
- 引数の式がものすごく計算コストが高いかもしれないが、計算結果を本当に使うかわからない。使われる箇所で計算させたい

似たような挙動は高階関数を使えば書けるのですが、名前渡しパラメータのほうが簡潔な記述ができるという点でより優れています。

15.5.6 Try

Scala の Try は Either と同じように正常な値とエラー値のどちらかを表現するデータ型です。Either との違いは、2 つの型が平等ではなく、エラー値が Throwable に限定されており、型引数を 1 つしか取らないことです。具体的には Try は以下の 2 つの値をとります。

- Success
- Failure

ここで Success は型変数を取り、任意の値を入れることができますが、Failure は Throwable しか入れることができません。そして Try には、コンパニオンオブジェクトの apply で生成する際に、例外を catch し、Failure にする機能があります。

```
import scala.util.Try

val v: Try[Int] = Try(throw new RuntimeException("to be caught"))
// v: Try[Int] = Failure(java.lang.RuntimeException: to be caught)
```

この機能を使って、例外が起こりそうな箇所を Try で包み、Failure にして値として扱えるようにするのが Try の特徴です。

```
val v1 = Try(3)
// v1: Try[Int] = Success(3)

val v2 = Try(5)
// v2: Try[Int] = Success(5)

val v3 = Try(7)
// v3: Try[Int] = Success(7)

for {
  i1 <- v1
  i2 <- v2
  i3 <- v3
} yield i1 * i2 * i3
// res24: Try[Int] = Success(105)
```

15.5.6.1　NonFatal の例外

Try.apply が catch するのはすべての例外ではありません。NonFatal という種類の例外だけです。NonFatal ではない例外はアプリケーション中で復旧が困難な非常に重度なものです。なので、NonFatal ではない例外は catch せずにアプリケーションを終了させて、外部から再起動などをしたほうがいいです。

Try 以外でも、たとえば扱うことができる全ての例外をまとめて処理したい場合などに、

```
import scala.util.control.NonFatal

try {
  ???
} catch {
  case NonFatal(e) => // 例外の処理
}
```

というパターンが実践的なコード中に出てくることがしばしばあるので覚えておくとよいと思います。

15.5.7　Option と Either と Try の使い分け

ではエラー処理において Option と Either と Try はどのように使い分けるべきなのでしょうか。

まず基本的に Java で null を使うような場面は Option を使うのがよいでしょう。コレクションの中に存在しなかったり、ストレージ中から条件に合うものを発見できなかったりした場合は Option で十分だと考えられます。

次に Either ですが、Option を使うのでは情報が不足しており、かつ、エラー状態が代数的データ型としてちゃんと定められるものに使うのがよいでしょう。Java でチェック例外を使っていたようなところで使う、つまり、復帰可能なエラーだけに使うという考え方でもよいです。Either と例外を併用するのもアリだと思います。

Try は Java の例外をどうしても値として扱いたい場合に用いるとよいです。非同期プログラミングで使ったり、実行結果を保存しておき、あとで中身を参照したい場合などに使うことも考えられます。

15.6　Option の例外処理を Either でリファクタする実例

Scala でリレーショナルデータベースを扱う場合、関連をたどっていく中でどのタイミングで情報が取得できなかったのかを返さねばならないことがあります。

None を盲目的に処理するのであれば、flatMap や for 式をつかえば畳み込んでスッキリかけるのですが、関連を取得していくなかでどのタイミングで None が取得されてしまったのか返したい場合にはそうは行かず、結局 match case の深いネストになってしまいます。

例を挙げます。

ユーザーとアドレスがそれぞれデータベースに格納されており、ユーザー ID を利用してそのユーザーを検索し、ユーザーが持つアドレス ID でアドレスを検索し、さらにその郵便番号を取得するような場合を考えます。

失敗結果としては

- ユーザーがみつからない
- ユーザーがアドレスを持っていない
- アドレスがみつからない
- アドレスが郵便番号を持っていない

という 4 つの失敗パターンがあり、それらを結果オブジェクトとして返さなくてはなりません。

以下のようなコードになります。

```
object MainBefore {

  case class Address(id: Int, name: String, postalCode: Option[String])
  case class User(id: Int, name: String, addressId: Option[Int])

  val userDatabase: Map[Int, User] = Map (
    1 -> User(1, "太郎", Some(1)),
    2 -> User(2, "二郎", Some(2)),
    3 -> User(3, "プー太郎", None)
  )

  val addressDatabase: Map[Int, Address] = Map (
    1 -> Address(1, "渋谷", Some("150-0002")),
    2 -> Address(2, "国際宇宙ステーション", None)
  )

  sealed abstract class PostalCodeResult
  case class Success(postalCode: String) extends PostalCodeResult
  sealed abstract class Failure extends PostalCodeResult
  case object UserNotFound extends Failure
  case object UserNotHasAddress extends Failure
  case object AddressNotFound extends Failure
  case object AddressNotHasPostalCode extends Failure

  // どこでNoneが生じたか取得しようとするとfor式がつかえず地獄のようなネストになる
  def getPostalCodeResult(userId: Int): PostalCodeResult = {
    findUser(userId) match {
      case Some(user) =>
        user.addressId match {
          case Some(addressId) =>
            findAddress(addressId) match {
              case Some(address) =>
                address.postalCode match {
                  case Some(postalCode) => Success(postalCode)
                  case None => AddressNotHasPostalCode
                }
```

```
              case None => AddressNotFound
            }
          case None => UserNotHasAddress
        }
      case None => UserNotFound
    }
  }

  def findUser(userId: Int): Option[User] = {
    userDatabase.get(userId)
  }

  def findAddress(addressId: Int): Option[Address] = {
    addressDatabase.get(addressId)
  }

  def main(args: Array[String]): Unit = {
    println(getPostalCodeResult(1)) // Success(150-0002)
    println(getPostalCodeResult(2)) // AddressNotHasPostalCode
    println(getPostalCodeResult(3)) // UserNotHasAddress
    println(getPostalCodeResult(4)) // UserNotFound
  }
}
```

getPostalCodeResult が鬼のような match case のネストになっていることがわかります。このような可読性の低いコードを、Either を使って書きなおすことができます。

以下のように全ての find メソッドを Either で Failure を Left に、正常取得できた場合の値の型を Right にして書き直します。

find の各段階で Failure オブジェクトに引き換えるという動きをさせるわけです。

リファクタリングした結果は以下のようになります。

```
object MainRefactored {

  case class Address(id: Int, name: String, postalCode: Option[String])
  case class User(id: Int, name: String, addressId: Option[Int])

  val userDatabase: Map[Int, User] = Map (
    1 -> User(1, "太郎", Some(1)),
    2 -> User(2, "二郎", Some(2)),
    3 -> User(3, "プー太郎", None)
  )

  val addressDatabase: Map[Int, Address] = Map (
    1 -> Address(1, "渋谷", Some("150-0002")),
    2 -> Address(2, "国際宇宙ステーション", None)
  )

  sealed abstract class PostalCodeResult
  case class Success(postalCode: String) extends PostalCodeResult
  abstract class Failure extends PostalCodeResult
  case object UserNotFound extends Failure
```

```
  case object UserNotHasAddress extends Failure
  case object AddressNotFound extends Failure
  case object AddressNotHasPostalCode extends Failure

  // 本質的に何をしているかわかりやすくリファクタリング
  def getPostalCodeResult(userId: Int): PostalCodeResult = {
    (for {
      user <- findUser(userId)
      address <- findAddress(user)
      postalCode <- findPostalCode(address)
    } yield Success(postalCode)).merge
  }

  def findUser(userId: Int): Either[Failure, User] = {
    userDatabase.get(userId).toRight(UserNotFound)
  }

  def findAddress(user: User): Either[Failure, Address] = {
    for {
      addressId <- user.addressId.toRight(UserNotHasAddress)
      address <- addressDatabase.get(addressId).toRight(AddressNotFound)
    } yield address
  }

  def findPostalCode(address: Address): Either[Failure, String] = {
    address.postalCode.toRight(AddressNotHasPostalCode)
  }

  def main(args: Array[String]): Unit = {
    println(getPostalCodeResult(1)) // Success(150-0002)
    println(getPostalCodeResult(2)) // AddressNotHasPostalCode
    println(getPostalCodeResult(3)) // UserNotHasAddress
    println(getPostalCodeResult(4)) // UserNotFound
  }
}
```

以上のようになり、

```
  def getPostalCodeResult(userId: Int): PostalCodeResult = {
    (for {
      user <- findUser(userId)
      address <- findAddress(user)
      postalCode <- findPostalCode(address)
    } yield Success(postalCode)).merge
  }
```

getPostalCodeResult が本質的に何をしているのかが非常にわかりやすいコードとなりました。何をしているかというと、for 式で値を取得した後、merge メソッドにより中身を畳み込んで取得しています。

第 16 章

implicit キーワード

Scala には他の言語には見られない implicit というキーワードで表現される機能があります。Scala 2 では implicit という単一の機能によって複数の用途を賄うようになっていますが、1 つの機能で色々な用途を表現できることがユーザーにとってわかりにくかったという反省もあり、Scala 3 では用途別に異なるキーワードや構文を使う形になっています。

この章では Scala 2 での implicit キーワードの 4 つの使い方を説明します。

16.1 Implicit conversion

implicit conversion は暗黙の型変換をユーザが定義できる機能です。Scala が普及し始めた時はこの機能が多用されたのですが、implicit conversion を多用するとプログラムが読みづらくなるということがわかったため、現在は積極的に使うことは推奨されていません。とはいえ、固定長整数から多倍長整数への変換など、標準ライブラリやサードパーティのライブラリで使われているケースもあるので知っておいた方が良いのは確かです。

implicit conversion は次のような形で定義します。

```
implicit def メソッド名(引数名: 引数の型): 返り値の型 = 本体
```

`implicit` というキーワードがついていることと引数が 1 つしかない[*1]ことを除けば通常のメソッド定義同様です。implicit conversion では引数の型と返り値の型に重要な意味があります。**引数の型**の式が現れたときに**返り値の型**を暗黙の型変換候補として登録することになるからです。

implicit conversion は次のようにして定義することができます。この例では、`Int` 型から `Boolean` 型への暗黙の型変換を定義しています。

[*1] 引数が 2 つ以上ある implicit def の定義も可能です。「implicit def のパラメーターに implicit が含まれる」という型クラス的な使い方をする場合は実際に implicit def に 2 つ以上のパラメーターが出現することがあります。ただしそういった定義は通常 implicit conversion とは呼ばれません

```
implicit def intToBoolean(arg: Int): Boolean = arg != 0

if(1) {
  println("1????")
}
// 1????
```

コンパイラは `if(1)` を見た時点で、本来 `Boolean` が要求されているのに `Int` 型の式である `1` が書かれていることがわかります。多くの静的型付き言語ではここで型エラーになります。しかし、Scala では引数が `Int` で返り値が `Boolean` である暗黙の型変換が定義されていないかを探索し、`intToBoolean` という暗黙の型変換を発見します。そして、以下のように `intToBoolean(1)` を挿入するのです。

```
if(intToBoolean(1)) {
  println("1????")
}
// 1????
```

このようにして、`if` の条件式に `Int` を渡すことができるようになるわけです。ただし、暗黙の型変換のこのような使い方はあまり良いものではありません。if 式の条件式に `Boolean` 型の式しか渡せないようになっているのは間違いを防止するためなのに、そのチェックを通り抜けてしまえるわけですから。

`BigInt` や `BigDecimal` など一部のライブラリでは Scala 標準の `Int` や `Double` と相互に変換するために implicit conversion を定義していますが、普通のユーザーが定義する必要があることは稀です。正当な理由を思いつかない限りは使わないようにしましょう。

Scala 3 では `scala.Conversion` クラスのインスタンスを型クラス（後述）のインスタンスとして定義することで、implicit conversion を実現しています。しかし、Scala 2 の場合と同様に利用するときは慎重になるべきです。

16.2　Enrich my library

Enrich my library パターンと呼ばれるものがあります。C#や Kotlin などにある拡張メソッドと同等のもので、既存のクラスにメソッドを追加したようにみせかけることができます。Scala 標準ライブラリの中にも利用例がありますし、サードパーティのライブラリでもよく見かけます。

たとえば、これまでみたプログラムの中には `(1 to 5)` という式がありましたが、本来 `Int` 型は `to` というメソッドを持っていません。

`to` メソッドは enrich my library パターンの典型的な利用例です。`Int` に対して `to` メソッドが定義されていないことがわかると、既存の implicit conversion で定義されたメソッドの返り値型に `to` メソッドの定義がないか検索して、メソッドが見つかった場合に適切な implicit conversion を挿入するのです。

この使い方では変換先の型は純粋にメソッドを追加するためだけに存在しているため、既存の型同

士を変換するときのような混乱は起こりません。試しに、String の末尾に":-)" という文字列を追加して返すように enrich my library パターンを使って

```
class RichString(src: String) {
  def smile: String = src + ":-)"
}

implicit def enrichString(arg: String): RichString = new RichString(arg)

"Hi, ".smile
// res2: String = "Hi, :-)"
```

文字列の末尾に":-)" を追加する smile メソッドが定義できています。このとき、Scala コンパイラは enrichString("Hi, ") の呼び出しを適切に挿入してくれます。

```
enrichString("Hi, ").smile
// res3: String = "Hi, :-)"
```

しかし、拡張メソッドのために implicit conversion を毎回定義するのは冗長です。Scala 2.10 以降では class に implicit キーワードをつけることで簡潔な記述が可能になりました。上の定義は

```
implicit class RichString(src: String) {
  def smile: String = src + ":-)"
}

"Hi, ".smile
// res5: String = "Hi, :-)"
```

という形で書きなおすことができます。implicit class は enrich my library パターン専用の機能なので、拡張メソッドを定義する意図を適切に表現できます。enrich my library パターンが必要なときは原則的に implicit class を使うべきです。

しかし、サードパーティのライブラリや標準ライブラリでは implicit class が使われていないこともあるので、そのようなコードも読めるようにしておくのが良いでしょう。

さらに、Scala 3 では以下のような拡張メソッドを定義するための専用構文が用意されました。

```
extension (src: String) {
  def smile: String = src + ":-)"
}
```

16.2.1　練習問題

Int から BigInt への implicit conversion のように、利用者にとって便利になる implicit conversion を考えて定義してみてください。その implicit conversion にはどのような利点と欠点があるかを答えてください。

16.2.2　練習問題

既存のクラスの利用を便利にするような形で、enrich my library パターンを適用してみましょう。どのような場面で役に立つでしょうか？

```
object Taps {
  implicit class Tap[T](self: T) {
    def tap[U](block: T => U): T = {
      block(self) //値は捨てる
      self
    }
  }

  def main(args: Array[String]): Unit = {
    "Hello, World".tap{s => println(s)}.reverse.tap{s => println(s)}
  }
}
```

```
import Taps._
Taps.main(Array())
// Hello, World
// dlroW ,olleH
```

定義した `tap()` メソッドは Ruby などの言語にありますが、メソッドチェインの中でデバッグプリントをはさみたいときに役に立ちます。

16.2.2.1　練習問題

Scala 標準ライブラリの中から enrich my library が使われている例を 1 つ以上見つけてください。どのような時に便利でしょうか？

16.3　Implicit parameter（文脈引き渡し）

implicit parameter は主に 2 つの目的で使われます。1 つ目の目的は、あちこちのメソッドに共通で渡されるオブジェクト（たとえば、ソケットやデータベースのコネクション）を明示的に引き渡すのを省略することです。

データベースとのコネクションを表す `Connection` 型があるとします。データベースと接続するメソッドには全て `Connection` 型を渡さなければなりません。

```
def readRecordsFromTable(columnName: String, tableName: String, connection: Connection
    ): List[Record]
def writeRecordsToTable(record: List[Record], tableName: String, connection: Connection
    ): Unit
def readAllFromTable(tableName: String, connection: Connection): List[Row]
```

3 つのメソッドは全て Connection 型を引数に取るのに、呼びだす度に明示的に Connection オブジェクトを渡さなければいけません。ここで implicit parameter の出番です。上のメソッド定義を

```
def readRecordsFromTable(columnName: String, tableName: String)(implicit connection:
    Connection): List[Record]
def writeRecordsToTable(records: List[Record], tableName: String)(implicit connection:
    Connection): Unit
def readAllFromTable(tableName: String, connection: Connection)(implicit connection:
    Connection): List[Record]
```

と書き換えます。implicit 修飾子は最後の引数リストに付けなければならないという制約があります。つまり、以下のようになっているのがポイントです。

```
(....)(implicit conn: Connection)
```

Scala コンパイラは、このように定義されたメソッドが呼び出されると、現在の呼び出しスコープからたどって直近の implicit とマークされた値を暗黙にメソッドに引き渡します。たとえば次のようにして、値を implicit としてマークします：

```
implicit val aConnection: Connection = connectDatabase(....)
```

こうすれば、最後の引数リストに暗黙に Connection オブジェクトを渡してくれるのです。以下のような呼び出しがあったとします。

```
val firstNames = readRecordsFromTable("first_name", "people")
```

この呼出しは次のように変換されます。

```
val firstNames = readRecordsFromTable("first_name", "people")(aConnection)
```

このような文脈を引き渡すための implicit parameter は Play Framework や O/R マッパーなどで出てきます。

16.4　Implicit parameter（型クラス）

implicit parameter のもう 1 つの使い方は風変わりです。Haskell などの型クラスがある言語をご存知の人なら、型クラスそのものであると言う説明がわかりやすいかもしれません。多くの読者は型クラスについては知らないと思いますから、ここでは一から説明します。

List の全ての要素の値を加算した結果を返す sum メソッドを定義したいとします。このような要求は頻繁にあるので、定義できれば嬉しいことは間違いありません。問題はそのようなメソッドを素直に定義できない点にあります。

ポイントは「何の」List か全くわかっていないことです。何のリストかわからないということは、整数や浮動小数点数の + メソッドをそのまま使うことはできないということです。このような時に implicit parameter の出番です。

2 つの同じ型を足す（0 の場合はそれに相当する値を返す）方法を知っている型を定義します。ここではその型を Additive とします。Additive の定義は次のようになります：

```
trait Additive[A] {
  def zero: A
  def plus(a: A, b: A): A
}
```

Additive の型パラメータ A は加算される List の要素型を表しています。また、

- zero：型パラメータ A の **0** に相当する値を返す
- plus()：型パラメータ A を持つ 2 つの値を**加算して**返す

です。

次に Additive 型を使って、List の全ての要素を合計するメソッドを定義します：

```
def sum[A](lst: List[A])(a: Additive[A]) = lst.foldLeft(a.zero)((x, y) => a.plus(x, y))
```

最後に、型に応じた zero と plus() の定義を持った object を定義します。ここでは String と Int について、Additive[Int] と Additive[String] を定義します。

```
object StringAdditive extends Additive[String] {
  def plus(a: String, b: String): String = a + b
  def zero: String = ""
}

object IntAdditive extends Additive[Int] {
  def plus(a: Int, b: Int): Int = a + b
  def zero: Int = 0
}
```

まとめると次のようになります。

```
trait Additive[A] {
  def plus(a: A, b: A): A
  def zero: A
}

object StringAdditive extends Additive[String] {
  def plus(a: String, b: String): String = a + b
  def zero: String = ""
}

object IntAdditive extends Additive[Int] {
  def plus(a: Int, b: Int): Int = a + b
  def zero: Int = 0
}

def sum[A](lst: List[A])(a: Additive[A]) = lst.foldLeft(a.zero)((x, y) => a.plus(x, y))
```

List[Int] 型と List[String] 型のどちらでも、要素の合計を計算できる汎用的な sum メソッドができました。

実際に呼び出したいときには、

```
sum(List(1, 2, 3))(IntAdditive)
// res8: Int = 6
sum(List("A", "B", "C"))(StringAdditive)
// res9: String = "ABC"
```

とすれば良いだけです。

これで目的は果たすことはできますが、何の List の要素を合計するかは型チェックする時点ではわかっているのだから IntAdditive , StringAdditive を明示的に渡さずとも賢く推論してほしいものです。実は、まさにそれを implicit parameter で実現することができるのです。

方法は簡単。StringAdditive と IntAdditive の定義の前に implicit と付けることと、sum の最後の引数リストである m に implicit を付けるだけです。implicit parameter を使った最終形は次のようになります。

```
trait Additive[A] {
  def plus(a: A, b: A): A
  def zero: A
}

implicit object StringAdditive extends Additive[String] {
  def plus(a: String, b: String): String = a + b
  def zero: String = ""
}

implicit object IntAdditive extends Additive[Int] {
  def plus(a: Int, b: Int): Int = a + b
  def zero: Int = 0
}

def sum[A](lst: List[A])(implicit m: Additive[A]) = lst.foldLeft(m.zero)((x, y) => m.
    plus(x, y))

sum(List(1, 2, 3))
// res10: Int = 6

sum(List("A", "B", "C"))
// res11: String = "ABC"
```

任意の List の要素の合計値を求める sum メソッドを自然な形で呼びだすことができています。

implicit parameter のこのような使い方はプログラミング言語 Haskell から借りてきたもので、Haskell では型クラスと呼ばれます。そのため、Scala でも型クラスと呼ばれることも多々あります。Haskell の用語だと、Additive に相当する宣言を型クラスの宣言、StringAdditive と IntAdditive を Additive 型クラスのインスタンスの定義と呼びます。

implicit parameter の型クラス的な用法は標準ライブラリにもあります。たとえば、

```
List[Int]().sum
// res12: Int = 0

List(1, 2, 3, 4).sum
// res13: Int = 10

List(1.1, 1.2, 1.3, 1.4).sum
// res14: Double = 5.0
```

のように整数や浮動小数点数の合計値を計算することができます。これは、implicit parameter のおかげです。Scala で型クラスを定義・使用する方法を覚えると設計の幅がグンと広がります。

16.4.1　練習問題

`m: Additive[T]` と値 `t1: T, t2: T, t3: T` は、次の条件を満たす必要があります。

```
m.plus(m.zero, t1) == t1  // 単位元
m.plus(t1, m.zero) == t1  // 単位元
m.plus(t1, m.plus(t2, t3)) == m.plus(m.plus(t1, t2), t3) // 結合則
```

条件を満たす型 `T` と単位元 `zero`、演算 `plus` を探し出し、`Additive[T]` を定義しましょう。また、条件が満たされていることを確認してみましょう。定義した `Additive[T]` を `implicit` にして、`T` の合計値を先ほどの `sum` で計算できることも確かめてみましょう。

ヒント：条件を満たす型は無数にありますが、たとえば `x` 座標と `y` 座標からなる点を表すクラス `Point` を考えてみると良いでしょう。

```

trait Additive[A] {
  def plus(a: A, b: A): A
  def zero: A
}

implicit object StringAdditive extends Additive[String] {
  def plus(a: String, b: String): String = a + b
  def zero: String = ""
}

implicit object IntAdditive extends Additive[Int] {
  def plus(a: Int, b: Int): Int = a + b
  def zero: Int = 0
}

case class Point(x: Int, y: Int)

implicit object PointAdditive extends Additive[Point] {
  def plus(a: Point, b: Point): Point = Point(a.x + b.x, a.y + b.y)
  def zero: Point = Point(0, 0)
}
```

```
23
24 def sum[A](lst: List[A])(implicit m: Additive[A]) = lst.foldLeft(m.zero)((x, y) => m.
       plus(x, y))
```

```
println(sum(List(Point(1, 1), Point(2, 2), Point(3, 3)))) // Point(6, 6)
// Point(6,6)
println(sum(List(Point(1, 2), Point(3, 4), Point(5, 6)))) // Point(9, 12)
// Point(9,12)
```

16.4.2　練習問題

List[Int] と List[Double] の sum を行うために、標準ライブラリでは何という型クラス（1 つ）と型クラスのインスタンスを定義しているかを、Scala 標準ライブラリから探して挙げなさい。

型クラス：

- Numeric[T]

型クラスのインスタンス：

- IntIsIntegral
- DoubleIsFractional

16.4.3　implicit の探索範囲

implicit conversion や implicit parameter の値が探索される範囲には、

- ローカルで定義されたもの
- import で指定されたもの
- スーパークラスで定義されたもの
- コンパニオンオブジェクトで定義されたもの

などがあります。この中で注目していただきたいのが、コンパニオンオブジェクトで implicit の値を定義するパターンです。

たとえば新しく Rational（有理数）型を定義したとして、コンパニオンオブジェクトに先ほど使った Additive 型クラスのインスタンスを定義しておきます。

```
case class Rational(num: Int, den: Int)

object Rational {
  implicit object RationalAdditive extends Additive[Rational] {
    def plus(a: Rational, b: Rational): Rational = {
      if (a == zero) {
        b
```

```
      } else if (b == zero) {
        a
      } else {
        Rational(a.num * b.den + b.num * a.den, a.den * b.den)
      }
    }
    def zero: Rational = Rational(0, 0)
  }
}
```

import をしていないのに、Additive 型クラスのインスタンスを使うことができます。

```
scala> sum(List(Rational(1, 1), Rational(2, 2)))
res0: Rational = Rational(4,2)
```

新しくデータ型を定義し、型クラスインスタンスも一緒に定義したい場合によく出てくるパターンなので覚えておくとよいでしょう。

第 17 章

型クラスへの誘い

本章では 16 章で、少しだけ触れた型クラスについて、より深く掘り下げます。

Implicit の章では、Additive という型クラスを定義することで、任意のコレクション（Additive が存在するもの）に対して、要素の合計値を計算することができたのでした。本章では、このような仕組みを利用して、色々なアルゴリズムをライブラリ化できることを見ていきます。

17.1 average メソッド

まず、sum に続いて、要素の平均値を計算するための average メソッドを作成することを考えます。average メソッドの素朴な実装は次のようになるでしょう。[*1]

```
def average(list: List[Int]): Int = list.foldLeft(0)(_ + _) / list.size
```

これを、前章のように Additive を使ってみます。

```
trait Additive[A] {
  def plus(a: A, b: A): A
  def zero: A
}
object Additive {
  implicit object IntAdditive extends Additive[Int] {
    def plus(a: Int, b: Int): Int = a + b
    def zero: Int = 0
  }
  implicit object DoubleAdditive extends Additive[Double] {
    def plus(a: Double, b: Double): Double = a + b
    def zero: Double = 0.0
  }
}
```

[*1] ここで、List の size を計算するのは若干効率が悪いですが、その点については気にしないことにします。

```
def average[A](lst: List[A])(implicit m: Additive[A]): A = {
  val length: Int = lst.length
  val sum: A = lst.foldLeft(m.zero)((x, y) => m.plus(x, y))
  sum / length
}
```

残念ながら、このコードはコンパイルを通りません。何故ならば、A 型の値を Int 型の値で割ろうとしているのですが、その方法がわからないからです。ここで、Additive をより一般化して、引き算、掛け算、割り算をサポートした型 Num を考えてみます。掛け算の メソッドを multiply 、割り算のメソッドを divide とすると、Num は次のようになるでしょう。ここで、Nums は、対話環境でコンパニオンクラス/オブジェクトを扱うために便宜的に作った名前空間であり、通常の Scala プログラムでは、コンパニオンクラス/オブジェクトを定義するときの作法に従えばよいです[*2]。

```
object Nums {
  trait Num[A] {
    def plus(a: A, b: A): A
    def minus(a: A, b: A): A
    def multiply(a: A, b: A): A
    def divide(a: A, b: A): A
    def zero: A
  }
  object Num{
    implicit object IntNum extends Num[Int] {
      def plus(a: Int, b: Int): Int = a + b
      def minus(a: Int, b: Int): Int = a - b
      def multiply(a: Int, b: Int): Int = a * b
      def divide(a: Int, b: Int): Int = a / b
      def zero: Int = 0
    }
    implicit object DoubleNum extends Num[Double] {
      def plus(a: Double, b: Double): Double = a + b
      def minus(a: Double, b: Double): Double = a - b
      def multiply(a: Double, b: Double): Double = a * b
      def divide(a: Double, b: Double): Double = a / b
      def zero: Double = 0.0
    }
  }
}
```

また、average メソッドは、リストの長さ、つまり整数で割る必要があるので、整数を A 型に変換するための型 FromInt も用意します。FromInt は次のようになります。to は Int 型を対象の型に変換するメソッドです。

```
object FromInts {
  trait FromInt[A] {
    def to(from: Int): A
```

[*2] つまり、単に Nums を削除して同一ファイルに両方の定義を置けば良いです

```
  }
  object FromInt {
    implicit object FromIntToInt extends FromInt[Int] {
      def to(from: Int): Int = from
    }
    implicit object FromIntToDouble extends FromInt[Double] {
      def to(from: Int): Double = from
    }
  }
}
```

Num と FromInt を使うと、average 関数は次のように書くことができます。

```
import Nums._
import FromInts._
def average[A](lst: List[A])(implicit a: Num[A], b: FromInt[A]): A = {
  val length: Int = lst.length
  val sum: A  = lst.foldLeft(a.zero)((x, y) => a.plus(x, y))
  a.divide(sum, b.to(length))
}
```

この average 関数は次のようにして使うことができます。

```
average(List(1, 3, 5))
// res0: Int = 3
average(List(1.5, 2.5, 3.5))
// res1: Double = 2.5
```

このようにして、複数の型クラスを組み合わせることで、より大きな柔軟性を手に入れることができました。ちなみに、

17.1.1　context bounds

上記のコードは、context bounds というシンタックスシュガーを使うことで、次のように書き換えることもできます。

```
import Nums._
import FromInts._
def average[A:Num:FromInt](lst: List[A]): A = {
  val a = implicitly[Num[A]]
  val b = implicitly[FromInt[A]]
  val length = lst.length
  val sum: A  = lst.foldLeft(a.zero)((x, y) => a.plus(x, y))
  a.divide(sum, b.to(length))
}
```

implicit parameter の名前 a と b が引数から見えなくなりましたが、implicitly[Type] とすることで、Type 型の implicit parameter の値を取得することができます。

17.2　max メソッドと min メソッド

別のアルゴリズムをライブラリ化した例を、Scala の標準ライブラリから紹介します。コレクションから最大値を取得する max と最小値を取得する min です。これらは、次のようにして使うことができます。

```
List(1, 3, 4, 2).max
// res2: Int = 4
List(1, 3, 2, 4).min
// res3: Int = 1
```

比較できない要素のリストに対して max 、min を求めようとするとコンパイルエラーになります。この max と min も型クラスで実現されています。

max と min のシグネチャは次のようになっています。

```
def max[B >: A](implicit cmp: Ordering[B]): A
```

```
def min[B >: A](implicit cmp: Ordering[B]): A
```

B >: A の必要性についてはおいておくとして、ポイントは、Ordering[B] 型の implicit parameter を要求するところです。Ordering[B] の implicit なインスタンスがあれば、B 型同士の大小関係を比較できるため、最大値と最小値を求めることができます。

17.3　median メソッド

さらに、別のアルゴリズムをライブラリ化してみます。リストの中央値を求めるメソッド median を定義することを考えます。中央値は、要素数が奇数の場合、リストをソートした場合のちょうど真ん中の値を、偶数の場合、真ん中の 2 つの値を足して 2 で割ったものになります。ここで、中央値の最初のケースには Ordering が、2 番目のケースにはそれに加えて、先程定義した Num と FromInt があれば良さそうです。

この 3 つを使って、median メソッドを定義してみます。先程出てきた context bounds を使って、シグネチャが見やすいようにしています。

```
import Nums._
import FromInts._
def median[A:Num:Ordering:FromInt](lst: List[A]): A = {
  val num = implicitly[Num[A]]
  val ord = implicitly[Ordering[A]]
  val int = implicitly[FromInt[A]]
  val size = lst.size
  require(size > 0)
  val sorted = lst.sorted
  if(size % 2 == 1) {
```

```
    sorted(size / 2)
  } else {
    val fst = sorted((size / 2) - 1)
    val snd = sorted((size / 2))
    num.divide(num.plus(fst, snd), int.to(2))
  }
}
```

このメソッドは次のようにして使うことができます。

```
assert(2 == median(List(1, 3, 2)))
assert(2.5 == median(List(1.5, 2.5, 3.5)))
assert(3 == median(List(1, 3, 4, 5)))
```

17.4　オブジェクトのシリアライズ

次はより複雑な例を考えてみます。次のように、オブジェクトをシリアライズするメソッド string を定義したいとします。

```
import Serializers.string
string(List(1, 2, 3)) // [1,2,3]
string(List(List(1),List(2),List(3)) // [[1],[2],[3]]
string(1) // 1
string("Foo") // Foo
class MyClass(val x: Int)
string(new MyClass(1)) // Compile Error!
class MyKlass(val x: Int)
implicit object MyKlassSerializer extends Serializer[MyKlass] {
  def serialize(klass: MyKlass): String = s"MyKlass(${klass.x})"
}
string(new MyKlass(1)) // OK
```

この string メソッドは、

- 整数をシリアライズ可能
- 文字列をシリアライズ可能
- 要素がシリアライズ可能なリストをシリアライズ可能

であり、自分で作成したクラスについては、次のトレイト Serializer を 継承して serialize メソッドを実装するオブジェクトを implicit にすることで、シリアライズ可能にできます。

```
trait Serializer[A] {
  def serialize(obj: A): String
}
```

これを仮に Serializer 型クラスと呼びます。

この string メソッドのシグニチャをまず考えてみます。このメソッドは Serializer 型クラスを必要としているので、Serializer[A] のような implicit parameter を必要としているはずです。また、引数は A 型の値 で、返り値は String なので、結果として次のようになります。

```
def string[A:Serializer](obj: A): String = ???
```

次に実装ですが、Serializer 型クラスを要求しているということは、Serializer の serialize メソッドを呼びだせばいいだけなので、次のようになります。

```
object Serializers {
  trait Serializer[A] {
    def serialize(obj: A): String
  }
  def string[A:Serializer](obj: A): String = {
    implicitly[Serializer[A]].serialize(obj)
  }
}
```

Serializers という object を作っていますが、これを import することで：

- string メソッドを使える
- Serializer 型クラスが公開される

ようになります。

さて、これでシグネチャの部分はできたので実装に入ります。とはいっても、今回の範囲内では、ほとんどオブジェクトを toString するだけのものなのですが...。

```
implicit object IntSerializer extends Serializer[Int] {
  def serialize(obj: Int): String = obj.toString
}
implicit object StringSerializer extends Serializer[String] {
  def serialize(obj: String): String = obj
}
```

以上は、整数と文字列の Serializer です。単に toString を呼び出しているか、自身を返しているだけなのがわかります。次が少しわかりにくいです。要素がシリアライズ可能なときだけ、リストがシリアライズ可能でなければいけないのですから、単純に以下のようにしてもだめです。

```
implicit def ListSerializer[A]: Serializer[List[A]] =
  new Serializer[List[A]] {
    def serialize(obj: List[A]): String = ???
  }
```

この定義では A にどのような操作が可能なのかわからないため、中身を単純に toString するくらいしか実装しようがないですし、また、そのような実装では要素型の Serializer の実装と整合性が取れません。これを解決するには、ListSerializer が implicit parameter を取るようにします。

```
implicit def ListSerializer[A](implicit serializer: Serializer[A]): Serializer[List[A]]
    =
  new Serializer[List[A]] {
    def serialize(obj: List[A]): String = {
      val serializedList = obj.map{o => serializer.serialize(o)}
      serializedList.mkString("[",",","]")
    }
  }
```

このように定義したとき、コンパイラは、要素型 A がシリアライズ可能でない場合（あらかじめ implicit def/object でそう定義されていない場合）コンパイルエラーにしてくれます。つまり、型安全にオブジェクトをシリアライズできるのです。

ここまでで、一通りの実装ができたので、定義を一箇所にまとめて実行結果を確認してみましょう。この節の最初の方の入力例を使って動作確認をします。

```
object Serializers {
  trait Serializer[A] {
    def serialize(obj: A): String
  }
  def string[A:Serializer](obj: A): String = {
    implicitly[Serializer[A]].serialize(obj)
  }
  implicit object IntSerializer extends Serializer[Int] {
    def serialize(obj: Int): String = obj.toString
  }
  implicit object StringSerializer extends Serializer[String] {
    def serialize(obj: String): String = obj
  }
  implicit def ListSerializer[A](implicit serializer: Serializer[A]): Serializer[List[A
    ]] = new Serializer[List[A]]{
    def serialize(obj: List[A]): String = {
      val serializedList = obj.map{o => serializer.serialize(o)}
      serializedList.mkString("[",",","]")
    }
  }
}
import Serializers._
string(List(1, 2, 3)) // [1,2,3]
// res7: String = "[1,2,3]"
string(List(List(1),List(2),List(3))) // [[1],[2],[3]]
// res8: String = "[[1],[2],[3]]"
string(1) // 1
// res9: String = "1"
string("Foo") // Foo
// res10: String = "Foo"
// class MyClass(val x: Int)
// string(new MyClass(1)) // Compile Error!
class MyKlass(val x: Int)
implicit object MyKlassSerializer extends Serializer[MyKlass] {
  def serialize(klass: MyKlass): String = s"MyKlass(${klass.x})"
}
```

```
36 string(new MyKlass(1)) // OK
37 // res11: String = "MyKlass(1)"
```

行コメントに書いた想定通りの動作をしていることがわかります。ここで重要なのは、`MyClass` に対しては `Serializer` を定義していないので、コンパイルエラーになる点です。多くの言語のシリアライズライブラリでは、リフレクションを駆使してシリアライズしようとするため、実行時になって初めてエラーがわかることが多いです。特に、静的型付き言語では、そのような場合、型によって安全を保証できないのはデメリットです。一方、今回用いた手法では、後付けでシリアライズする型を追加でき、かつコンパイル時にその正当性を検査できるのです。

17.5　まとめ

型クラス（≒ implicit parameter）は、うまく使うと、後付けのデータ型に対して既存のアルゴリズムを型安全に適用するのに使うことができます。この特徴は、特にライブラリ設計のときに重要になってきます。ライブラリ設計時点で定義されていないデータ型に対していかにしてライブラリのアルゴリズムを適用するか、つまり、拡張性が高いように作るかというのは、なかなか難しい問題です。簡潔に書けることを重視すると、拡張性が狭まりがちですし、拡張性が高いように作ると、デフォルトの動作でいいところを毎回書かなくてはいけなくて利用者にとって不便です。型クラスを使ったライブラリを提供することによって、この問題をある程度緩和することができます。皆さんも、型クラスを使って、既存の問題をより簡潔に、拡張性が高く解決できないか考えてみてください。

第 18 章

Future/Promise について

Future と Promise は非同期プログラミングにおいて、終了しているかどうかわからない処理結果を抽象化した型です。Future は未来の結果を表す型です。Promise は一度だけ、成功あるいは失敗を表す、処理または値を設定することで Future に変換できる型です。

JVM 系の言語では、マルチスレッドで並行処理を使った非同期処理を行うことが多々あります。無論ブラウザ上の JavaScript のようなシングルスレッドで行うような非同期処理もありますが、マルチスレッドで行う非同期処理は定義した処理群が随時行われるのではなく、マルチコアのマシンならば大抵の場合、複数の CPU で別々に実行されることとなります。

具体的に非同期処理が行われている例としては、UI における読み込み中のインジケーターなどがあげられます。読み込み中のインジケーターがアニメーションしている間も、ダイアログを閉じたり、別な操作をすることができるのは、読み込み処理が非同期でおこなわれているからです。

なお、このような特定のマシンの限られたリソースの中で、マルチスレッドやマルチプロセスによって順不同もしくは同時に処理を行うことを、並行（Concurrent）処理といいます。マルチスレッドの場合はプロセスとメモリ空間とファイルハンドラを複数のスレッドで共有し、マルチプロセスの場合はメモリ管理は別ですが CPU リソースを複数のプロセスで共有しています。（注、スレッドおよびプロセスのような概念については知っているものとみなして説明していますのでご了承ください）

リソースが共有されているかどうかにかかわらず、完全に同時に処理を行っていくことを、並列（Parallel）処理といいます。大抵の場合、複数のマシンで分散実行させるような分散系を利用したスケールするような処理を並列処理系と呼びます。

このたびはこのような並行処理を使った非同期処理を行った場合に、とても便利な Future と Promise というそれぞれのクラスの機能と使い方について説明を行います。

18.1 Future とは

Future とは、非同期に処理される結果が入った Option 型のようなものです。map や flatMap や

filter、for 式の適用といったような Option や List でも利用できる性質を持っています。

ライブラリやフレームワークの処理が非同期主体となっている場合、この Future は基本的で重要な役割を果たすクラスとなります。

なお Java にも Future というクラスがありますが、こちらには関数を与えたり[*1]、Option の持つ特性はありません。また、ECMAScript 6 にある Promise という機能がありますが、そちらの方が Scala の Future の機能に似ています。この ECMAScript 6 の Promise と Scala の Promise は、全く異なる機能であるため注意が必要です。

実際のコード例を見てみましょう。

```
1  import scala.concurrent.Future
2  import scala.concurrent.ExecutionContext.Implicits.global
3
4  object FutureSample {
5
6    def main(args: Array[String]): Unit = {
7      val s = "Hello"
8      val f: Future[String] = Future {
9        Thread.sleep(1000)
10       s + " future!"
11     }
12
13     f.foreach { s =>
14       println(s)
15     }
16
17     println(f.isCompleted) // false
18
19     Thread.sleep(5000) // Hello future!
20
21     println(f.isCompleted) // true
22
23     val f2: Future[String] = Future {
24       Thread.sleep(1000)
25       throw new RuntimeException("わざと失敗")
26     }
27
28     f2.failed.foreach { e =>
29       println(e.getMessage)
30     }
31
32     println(f2.isCompleted) // false
33
34     Thread.sleep(5000) // わざと失敗
35
36     println(f2.isCompleted) // true
37   }
38 }
```

[*1] ただし、Java 8 から追加された java.util.concurrent.Future のサブクラスである CompletableFuture には、関数を引数にとるメソッドがあります。

出力結果は、

```
false
Hello future!
true
false
わざと失敗
true
```

のようになります。

以上は Future 自体の機能を理解するためのサンプルコードです。非同期プログラミングは、sbt console で実装するのが難しいのでファイルに書かせてもらいました。Future シングルトンは関数を与えるとその関数を非同期に与える `Future[+T]` を返します。上記の実装例ではまず、1000 ミリ秒待機して、`"Hello"` と`" future!"` を文字列結合するという処理を非同期に処理します。そして成功時の処理を定義した後 future が処理が終わっているかを確認し、future の結果取得を 5000 ミリ秒間待つという処理を行った後、その結果がどうなっているのかをコンソールに出力するという処理をします。

なお以上のように 5000 ミリ秒待つという他に、その Future 自体の処理を待つという書き方もすることができます。`Thread.sleep(5000)` を `Await.ready(f, 5000.millisecond)` とすることで、Future が終わるまで最大 5000 ミリ秒を待つという書き方となります。ただし、この書き方をする前に、

```
1 import scala.concurrent.Await
2 import scala.concurrent.duration._
```

以上を import 文に追加する必要があります。さらにこれらがどのように動いているのかを、スレッドの観点から見てみましょう。以下のようにコードを書いてみます。

```
1  import scala.concurrent.{Await, Future}
2  import scala.concurrent.ExecutionContext.Implicits.global
3  import scala.concurrent.duration._
4
5  object FutureSample {
6
7    def main(args: Array[String]): Unit = {
8      val s = "Hello"
9      val f: Future[String] = Future {
10       Thread.sleep(1000)
11       println(s"[ThreadName] In Future: ${Thread.currentThread.getName}")
12       s + " future!"
13     }
14
15     f.foreach { s =>
16       println(s"[ThreadName] In Success: ${Thread.currentThread.getName}")
17       println(s)
18     }
19
20     println(f.isCompleted) // false
```

```
21
22      Await.ready(f, 5000.millisecond) // Hello future!
23
24      println(s"[ThreadName] In App: ${Thread.currentThread.getName}")
25      println(f.isCompleted) // true
26    }
27  }
```

この実行結果については、

```
false
[ThreadName] In Future: ForkJoinPool-1-worker-5
[ThreadName] In App: main
true
[ThreadName] In Success: ForkJoinPool-1-worker-5
Hello future!
```

となります。以上のコードではそれぞれのスレッド名を各箇所について出力してみました。非常に興味深い結果ですね。Future と foreach に渡した関数に関しては、ForkJoinPool-1-worker-5 という main スレッドとは異なるスレッドで実行されています。

つまり Future を用いることで知らず知らずのうちのマルチスレッドのプログラミングが実行されていたということになります。また、Await.ready(f, 5000.millisecond) で処理を書いたことで、isCompleted の確認処理のほうが、"Hello future!" の文字列結合よりも先に出力されていることがわかります。これは文字列結合の方が値参照よりもコストが高いためこのようになります。

ForkJoinPool に関しては、Java の並行プログラミングをサポートする ExecutorService というインタフェースを被ったクラスとなります。内部的にスレッドプールを持っており、スレッドを使いまわすことによって、スレッドを作成するコストを低減し高速化を図っています。

Future についての動きがわかった所で、Future が Option のように扱えることも説明します。

```
1   import scala.concurrent.ExecutionContext.Implicits.global
2   import scala.concurrent.Future
3   import scala.util.{Failure, Random, Success}
4
5   object FutureOptionUsageSample {
6     val random = new Random()
7     val waitMaxMilliSec = 3000
8
9     def main(args: Array[String]): Unit = {
10      val futureMilliSec: Future[Int] = Future {
11        val waitMilliSec = random.nextInt(waitMaxMilliSec)
12        if(waitMilliSec < 1000) throw new RuntimeException(s"waitMilliSec is ${
      waitMilliSec}" )
13        Thread.sleep(waitMilliSec)
14        waitMilliSec
15      }
16
17      val futureSec: Future[Double] = futureMilliSec.map(i => i.toDouble / 1000)
18
19      futureSec onComplete {
```

```
      case Success(waitSec) => println(s"Success! ${waitSec} sec")
      case Failure(t) => println(s"Failure: ${t.getMessage}")
    }

    Thread.sleep(3000)
  }
}
```

出力例としては、`Success! 1.538 sec` や `Failure: waitMilliSec is 971` というものになります。この処理では、3000 ミリ秒を上限としたランダムな時間を待ってその待ったミリ秒を返す Future を定義しています。ただし、1000 ミリ秒未満しか待たない場合には失敗とみなし例外を投げます。この最初にえられる Future を `futureMilliSec` としていますが、その後、`map` メソッドを利用して `Int` のミリ秒を `Double` の秒に変換しています。なお先ほどと違ってこの度は、`foreach` ではなく `onComplete` を利用して成功と失敗の両方の処理を記述しました。

以上の実装のように Future は結果を Option のように扱うことができるわけです。無論 map も使えますが Option がネストしている場合に flatMap を利用できるのと同様に、flatMap も Future に対して利用することもできます。つまり map の中での実行関数がさらに Future を返すような場合も問題なく Future を利用していけるのです。`val futureSec: Future[Double] = futureMilliSec.map(i => i.toDouble / 1000)` を上記のミリ秒を秒に変換する部分を 100 ミリ秒はかかる非同期の Future にしてみた例は以下のとおりです。

```
val futureSec: Future[Double] = futureMilliSec.flatMap(i => Future {
  Thread.sleep(100)
  i.toDouble / 1000
})
```

map で適用する関数で Option がとれてきてしまうのを flatten できるという書き方と同じように、Future に適用する関数の中でさらに Future が取得できるような場合では、flatMap が適用できます。この書き方のお陰で非常に複雑な非同期処理を、比較的シンプルなコードで表現してやることができるようになります。

18.1.1　Future を使って非同期に取れてくる複数の結果を利用して結果を作る

さて、flatMap が利用できるということは、for 式も利用できます。これらはよく複数の Future を組み合わせて新しい Future を作成するのに用いられます。実際に実装例を見てみましょう。

```
import scala.concurrent.ExecutionContext.Implicits.global
import scala.concurrent.Future
import scala.util.{Failure, Success, Random}

object CompositeFutureSample {
  val random = new Random()
  val waitMaxMilliSec = 3000

  def main(args: Array[String]): Unit = {
```

```
10      def waitRandom(futureName: String): Int = {
11        val waitMilliSec = random.nextInt(waitMaxMilliSec)
12        if(waitMilliSec < 500) throw new RuntimeException(s"${futureName} waitMilliSec is
          ${waitMilliSec}" )
13        Thread.sleep(waitMilliSec)
14        waitMilliSec
15      }
16
17      val futureFirst: Future[Int] = Future { waitRandom("first") }
18      val futureSecond: Future[Int] = Future { waitRandom("second") }
19
20      val compositeFuture: Future[(Int, Int)] = for {
21        first <- futureFirst
22        second <- futureSecond
23      } yield (first, second)
24
25      compositeFuture onComplete  {
26        case Success((first, second)) => println(s"Success! first:${first} second:${
          second}")
27        case Failure(t) => println(s"Failure: ${t.getMessage}")
28      }
29
30      Thread.sleep(5000)
31    }
32  }
```

先ほど紹介した例に似ていますが、ランダムで生成した最大 3 秒間待つ関数を用意し、500 ミリ秒未満しか待たなかった場合は失敗とみなします。その関数を実行する関数を Future として 2 つ用意し、それらを for 式で畳み込んで新しい Future を作っています。そして最終的に新しい Future に対して成功した場合と失敗した場合を出力します。

出力結果としては、`Success! first:1782 second:1227` や `Failure: first waitMilliSec is 412` や `Failure: second waitMilliSec is 133` といったものとなります。

なお Future には filter の他、様々な並列実行に対するメソッドが存在しますので、API ドキュメントを見てみてください。また複数の Future 生成や並列実行に関してのまとめられた日本語の記事もありますので、複雑な操作を試してみたい際にはぜひ参考にしてみてください。

18.2　Promise とは

Promise とは、

成功あるいは失敗を表す値を設定することによって Future に変換することのできるクラスです。実際にサンプルコードを示します。

```
1  import scala.concurrent.ExecutionContext.Implicits.global
2  import scala.concurrent.{Await, Promise, Future}
3  import scala.concurrent.duration._
4  import scala.util.{Success, Failure}
5
```

```
object PromiseSample {
  def main(args: Array[String]): Unit = {
    val promiseGetInt: Promise[Int] = Promise[Int]()
    val futureByPromise: Future[Int] = promiseGetInt.future // PromiseからFutureを作ることが出来る

    // Promiseが解決されたときに実行される処理をFutureを使って書くことが出来る
    val mappedFuture = futureByPromise.map { i =>
      println(s"Success! i: ${i}")
    }

    // 別スレッドで何か重い処理をして、終わったらPromiseに値を渡す
    Future {
      Thread.sleep(300)
      promiseGetInt.success(1)
    }

    Await.ready(mappedFuture, 5000.millisecond)
  }
}
```

この処理は必ず `Success! i: 1` という値を表示します。このように Promise に値を渡すことで（Promise から生成した）Future を完了させることができます。

上の例は Promise 自体の動作説明のために Future 内で Promise を使っています。通常は Future の返り値を利用すればよいため、今の使い方ではあまりメリットがありません。そこで今度は Promise のよくある使い方の例として、callback を指定するタイプの非同期処理をラップして Future を返すパターンを紹介します。

下記の例では、CallBackSomething をラップした FutureSomething を定義しています。`doSomething` の中で Promise が使われていることに注目してください。

```
import scala.concurrent.ExecutionContext.Implicits.global
import scala.concurrent.{Await, Future, Promise}
import scala.concurrent.duration._
import scala.util.{Failure, Random, Success}

class CallbackSomething {
  val random = new Random()

  def doSomething(onSuccess: Int => Unit, onFailure: Throwable => Unit): Unit = {
    val i = random.nextInt(10)
    if(i < 5) onSuccess(i) else onFailure(new RuntimeException(i.toString))
  }
}

class FutureSomething {
  val callbackSomething = new CallbackSomething

  def doSomething(): Future[Int] = {
    val promise = Promise[Int]()
    callbackSomething.doSomething(i => promise.success(i), t => promise.failure(t))
```

```
21      promise.future
22    }
23  }
24
25  object CallbackFuture {
26    def main(args: Array[String]): Unit = {
27      val futureSomething = new FutureSomething
28
29      val iFuture = futureSomething.doSomething()
30      val jFuture = futureSomething.doSomething()
31
32      val iplusj = for {
33        i <- iFuture
34        j <- jFuture
35      } yield i + j
36
37      val result = Await.result(iplusj, Duration.Inf)
38      println(result)
39    }
40  }
```

「Promise には成功/失敗した時の値を設定できる」「Promise から Future を作ることが出来る」という 2 つの性質を利用して、callback を Future にすることができました。

callback を使った非同期処理は今回のような例に限らず、Http クライアントで非同期リクエストを行う場合などで必要になることがあります。柔軟なエラー処理が必要な場合、callback より Future の方が有利な場面があるため、Promise を使って変換可能であることを覚えておくとよいでしょう。

18.2.1　演習：カウントダウンラッチ

それでは、演習をやってみましょう。Future や Promise の便利な特性を利用して、0〜1000 ミリ秒間のランダムな時間を待つ 8 個の Future を定義し、そのうちの 3 つが終わり次第すぐにその 3 つの待ち時間を全て出力するという実装をしてみましょう。なお、この動きは、Java の並行処理のためのユーティリティである、CountDownLatch というクラスの動きの一部を模したものとなります。

```
1   import java.util.concurrent.atomic.AtomicInteger
2   import scala.concurrent.ExecutionContext.Implicits.global
3   import scala.concurrent.{Promise, Future}
4   import scala.util.Random
5
6   object CountDownLatchSample {
7     def main(args: Array[String]): Unit = {
8       val indexHolder = new AtomicInteger(0)
9       val random = new Random()
10      val promises: Seq[Promise[Int]] = for {i <- 1 to 3} yield Promise[Int]()
11      val futures: Seq[Future[Int]] = for {i <- 1 to 8} yield Future[Int] {
12        val waitMilliSec = random.nextInt(1001)
13        Thread.sleep(waitMilliSec)
14        waitMilliSec
15      }
```

```
16      futures.foreach { f => f.foreach { waitMilliSec =>
17        val index = indexHolder.getAndIncrement
18        if(index < promises.length) {
19          promises(index).success(waitMilliSec)
20        }
21      }}
22      promises.foreach { p => p.future.foreach { waitMilliSec => println(waitMilliSec)}}
23      Thread.sleep(5000)
24    }
25  }
```

上記のコードを簡単に説明すると、指定された処理を行う Future の配列を用意し、それらがそれぞれ成功した時に AtomicInteger で確保されている index をアトミックにインクリメントさせながら、Promise の配列のそれぞれに成功結果を定義しています。そして、最後に Promise の配列から作り出した全ての Future に対して、コンソールに出力をさせる処理を定義します。基本的な Future と Promise を使った処理で表現されていますが、ひとつ気をつけなくてはいけないのは AtomicInteger の部分です。これは Future に渡した関数の中では、同じスレッドが利用されているとは限らないために必要となる部分です。別なスレッドから変更される値に関しては、値を原子的に更新するようにコードを書かなければなりません。プリミティブな値に関して原子的な操作を提供するのが AtomicInteger という Java のクラスとなります。[*2]以上が解答例でした。

ちなみに、このような複雑なイベント処理は既に Java の concurrent パッケージにいくつか実装があるので実際の利用ではそれらを用いることもできます。

[*2] 値の原子的な更新や同期の必要性などの並行処理に関する様々な話題の詳細な解説は本書の範囲をこえてしまうため割愛します。「Java Concurrency in Practice」ないしその和訳「Java 並行処理プログラミング ーその「基盤」と「最新 API」を究める」や「Effective Java」といった本でこれらの話題について学ぶことが出来ます。

第19章

テスト

ソフトウェアをテストすることは多くの開発者が必要なことだと認識していますが、テストという言葉の定義は各人で異なり話が噛み合わない、という状況が多々発生します。このような状況に陥る原因の多くは人や組織、開発するソフトウェアによってコンテキストが異なるにもかかわらず、言葉の定義について合意形成せずに話し始めるためです。

言葉に食い違いがある状態で会話をしても不幸にしかなりません。世の開発者全員で合意することは不可能かもしれませんが、プロジェクト内ではソフトウェアテストという言葉の定義について合意を形成しましょう。また、新しくプロジェクトに配属された場合は定義を確認しておきましょう。

本章では、ソフトウェアテストを下記の定義とします。この定義は古典と呼ばれている書籍『ソフトウェアテストの技法　第二版』をベースにしたものです。

> ソフトウェアテストとは、ソフトウェアが意図されたように動作し意図されないことは全て実行されないように設計されていることを検証するように設計されたプロセス、あるいは一連のプロセスである。

限られたリソースの中でうまくバグを発見できるテストを設計するためには、プロジェクトの仕様、採用した技術、開発の目的を理解する必要があります。こういった知識を得てこそ何を、なぜ、どのように検証するか考えられるようになります。そうして考えられた計画を用いて対象のソフトウェアを検証することが、テストという行為なのです。

19.1　テストの分類

テストは幾つかのグループに分類できますが、分類方法についても書籍、組織、チームによってその定義は様々です。プロジェクトに携わる際は、どういった定義でテストを分類しているか確認しておきましょう。参考までに、いくつかの分類例を示します。

- 実践テスト駆動開発での定義

  - ユニットテスト
    - オブジェクトは正しく振る舞っているか、またオブジェクトが扱いやすいかどうかをテストします。
  - インテグレーションテスト
    - 変更できないコードに対して、書いたコードが機能するかテストします。
  - 受け入れテスト
    - システム全体が機能するかテストします。
- JSTQB　ソフトウェアテスト標準用語集[*1]での定義
  - コンポーネントテスト（component testing）
    - ユニットテスト とも呼びます。
    - 個々のソフトウェアコンポーネントのテストを指します。
    - 独立してテストできるソフトウェアの最小単位をコンポーネントと呼びます。
  - 統合テスト
    - 統合したコンポーネントやシステムのインタフェースや相互作用の欠陥を摘出するためのテストです。
  - システムテスト
    - 統合されたシステムが、特定の要件を満たすことを実証するためのテストのプロセスです。
  - 受け入れテスト
    - システムがユーザのニーズ、要件、ビジネスプロセスを満足するかをチェックするためのテストです。
    - このテストによって、システムが受け入れ基準を満たしているか判定したり、ユーザや顧客がシステムを受け入れるかどうかを判定できます。
- 本テキストの初期執筆時に書かれていた定義
  - ユニットテスト（Unit Test）
    - 単体テスト とも呼びます。
    - プログラム全体ではなく小さな単位（例えば関数ごと）に実行されます。このテストの目的はその単体が正しく動作していることを確認することです。ユニットテスト用のフレームワークの種類はとても多く Java の JUnit や PHP の PHPUnit など xUnit と呼ばれることが多いです。
  - 結合テスト・統合テスト (Integration Test)
    - プログラム全体が完成してから実際に動作するかを検証するために実行します。人力で行う（例えば機能を開発した本人）ものや Selenium などを使って自動で実行するものがあります。
  - システムテスト・品質保証テスト (System Test)

[*1] https://jstqb.jp/dl/JSTQB-glossary-introduction.V3.2.J01.pdf

* 実際に利用される例に則した様々な操作を行い問題ないかを確認します。ウェブアプリケーションの場合ですと、例えばフォームに入力する値の境界値を超えた場合、超えない場合のレスポンスの検証を行ったり様々なウェブブラウザで正常に動作するかなどを確認します。

共通しているのは、分類ごとにテストの目的や粒度が異なること、どのような分類にせよ数多くのテストを通過する必要があることです。

19.2　ユニットテスト

ここでは、ユニットテストを小さな単位で自動実行できるテスト と定義して、ユニットテストにフォーカスして解説を行います。ユニットテストは Scala でのプログラミングにも密接に関わってくるためです。

ユニットテストを行う理由は大きく 3 つあげられます。

1. 実装の前に満たすべき仕様をユニットテストとして定義し、実装を行うことで要件漏れのない機能を実装することができる
2. ユニットテストによって満たすべき仕様がテストされた状態ならば、安心してリファクタリングすることができる
3. 全ての機能を実装する前に、単体でテストをすることができる

最初の理由であるテストコードをプロダクトコードよりも先に書くことをテストファーストと呼びます。そして、失敗するテストを書きながら実装を進めていく手法のことをテスト駆動開発（TDD: Test Driven Development）といいます。なお、TDD についてはそれ単独で章になり得るので本節では解説しません。

リファクタリングについては、次節で詳しく触れます。

ユニットテストを実装するにあたって、気をつけておきたいことが二点あります。

1. 高速に実行できるテストを実装する
2. 再現性のあるテストを実装する

高速に実行できることでストレスなくユニットテストを実行できるようになります。また、再現性を確保することでバグの原因特定を容易にできます。

19.3　リファクタリング

リファクタリングとは、ソフトウェアの仕様を変えること無く、プログラムの構造を扱いやすく変化させることです。

マーチン・ファウラーのリファクタリングでは、リファクタリングを行う理由として 4 つの理由が挙げられています。

1. ソフトウェア設計を向上させるため
2. ソフトウェアを理解しやすくするため
3. バグを見つけやすくするため
4. 早くプログラミングできるようにするため

また同書の中で TDD の提唱者であるケント・ベックは、以下に示す経験則から、リファクタリングは有効であると述べています。

1. 読みにくいプログラムは変更しにくい
2. ロジックが重複しているプログラムは変更しにくい
3. 機能追加に伴い、既存のコード修正が必要になるプログラムは変更しにくい
4. 複雑な条件分岐の多いプログラムは変更しにくい

リファクタリングは中長期的な開発の効率をあげるための良い手段であり、変更が要求されるソフトウェアでは切っても切ることができないプラクティスです。そしてリファクタリングを行う際に、ソフトウェアの仕様を変えなかったことを確認する手段としてユニットテストが必要になるのです。

19.4　テスティングフレームワーク

実際にユニットテストのテストコードを書く際には、テスティングフレームワークを利用します。Scala で広く利用されているテスティングフレームワークとして紹介されるのは以下の 2 つです。

- Specs2
- ScalaTest

今回は、マクロを用いて実装されている power assert という便利な機能を使いたいため、ScalaTest を利用します[*2]。

ScalaTest は、テスティングフレームワークの中でも 振舞駆動開発（BDD :Behavior Driven Development）をサポートしているフレームワークです。BDD では、テスト内にそのプログラムに与えられた機能的な外部仕様を記述させることで、テストが本質的に何をテストしようとしているのかをわかりやすくする手法となります。基本この書き方に沿って書いていきます。

19.5　テストができる sbt プロジェクトの作成

では、実際にユニットテストを書いてみましょう。まずはプロジェクトを作成します。

適当な作業フォルダにて以下を実行します。ここでは、`scalatest_study` を作り、さらに中に `src/main/scala` と `src/test/scala` の 2 つのフォルダを作りましょう。

`build.sbt` を用意して、以下を記述しておきます。

[*2] 渡された条件式の実行過程をダイアグラムで表示する `assert` は、一般に “power assert” と呼ばれています

```
name := "scalatest_study"

version := "1.0"

scalaVersion := "3.8.4"

libraryDependencies ++= Seq(
  "org.scalatest" %% "scalatest-flatspec" % "3.2.20" % "test",
  "org.scalatest" %% "scalatest-diagrams" % "3.2.20" % "test",
)
```

その後、scalatest_study フォルダ内で、sbt compile を実行してみましょう。

```
[info] Set current project to scalatest_study (in build file:/Users/dwango/workspace/
    scalatest_study/scalatest_study/)
[info] Updating {file:/Users/dwango/workspace/scalatest_study/scalatest_study/}
    scalatest_study...
[info] Resolving jline#jline;2.12.1 ...
[info] downloading https://repo1.maven.org/maven2/org/scalatest/scalatest-flatspec_3
    /3.2.17/scalatest-flatspec_3-3.2.17.jar ...
[info]  [SUCCESSFUL ] org.scalatest#scalatest-flatspec_3;3.2.17!scalatest-flatspec_3.
    jar(bundle) (5456ms)
[info] downloading https://repo1.maven.org/maven2/org/scalatest/scalatest-diagrams_3
    /3.2.17/scalatest-diagrams_3-3.2.17.jar ...
[info]  [SUCCESSFUL ] org.scalatest#scalatest-diagrams_3;3.2.17!scalatest-diagrams_3.
    jar(bundle) (5199ms)
[info] Done updating.
[success] Total time: 11 s, completed 2023/02/09 16:48:42
```

以上のように表示されれば、これで準備は完了です。

19.6　Calc クラスとそのテストを実際に作る

それでは、具体的なテストを実装してみましょう。下記の仕様を満たす Calc クラスを作成し、それらをテストしていきます。

- 整数の配列を取得し、それらを足し合わせた整数を返すことができる sum 関数を持つ
- 整数を 2 つ受け取り、分子を分母で割った浮動小数点の値を返すことができる div 関数を持つ
- 整数値を 1 つ受け取り、その値が素数であるかどうかのブール値を返す isPrime 関数を持つ

これを実装した場合、src/main/scala/Calc.scala は以下のようになります。

```
class Calc {

  /** 整数の配列を取得し、それらを足し合わせた整数を返す
    *
    * Intの最大を上回った際にはオーバーフローする
    */
  def sum(seq: Seq[Int]): Int = seq.foldLeft(0)(_ + _)
```

```
8
9   /** 整数を2つ受け取り、分子を分母で割った浮動小数点の値を返す
10    *
11    * 0で割ろうとした際には実行時例外が投げられる
12    */
13  def div(numerator: Int, denominator: Int): Double = {
14    if (denominator == 0) throw new ArithmeticException("/ by zero")
15    numerator.toDouble / denominator.toDouble
16  }
17
18  /** 整数値を一つ受け取り、その値が素数であるかどうかのブール値を返す */
19  def isPrime(n: Int): Boolean = {
20    if (n < 2) false else !((2 to Math.sqrt(n).toInt) exists (n % _ == 0))
21  }
22 }
```

次にテストケースについて考えます。

- sum 関数
  - 整数の配列を取得し、それらを足し合わせた整数を返すことができる
  - Int の最大を上回った際にはオーバーフローする
- div 関数
  - 整数を 2 つ受け取り、分子を分母で割った浮動小数点の値を返す
  - 0 で割ろうとした際には実行時例外が投げられる
- isPrime 関数
  - その値が素数であるかどうかのブール値を返す
  - 1000 万以下の値の素数判定を一秒以内で処理できる

以上のようにテストを行います。基本的にテストの設計は、

1. 機能を満たすことをテストする
2. 機能が実行できる境界値に対してテストする
3. 例外やログがちゃんと出ることをテストする

以上の考えが重要です。

XP（エクストリームプログラミング）のプラクティスに、不安なところを徹底的にテストするという考えがあり、基本それに沿います。

ひとつ目の満たすべき機能が当たり前に動くは当たり前のこととして、2 つ目の不安な要素のある境界値をしっかりテストする、というのはテストするケースを減らし、テストの正確性をあげるためのプラクティスの境界値テストとしても知られています。そして最後は、レアな事象ではあるけれど動かないと致命的な事象の取り逃しにつながる例外やログについてテストも非常に重要なテストです。このような例外やログにテストがないと例えば 1 か月に 1 度しか起こらないような不具合に対する対処が原因究明できなかったりと大きな問題につながってしまいます。

最小のテストを書いてみます。src/test/scala/CalcSpec.scala を以下のように記述します。

```
import org.scalatest.flatspec.AnyFlatSpec
import org.scalatest.diagrams.Diagrams

class CalcSpec extends AnyFlatSpec with Diagrams {

  val calc = new Calc

  "sum関数" should "整数の配列を取得し、それらを足し合わせた整数を返すことができる" in {
    assert(calc.sum(Seq(1, 2, 3)) === 6)
    assert(calc.sum(Seq(0)) === 0)
    assert(calc.sum(Seq(-1, 1)) === 0)
    assert(calc.sum(Seq()) === 0)
  }

  it should "Intの最大を上回った際にはオーバーフローする" in {
    assert(calc.sum(Seq(Integer.MAX_VALUE, 1)) === Integer.MIN_VALUE)
  }
}
```

テストクラスに Diagrams をミックスインし、assert メソッドの引数に期待する条件を記述していきます[*3]。Diagrams を使うことで、覚えるべき API を減らしつつテスト失敗時に多くの情報を表示できるようになります。

テストを実装したら sbt test でテストを実行してください。以下のような実行結果が表示されます。

```
[info] Loading project definition from /Users/dwango/workspace/scalatest_study/project
[info] Set current project to scalatest_study (in build file:/Users/dwango/workspace/
    scalatest_study/)
[info] Compiling 1 Scala source to /Users/dwango/workspace/scalatest_study/target/scala
    -3.8.4/classes...
[info] Compiling 1 Scala source to /Users/dwango/workspace/scalatest_study/target/scala
    -3.8.4/test-classes...
[info] CalcSpec:
[info] sum関数
[info] - should 整数の配列を取得し、それらを足し合わせた整数を返すことができる
[info] - should Intの最大を上回った際にはオーバーフローする
[info] Run completed in 570 milliseconds.
[info] Total number of tests run: 2
[info] Suites: completed 1, aborted 0
[info] Tests: succeeded 2, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 12 s, completed 2015/12/25 1:25:56
```

実行結果から、すべてのテストに成功したことを確認できます。なお、わざと失敗した場合にはどのように表示されるのか確認してみましょう。

[*3] Scala には Predef にも assert が存在しますが、基本的に使うことはありません

```
[info] Loading project definition from /Users/dwango/workspace/scalatest_study/project
[info] Set current project to scalatest_study (in build file:/Users/dwango/workspace/
    scalatest_study/)
[info] Compiling 1 Scala source to /Users/dwango/workspace/scalatest_study/target/scala
    -3.8.4/test-classes...
[info] CalcSpec:
[info] sum関数
[info] - should 整数の配列を取得し、それらを足し合わせた整数を返すことができる *** 
    FAILED ***
[info]   assert(calc.sum(Seq(1, 2, 3)) === 7)
[info]          |    |  ||   |  |  |   |   |
[info]          |    6  ||   1  2  3   |   7
[info]          |       |List(1, 2, 3) false
[info]          |       6
[info]          Calc@e72a964 (CalcSpec.scala:8)
[info] - should Intの最大を上回った際にはオーバーフローする
[info] Run completed in 288 milliseconds.
[info] Total number of tests run: 2
[info] Suites: completed 1, aborted 0
[info] Tests: succeeded 1, failed 1, canceled 0, ignored 0, pending 0
[info] *** 1 TEST FAILED ***
[error] Failed tests:
[error]     CalcSpec
[error] (test:test) sbt.TestsFailedException: Tests unsuccessful
[error] Total time: 7 s, completed 2015/12/25 1:39:59
```

どこがどのように間違ったのかを指摘してくれます。

次に、例外が発生することをテストする場合について記述してみましょう。div 関数までテストの実装を進めます。

```
1  import org.scalatest.flatspec.AnyFlatSpec
2  import org.scalatest.diagrams.Diagrams
3
4  class CalcSpec extends AnyFlatSpec with Diagrams {
5
6    val calc = new Calc
7
8    // ...
9
10   "div関数" should "整数を2つ受け取り、分子を分母で割った浮動小数点の値を返す" in {
11     assert(calc.div(6, 3) === 2.0)
12     assert(calc.div(1, 3) === 0.3333333333333333)
13   }
14
15   it should "0で割ろうとした際には実行時例外が投げられる" in {
16     intercept[ArithmeticException] {
17       calc.div(1, 0)
18     }
19   }
20 }
```

上記では最後の部分でゼロ除算の際に投げられる例外をテストしています。intercept[Exception] という構文で作ったスコープ内で投げられる例外がある場合には成功となり、例外がない場合には逆にテストが失敗します。

最後にパフォーマンスを保証するテストを書きます。なお、本来ユニットテストは時間がかかるテストを書くべきではありませんが、できるだけ短い時間でそれを判定できるように実装します。

```
import org.scalatest.flatspec.AnyFlatSpec
import org.scalatest.diagrams.Diagrams
import org.scalatest.concurrent.TimeLimits
import org.scalatest.time.SpanSugar._

class CalcSpec extends AnyFlatSpec with Diagrams with TimeLimits {

  val calc = new Calc

  // ...

  "isPrime関数" should "その値が素数であるかどうかのブール値を返す" in {
    assert(calc.isPrime(0) === false)
    assert(calc.isPrime(-1) === false)
    assert(calc.isPrime(2))
    assert(calc.isPrime(17))
  }

  it should "1000万以下の値の素数判定を一秒以内で処理できる" in {
    failAfter(1000.millis) {
      assert(calc.isPrime(9999991))
    }
  }
}
```

TimeLimits というトレイトを利用することで failAfter という処理時間をテストする機能を利用できるようになります。

最終的に全てのテストをまとめて sbt test で実行すると以下の様な出力が得られます。

```
[info] Loading project definition from /Users/dwango/workspace/scalatest_study/project
[info] Set current project to scalatest_study (in build file:/Users/dwango/workspace/
    scalatest_study/)
[info] Compiling 1 Scala source to /Users/dwango/workspace/scalatest_study/target/scala
    -3.8.4/test-classes...
[info] CalcSpec:
[info] sum関数
[info] - should 整数の配列を取得し、それらを足し合わせた整数を返すことができる
[info] - should Intの最大を上回った際にはオーバーフローする
[info] div関数
[info] - should 整数を2つ受け取り、分子を分母で割った浮動小数点の値を返す
[info] - should 0で割ろうとした際には実行時例外が投げられる
[info] isPrime関数
[info] - should その値が素数であるかどうかのブール値を返す
[info] - should 1000万以下の値の素数判定を一秒以内で処理できる
[info] Run completed in 280 milliseconds.
```

```
[info] Total number of tests run: 6
[info] Suites: completed 1, aborted 0
[info] Tests: succeeded 6, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 8 s, completed 2015/12/25 1:43:22
```

以上が基本的なテストを実装するための機能の紹介でした。BDD でテストを書くことによってテストによってどのような仕様が満たされた状態であるのかというのがわかりやすい状況になっていることがわかります。

19.7　モック

モックとは、テストをする際に必要となるオブジェクトを偽装して用意できる機能です。以下の様なモックライブラリが存在しています。

- ScalaMock
- JMock
- Mockito

ここでは、よく使われている Mockito を利用してみましょう。`build.sbt` に以下を追記することで利用可能になります。

```
1 libraryDependencies += "org.mockito" % "mockito-core" % "5.24.0" % "test"
```

なお、mockito の version 5 以降は Java 11 以上が必要です。8 を使用している場合は mockito の version 4 を使ってください。

せっかくなので、先ほど用意した Calc クラスのモックを用意して、モックに sum の振る舞いを仕込んで見ましょう。

```
1  import org.scalatest.time.SpanSugar._
2  import org.scalatest.flatspec.AnyFlatSpec
3  import org.scalatest.diagrams.Diagrams
4  import org.scalatest.concurrent.TimeLimits
5  import org.mockito.Mockito._
6
7  class CalcSpec extends AnyFlatSpec with Diagrams with TimeLimits {
8
9    // ...
10
11   "Calcのモックオブジェクト" should "振る舞いを偽装することができる" in {
12     val mockCalc = mock(classOf[Calc])
13     when(mockCalc.sum(Seq(3, 4, 5))).thenReturn(12)
14     assert(mockCalc.sum(Seq(3, 4, 5)) === 12)
15   }
16 }
```

`val mockCalc = mock(classOf[Calc])` でモックオブジェクトを作成し、`when(mockCalc.sum(Seq(3, 4, 5)).thenReturn(12)` で振る舞いを作成しています。

そして最後に、`assert(mockCalc.sum(Seq(3, 4, 5)) === 12)` でモックに仕込んだ偽装された振る舞いをテストしています。

以上のようなモックの機能は、実際には時間がかかってしまう通信などの部分を高速に動かすために利用されています。

モックを含め、テストの対象が依存しているオブジェクトを置き換える代用品の総称をテストダブル[*4]と呼びます。

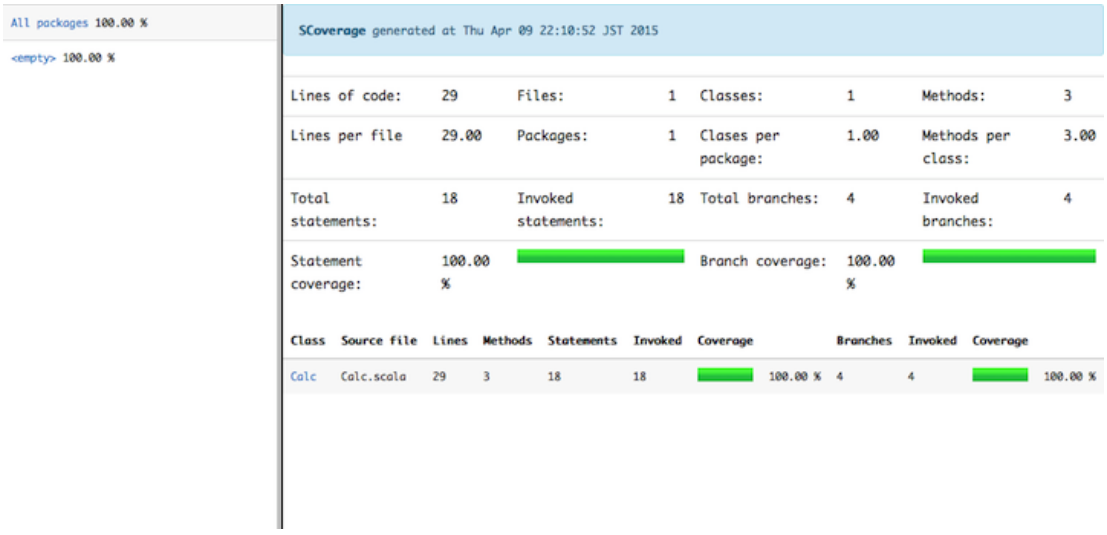
19.8　コードカバレッジの測定

テストを行った際に、テストが機能のどれぐらいを網羅できているのかを知る方法として、コードカバレッジを計測するという方法があります。ここでは、scoverage を利用します。

`project/plugins.sbt` に以下のコードを記述します。

```
1 addSbtPlugin("org.scoverage" % "sbt-scoverage" % "<latest>")
```

その後、`sbt clean coverage test coverageReport` を実行することで、`target/scala-3.8.4/scoverage-report/index.html` にレポートが出力されます。



以上の出力から、今回のテストはカバレッジ 100 %であることがわかります。

ここで、カバレッジを計測することについて注意点を述べておきます。

得られたカバレッジはあくまで” そのカバレッジツールによって得られた数値” でしかありません。数値自体が絶対的な評価を表しているわけではなく、書かれたプロダクトコードに対して、テストコードが N %カバーしているという事実を示しているだけです。カバレッジはツールや測定手法に

[*4] モック以外の仕組みについては xUnit Test Patterns を参照してください

よって結果が変動します。例えば JavaScript 用のカバレッジツールである istanbul は total が 0 件の場合 100% と表示しますが、これは単にツールの実装がそうなっているというだけの話です。

利用しているカバレッジツールがどのように動作するのか把握した上で、カバレッジレポートからどのコードが実行されなかったのか情報収集しましょう。数値だけに目を向けるのではなく、カバレッジ測定によってどういった情報が得られたのかを考えてください。そうすれば、ツールに振り回されることなくテストの少ない部分を発見できるでしょう。

19.9　コードスタイルチェック

なおテストとは直接は関係ありませんが、ここまでで紹介したテストは、実際には Jenkins などの継続的インテグレーションツール（CI ツール）で実施され、リグレッションを検出するためにつかわれます。その際に、CI の一環として一緒に行われることが多いのがコードスタイルチェックです。

コードスタイルチェックにおいて、インデントの数や定義の前後のスペースなどコードのフォーマットをチェックするにはフォーマッタを、静的解析で推奨される Scala コードの書き方になっているかをチェックするにはリンターを使います。例えば、次のメソッド定義は Procedure Syntax と呼ばれ、比較的新しいバージョンの Scala では非推奨になっています。リンターはこのような望ましくないコードを検知して報告します。

```
object Example {
  def myProcedure {}
}
```

2022 年現在では scalafmt がデファクトのフォーマッタとして、scalafix や wartremover がリンターとして使われています。scalafmt、scalafix、wartremover は Scala 3 に対応しています。scalafmt、scalafix のどちらもコードスタイルチェックに加えて自動修正機能があります。

scalafmt、scalafix は CLI でも sbt plugin でも使えますがここでは sbt plugin を利用する前提で話をすすめます。

`project/plugins.sbt` に以下のコードを記述します。

```
addSbtPlugin("org.scalameta" %% "scalafmt" % "<latest>")
addSbtPlugin("ch.epfl.scala" % "sbt-scalafix" % "<latest>")
```

19.9.1　scalafmt

まずは scalafmt を試してみましょう。sbt シェルを開いて次のコマンドを実行してください。もしソースにフォーマットに問題があるコードが含まれているなら警告が表示されます。

```
scalafmtCheckAll
```

次のコマンドを実行するとフォーマットを自動的に修正します。

```
scalafmtAll
```

また、build.sbt などのビルドにつかう Scala ファイルのフォーマットは scalafmtSbt で修正できます。

フォーマットは.scalafmt.conf ファイルで設定できます。詳しくは公式ドキュメントにひととおり目を通してみるといいでしょう。https://scalameta.org/scalafmt/docs/configuration.html

19.9.2　scalafix

さて、次は scalafix を試してみましょう。sbt シェルで次のコマンドを実行してください。

```
scalafix --check --rules ProcedureSyntax
```

下記のように警告が表示されます。

```
[info] Running scalafix on 1 Scala sources
[error] --- /path/to/problematic/File.scala
[error] +++ <expected fix>
...省略...
[error] -  def myProcedure {}
[error] +  def myProcedure: Unit = {}
[error]
...省略...
[error]
ScalafixFailed: TestError
```

問題のあるコードを自動的に書き換えるには--check オプションを外して scalafix ProcedureSyntax コマンドを実行します。先ほどの myProcedure は次のように修正されます。

```
1 object Example {
2   def myProcedure: Unit = {}
3 }
```

Procedure Syntax の警告と修正は scalafix にデフォルトで用意されていますが、ライブラリとして公開されている scalafix のルールを使うことでリンターのルールを追加できます。公開されているルールは scalafix のドキュメントに書いてあります。

19.10　テストを書こう

開発者として意識してほしいことを挙げておきます。

- 不具合を見つけたら、可能であれば再現手順もあわせて報告する
- パッチを送る際はテストを含める
- 不具合にテストを書いて立ち向かう

他人から報告された不具合は、伝聞ということもありどうしても再現させづらい場合があります。その結果として「私の環境では再現しませんでした」と言われて終わってしまうのでは、不具合を修

正するせっかくのチャンスを逃すことになってしまいます。そんな状況を回避するためにも、可能であれば不具合の再現手順や再現コードを用意し、不具合報告に含めましょう。再現手順を元に素早く不具合が修正される可能性が高まります。

　他人のコードをテストするのは、ちょっとしたコードであっても骨の折れる作業です。そんな作業を、限られたリソースですべてのパッチに対して行うのは不可能に近いので、プロジェクトによってはテストコードのないパッチはレビュー対象から外すことが多いです。パッチを送る場合は相手側が自信を持ってパッチを取り込めるよう、テストを含めておきましょう。

第 20 章

Java との相互運用

20.1 Scala と Java

Scala は JVM(Java Virtual Machine) の上で動作するため、Java のライブラリのほとんどをそのまま Scala から呼びだすことができます。また、現状では、Scala の標準ライブラリだけでは、どうしても必要な機能が足りず、Java の機能を利用せざるを得ないことがあります。ただし、Java の機能と言っても、Scala のそれとほとんど同じように利用することができます。

20.1.1 import

Java のライブラリを import するためには、Scala でほとんど同様のことを記述すれば OK です。

```
import java.util.*;
import java.util.ArrayList;
```

ワイルドカードインポートは Scala 2 では_ を、Scala 3 では* を使います。

```
1 import java.util._
2 import java.util.ArrayList
```

20.1.2 インスタンスの生成

インスタンスの生成も Java と同様にできます。Java での

```
ArrayList<String> list = new ArrayList<>();
```

というコードは Scala では

```
1 val list = new ArrayList[String]()
2 // list: ArrayList[String] = [Hello, World]
```

と記述することができます。

20.1.2.1　練習問題

java.util.HashSet クラスのインスタンスを new を使って生成してみましょう。

```
import java.util.HashSet
val set = new HashSet[String]
// set: HashSet[String] = []
```

20.1.3　インスタンスメソッドの呼び出し

インスタンスメソッドの呼び出しも同様です。

```
list.add("Hello");
list.add("World");
```

は

```
list.add("Hello")
// res0: Boolean = true
list.add("World")
// res1: Boolean = true
```

と同じです。

20.1.3.1　練習問題

java.lang.System クラスのフィールド out のインスタンスメソッド println を引数"Hello, World!" として呼びだしてみましょう。

```
System.out.println("Hello, World!")
```

20.1.3.2　static メソッドの呼び出し

static メソッドの呼び出しも Java の場合とほとんど同様にできますが、1 つ注意点があります。それは、Scala では static メソッドは継承されない（というより static メソッドという概念がない）ということです。これは、クラス A が static メソッド foo を持っていたとして、A を継承した B に対して B.foo() とすることはできず、A.foo() としなければならないという事を意味します。それ以外の点については Java の場合とほぼ同じです。

現在時刻をミリ秒単位で取得する System.currentTimeMillis() を Scala から呼び出してみましょう。

```
scala> System.currentTimeMillis()
res0: Long = 1416357548906
```

表示される値はみなさんのマシンにおける時刻に合わせて変わりますが、問題なく呼び出せているはずです。

■練習問題　java.lang.System クラスの static メソッド exit() を引数 0 として呼びだしてみましょう。どのような結果になるでしょうか。

```
System.exit(0)
```

実行中の Scala プログラム（プロセス）が終了する。

20.1.3.3　static フィールドの参照

static フィールドの参照も Java の場合と基本的に同じですが、static メソッドの場合と同じ注意点が当てはまります。つまり、static フィールドは継承されない、ということです。たとえば、Java では JFrame.EXIT_ON_CLOSE が継承されることを利用して、

```
import javax.swing.JFrame;

public class MyFrame extends JFrame {
  public MyFrame() {
    setDefaultCloseOperation(EXIT_ON_CLOSE); //JFrameを継承しているので、EXIT_ON_CLOSE
        だけでOK
  }
}
```

のようなコードを書くことができますが、Scala では同じように書くことができず、

```
import javax.swing.JFrame

class MyFrame extends JFrame {
  // EXIT_ON_CLOSE?JFrame??????????????
  // ????interface?????????
  setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE)
}
```

のように書く必要があります。

現実のプログラミングでは、Scala の標準ライブラリだけでは必要なライブラリが不足している場面に多々遭遇しますが、そういう場合は既にあるサードパーティの Scala ライブラリか Java ライブラリを直接呼びだすのが基本になります。

■練習問題　Scala で Java の static フィールドを参照しなければならない局面を 1 つ以上挙げてみましょう。

java.lang.System クラスの static フィールド err を参照する場合

20.1.3.4　Scala の型と Java の型のマッピング

Java の型は適切に Scala にマッピングされます。たとえば、System.currentTimeMillis() が返す型は long 型ですが、Scala の標準の型である scala.Long にマッピングされます。Scala の型と Java の型のマッピングは次のようになります。

Java のすべてのプリミティブ型に対応する Scala の型が用意されていることがわかりますね！
また、`java.lang` パッケージにあるクラスは全て Scala から import 無しに使えます。

また、参照型についても Java 同様にクラス階層の中に組み込まれています。たとえば、Java で言う `int[]` は `Array[Int]` と書きますが、これは `AnyRef` のサブクラスです。ということは、Scala で `AnyRef` と書くことで `Array[Int]` を `AnyRef` 型の変数に代入可能です。ユーザが定義したクラスも同様で、基本的に `AnyRef` を継承していることになっています。（ただし、value class というものがあり、それを使った場合は少し事情が異なりますがここでは詳細には触れません）

20.1.3.5　null と Option

Scala の世界では null を使うことはなく、代わりに Option 型を使います。一方で、Java のメソッドを呼び出したりすると、返り値として null が返ってくることがあります。Scala の世界ではできるだけ null を取り扱いたくないのでこれは少し困ったことです。幸いにも、Scala では `Option(value)` とすることで、`value` が null のときは `None` が、null でないときは `Some(value)` を返すようにできます。

java.util.Map を使って確かめてみましょう。

```
1  val map = new java.util.HashMap[String, Int]()
2  // map: HashMap[String, Int] = {A=1, B=2, C=3}
3
4  map.put("A", 1)
5  // res2: Int = 0
6
7  map.put("B", 2)
8  // res3: Int = 0
9
10 map.put("C", 3)
11 // res4: Int = 0
12
13 Option(map.get("A"))
14 // res5: Option[Int] = Some(1)
15
16 Option(map.get("B"))
17 // res6: Option[Int] = Some(2)
18
19 Option(map.get("C"))
20 // res7: Option[Int] = Some(3)
21
22 Option(map.get("D"))
23 // res8: Option[Int] = None
```

ちゃんと null が Option にラップされていることがわかります。Scala の世界から Java のメソッドを呼びだすときは、返り値をできるだけ Option() でくるむように意識しましょう。

20.1.3.6　scala.jdk.CollectionConverters

Java のコレクションと Scala のコレクションはインタフェースに互換性がありません。これでは、Scala のコレクションを Java のコレクションに渡したり、逆に返ってきた Java のコレクションを Scala のコレクションに変換したい場合に不便です。そのような場合に便利なのが scala.jdk.CollectionConverters です。Scala 2.12 以前は同様の機能は scala.collection.JavaConverters で提供されていましたが、Scala 2.13 以降はそれが非推奨になりました。使い方はいたって簡単で、

```
import scala.jdk.CollectionConverters._
```

とするだけです。これで、Java と Scala のコレクションのそれぞれに asJava() や asScala() といったメソッドが追加されるのでそのメソッドを以下のように呼び出せば良いです。

```
import scala.jdk.CollectionConverters._
import java.util.ArrayList

val list = new ArrayList[String]()
// list: ArrayList[String] = [A, B]

list.add("A")
// res9: Boolean = true

list.add("B")
// res10: Boolean = true

val scalaList = list.asScala
// scalaList: Buffer[String] = Buffer("A", "B")
```

Buffer は Scala の変更可能なリストのスーパークラスですが、ともあれ、asScala メソッドによって Java のコレクションを Scala のそれに変換することができていることがわかります。そのほかのコレクションについても同様に変換できますが、詳しくは API ドキュメントを参照してください。

また、scala.jdk パッケージには、コレクションの変換以外の機能も提供されています。

■練習問題　scala.collection.mutable.ArrayBuffer 型の値を生成してから、scala.jdk.CollectionConverters を使って java.util.List 型に変換してみましょう。なお、ArrayBuffer には 1 つ以上の要素を入れておくこととします。

```
import scala.collection.mutable.ArrayBuffer
import scala.jdk.CollectionConverters._
val buffer = new ArrayBuffer[String]
// buffer: ArrayBuffer[String] = ArrayBuffer("A", "B", "C")
buffer += "A"
// res11: ArrayBuffer[String] = ArrayBuffer("A", "B", "C")
buffer += "B"
// res12: ArrayBuffer[String] = ArrayBuffer("A", "B", "C")
buffer += "C"
// res13: ArrayBuffer[String] = ArrayBuffer("A", "B", "C")
```

```
val list = buffer.asJava
// list: List[String] = [A, B, C]
```

20.1.3.7　ワイルドカードと存在型

Java では、

```
import java.util.List;
import java.util.ArrayList;
List<? extends Object> objects = new ArrayList<String>();
```

のようにして、クラス宣言時には不変であった型パラメータを共変にしたり、

```
import java.util.Comparator;
Comparator<? super String> cmp = new Comparator<Object>() {
  public int compare(Object o1, Object o2) {
    return o1.hashCode() - o2.hashCode();
  }
};
```

のようにして反変にすることができます。ここで、`? extends Object` の部分を共変ワイルドカード、`? super String` の部分を反変ワイルドカードと呼びます。より一般的には、このような機能を、利用側で変位指定するという意味で use-site variance と呼びます。

この機能に対応するものとして、Scala には存在型があります。上記の Java コードは、Scala では次のコードで表現することができます。

```
import java.util.{List => JList, ArrayList => JArrayList}

val objects: JList[? <: Object] = new JArrayList[String]()
// objects: List[_ >: Nothing <: Object] = []
```

```
import java.util.{Comparator => JComparator}

val cmp: JComparator[? >: String] = new JComparator[Any] {
  override def compare(o1: Any, o2: Any): Int = {
    o1.hashCode() - o2.hashCode()
  }
}
// cmp: Comparator[_ >: String <: Any] = repl.MdocSession$MdocApp$$anon$23@2b22b295
```

より一般的には、`G<? extends T>` は `G[? <: T]` に、`G<? super T>` は `G[? >: T]` に置き換えることができます。Scala のプログラム開発において、Java のワイルドカードを含んだ型を扱いたい場合は、この機能を使いましょう。一方で、Scala プログラムでは定義側の変位指定、つまり declaration-site variance を使うべきであって、Java と関係ない部分においてこの機能を使うのはプログラムをわかりにくくするため、避けるべきです。

20.1.3.8　SAM 変換

Scala 2.12 では SAM(Single Abstract Method) 変換が導入され[*1]、Java 8 のラムダ式を想定したライブラリを簡単に利用できるようになりました。Java 8 におけるラムダ式とは、関数型インタフェースと呼ばれる、メソッドが 1 つしかないようなインタフェースに対して無名クラスを簡単に記述できる構文です[*2]。例えば、10 の階乗を例にすると以下のように簡潔に書くことができます。

```
import java.util.stream.IntStream;
int factorial10 = IntStream.rangeClosed(1, 10).reduce(1, (i1, i2) -> i1 * i2);
```

ちなみに、これをラムダ式を使わずに書くと、以下のようにとても大変です。

```
import java.util.stream.IntStream;
import java.util.function.IntBinaryOperator;
int factorial10 = IntStream.rangeClosed(1, 10).reduce(1,
  new IntBinaryOperator() {
    @Override public int applyAsInt(int left, int right) {
      return left * right;
    }
  });
```

関数の章で説明したように、元々 Scala にもラムダ式に相当する無名関数という構文があります。しかし、以前の Scala では `FunctionN` 型が期待される箇所に限定されており、Java においてラムダ式が期待される箇所の大半において使用することができませんでした。例えば、10 の階乗の例は `IntBinaryOperator` 型が期待されているので以下のように無名クラスを使う必要がありました。

```
1 import java.util.stream.IntStream;
2 import java.util.function.IntBinaryOperator;
3 val factorial10 = IntStream.rangeClosed(1, 10).reduce(1,
4   new IntBinaryOperator {
5     def applyAsInt(left: Int, right: Int) = left * right;
6   });
7 // factorial10: Int = 3628800
```

SAM 変換を利用すると以下のようにここにも無名関数を利用できるようになります。

```
1 import java.util.stream.IntStream;
2 val factorial10 = IntStream.rangeClosed(1, 10).reduce(1, _ * _);
3 // factorial10: Int = 3628800
```

[*1] 正確には`-Xexperimetal` オプションにより、Scala 2.11 でも SAM 変換を有効にすることができます。

[*2] 厳密に言うと、無名クラスを用いたコードとラムダ式もしくは無名関数を用いたコードの間には、Java と Scala いずれにおいても細かな違いが存在します。例えば、スコープや出力されるバイトコードなどです。より詳しくは言語仕様などを当たってみてください。

第 21 章

S99 の案内

ここでは、Scala を用いたプログラミングについてより深く理解するために適していると思われる問題集である S99 について簡単に紹介します。

21.1 S-99: Ninety-Nine Scala Problems

URL: https://aperiodic.net/pip/scala/s-99/
元々は Prolog 用の Ninety-Nine Prolog Problems を Scala 版に移植した問題集です。

- P01〜P28：リスト操作
- P31〜P41：数値演算等
- P46〜P50：論理演算
- P55〜P69：二分木
- P70〜P73：多分木
- P80〜P89：グラフ
- P90〜P99：その他の問題

と問題の種類に応じて区分けされています。とりわけ、Scala を使うときはコレクションやリスト操作を多用するので、P01〜P28 については一通り解けるようになっておくと良いでしょう。その他は個人の興味に応じて適宜色々な問題を解いてみてください。

なお、S99 の問題の模範解答として、
https://github.com/scala-text/S99
というリポジトリを用意してあります。全ての解答が整備されているわけではありませんが、考えても解答がわからない場合は参考にしても良いでしょう。また、各問題の解答についてのテストケースも書いてあります。

第22章

トレイトの応用編：依存性の注入によるリファクタリング

ここではトレイトの応用編として、大きなクラスをリファクタリングする過程を通してトレイトを実際どのように使っていくかを学んでいきましょう。さらに大規模システム開発で使われる依存性の注入という技法についても紹介します。

22.1 サンプルプログラム

今回使われるサンプルプログラムは以下の場所にあります。実際に動かしたい場合は個々のディレクトリに入り、sbt を起動してください。

リファクタリング前のプログラム

リファクタリング後のプログラム

22.2 リファクタリング前のプログラムの紹介

今回は今までより実践的なプログラムを考えてみましょう。ユーザーの登録と認証を管理する `UserService` というクラスです。

```
scalaVersion := "3.8.4"

libraryDependencies ++= Seq(
  "org.scalikejdbc" %% "scalikejdbc" % "4.3.5",
  "org.mindrot"     %  "jbcrypt"     % "0.4"
)
```

./example_projects/trait-example/build.sbt

```
package domain

```

```
case class User(id: Long, name: String, hashedPassword: String)

object User {
  def apply(name: String, hashedPassword: String): User =
    User(0L, name, hashedPassword)
}
```

./example_projects/trait-example/src/main/scala/domain/User.scala

```
package domain

import org.mindrot.jbcrypt.BCrypt
import scalikejdbc._

class UserService {
  val maxNameLength = 32

  // ストレージ機能
  def insert(user: User): User = DB localTx { implicit s =>
    val id = sql"""insert into users (name, password) values (${user.name}, ${user.
    hashedPassword})"""
      .updateAndReturnGeneratedKey.apply()
    user.copy(id = id)
  }

  def createUser(rs: WrappedResultSet): User =
    User(rs.long("id"), rs.string("name"), rs.string("password"))

  def find(name: String): Option[User] = DB readOnly { implicit s =>
    sql"""select * from users where name = $name """
      .map(createUser).single.apply()
  }

  def find(id: Long): Option[User] = DB readOnly { implicit s =>
    sql"""select * from users where id = $id """
      .map(createUser).single.apply()
  }

  // パスワード機能
  def hashPassword(rawPassword: String): String =
    BCrypt.hashpw(rawPassword, BCrypt.gensalt())

  def checkPassword(rawPassword: String, hashedPassword: String): Boolean =
    BCrypt.checkpw(rawPassword, hashedPassword)

  // ユーザー登録
  def register(name: String, rawPassword: String): User = {
    if (name.length > maxNameLength) {
      throw new Exception("Too long name!")
    }
    if (find(name).isDefined) {
      throw new Exception("Already registered!")
    }
```

```
44      insert(User(name, hashPassword(rawPassword)))
45    }
46
47    // ユーザー認証
48    def login(name: String, rawPassword: String): User = {
49      find(name) match {
50        case None       => throw new Exception("User not found!")
51        case Some(user) =>
52          if (!checkPassword(rawPassword, user.hashedPassword)) {
53            throw new Exception("Invalid password!")
54          }
55          user
56      }
57    }
58  }
```

./example_projects/trait-example/src/main/scala/domain/UserService.scala

UserService は以下のような機能があります。

- register メソッドはユーザーの登録をおこなうメソッドで、ユーザーの名前とパスワードを引数として受け取り、名前の最大長のチェックと既に名前が登録されているかどうかを調べて、ストレージに保存する
- login メソッドはユーザーの認証をおこなうメソッドで、ユーザーの名前とパスワードを受け取り、ストレージに保存されているユーザーの中から同名のユーザーを見つけ出し、パスワードをチェックする
- この他のストレージ機能とパスワード機能のメソッドは内部的に使われるのみである

上記のプログラムでは実際に動かすことができるように ScalikeJDBC というデータベース用のライブラリと jBCrypt というパスワースのハッシュ値を計算するライブラリが使われていますが、実装の詳細を理解する必要はありません。既にメソッドの実装を説明したことがある場合は実装を省略し ??? で書くことがあります。

22.3　リファクタリング：公開する機能を制限する

さて、モジュール化という観点で UserService を見ると、どういった問題が考えられるでしょうか？

1 つは UserService が必要以上に情報を公開しているということです。UserService の役割は register メソッドを使ったユーザー登録と login メソッドを使ったユーザーの認証です。しかしストレージ機能である insert メソッドや find メソッドも公開してしまっています。

register メソッドはユーザーの名前の最大長や既にユーザーが登録されているかどうかチェックしていますが、insert メソッドはそういったチェックをせずにデータベースに保存しています。もし insert メソッドを直接使われた場合、想定外に名前が長いユーザーや、名前が重複したユーザーが保存されてしまいます。

find メソッドも同様の問題があります。login メソッドではなく直接 find メソッドが使われた場合、パスワードのチェックなしにユーザーの情報を取得することができます。

では、どのような修正が考えられるでしょうか。まず考えられるのは外部から使ってほしくないメソッドを private にすることです。

```
class UserService {
  // メソッドの実装は同じなので???で代用しています
  val maxNameLength = 32

  // ストレージ機能
  private def insert(user: User): User = ???

  private def createUser(rs: WrappedResultSet): User = ???

  private def find(name: String): Option[User] = ???

  private def find(id: Long): Option[User] = ???

  // パスワード機能
  private def hashPassword(rawPassword: String): String = ???

  private def checkPassword(rawPassword: String, hashedPassword: String): Boolean = ???

  // ユーザー登録
  def register(name: String, rawPassword: String): User = ???

  // ユーザー認証
  def login(name: String, rawPassword: String): User = ???
}
```

これで insert メソッドや find メソッドは外部から呼びだすことができなくなったので、先ほどの問題は起きなくなりました。

しかしトレイトを使って同じように公開したい機能を制限することもできます。まず、公開するメソッドだけを集めた新しいトレイト UserService を作ります。

```
trait UserService {
  val maxNameLength = 32

  def register(name: String, rawPassword: String): User

  def login(name: String, rawPassword: String): User
}
```

そして、このトレイトの実装クラス UserServiceImpl を作ります。

```
class UserServiceImpl extends UserService {
  // メソッドの実装は同じなので???で代用しています

  // ストレージ機能
  def insert(user: User): User = ???
```

```
6
7   def createUser(rs: WrappedResultSet): User = ???
8
9   def find(name: String): Option[User] = ???
10
11  def find(id: Long): Option[User] = ???
12
13  // パスワード機能
14  def hashPassword(rawPassword: String): String = ???
15
16  def checkPassword(rawPassword: String, hashedPassword: String): Boolean = ???
17
18  // ユーザー登録
19  def register(name: String, rawPassword: String): User = ???
20
21  // ユーザー認証
22  def login(name: String, rawPassword: String): User = ???
23 }
```

UserService の利用者は実装クラスではなく公開トレイトのほうだけを参照するようにすれば、チェックされていないメソッドを使って不整合データができてしまう問題も起きません。

このようにモジュールのインタフェースを定義し、公開する機能を制限するのもトレイトの使われ方の 1 つです。Java のインタフェースと同じような使い方ですね。

22.3.1　リファクタリング：大きなモジュールを分割する

次にこのモジュールの問題点として挙げられるのは、モジュールが多くの機能を持ちすぎているということです。UserService はユーザー登録とユーザー認証をおこなうサービスですが、付随してパスワードをハッシュ化したり、パスワードをチェックする機能も持っています。

このパスワード機能は UserService 以外でも使いたいケースが出てくるかもしれません。たとえばユーザーが重要な操作をした場合に再度パスワードを入力させ、チェックしたい場合などです。

このような場合、1 つのモジュールを複数のモジュールに分割することが考えられます。あたらしくパスワード機能だけを持つ PasswordService と PasswordServiceImpl を作ってみます。

```
1  package domain
2
3  import org.mindrot.jbcrypt.BCrypt
4
5  trait PasswordService {
6    def hashPassword(rawPassword: String): String
7
8    def checkPassword(rawPassword: String, hashedPassword: String): Boolean
9  }
10
11 trait PasswordServiceImpl extends PasswordService {
12   def hashPassword(rawPassword: String): String =
13     BCrypt.hashpw(rawPassword, BCrypt.gensalt())
14
```

```
15    def checkPassword(rawPassword: String, hashedPassword: String): Boolean =
16      BCrypt.checkpw(rawPassword, hashedPassword)
17  }
```

./example_projects/trait-refactored-example/src/main/scala/domain/PasswordService.scala

そして、先ほど作った UserServiceImpl に PasswordServiceImpl を継承して使うようにします。

```
1   class UserServiceImpl extends UserService with PasswordServiceImpl {
2     // メソッドの実装は同じなので???で代用しています
3
4     // ストレージ機能
5     def insert(user: User): User = ???
6
7     def createUser(rs: WrappedResultSet): User = ???
8
9     def find(name: String): Option[User] = ???
10
11    def find(id: Long): Option[User] = ???
12
13    // ユーザー登録
14    def register(name: String, rawPassword: String): User = ???
15
16    // ユーザー認証
17    def login(name: String, rawPassword: String): User = ???
18  }
```

これでパスワード機能を分離することができました。分離したパスワード機能は別の用途で使うこともできるでしょう。

同じようにストレージ機能も UserRepository として分離してみます。

```
1   package domain
2
3   import scalikejdbc._
4
5   trait UserRepository {
6     def insert(user: User): User
7
8     def find(name: String): Option[User]
9
10    def find(id: Long): Option[User]
11  }
12
13  trait UserRepositoryImpl extends UserRepository {
14    def insert(user: User): User = DB localTx { implicit s =>
15      val id = sql"""insert into users (name, password) values (${user.name}, ${user.
        hashedPassword})"""
16        .updateAndReturnGeneratedKey.apply()
17      user.copy(id = id)
18    }
19
20    def createUser(rs: WrappedResultSet): User =
21      User(rs.long("id"), rs.string("name"), rs.string("password"))
```

```
22
23    def find(name: String): Option[User] = DB readOnly { implicit s =>
24      sql"""select * from users where name = $name """
25        .map(createUser).single.apply()
26    }
27
28    def find(id: Long): Option[User] = DB readOnly { implicit s =>
29      sql"""select * from users where id = $id """
30        .map(createUser).single.apply()
31    }
32  }
```

./example_projects/trait-refactored-example/src/main/scala/domain/UserRepository.scala

すると、UserServiceImpl は以下のようになります。

```
1  class UserServiceImpl extends UserService with PasswordServiceImpl with
       UserRepositoryImpl {
2    // メソッドの実装は同じなので???で代用しています
3
4    // ユーザー登録
5    def register(name: String, rawPassword: String): User = ???
6
7    // ユーザー認証
8    def login(name: String, rawPassword: String): User = ???
9  }
```

これで大きな UserService モジュールを複数の機能に分割することできました。

22.4　依存性の注入によるリファクタリング

さて、いよいよこの節の主題である依存性の注入によるリファクタリングの話に入りましょう。

ここまでトレイトを使って公開する機能を定義し、モジュールを分割し、リファクタリングを進めてきましたが、UserService にはもう 1 つ大きな問題があります。モジュール間の依存関係が分離できていないことです。

たとえば UserServiceImpl のユニットテストをすることを考えてみましょう。ユニットテストは外部のシステムを使わないので、ローカル環境でテストしやすく、並行して複数のテストをすることもできます。また、単体の機能のみをテストするため失敗した場合、問題の箇所がわかりやすいという特徴もあります。

UserServiceImpl は UserRepositoryImpl に依存しています。UserRepositoryImpl は ScalikeJDBC を使った外部システムであるデータベースのアクセスコードがあります。このままの UserServiceImpl でユニットテストを作成した場合、テストを実行するのにデータベースを用意しなければならず、データベースを共用する場合複数のテストを同時に実行するのが難しくなります。さらにテストに失敗した場合 UserServiceImpl に原因があるのか、もしくはデータベースの設定やテーブルに原因があるのか、調査しなければなりません。これではユニットテストとは言えません。

そこで具体的な `UserRepositoryImpl` への依存を分離することが考えられます。このために使われるのが依存性の注入と呼ばれる手法です。

22.4.1　依存性の注入とは？

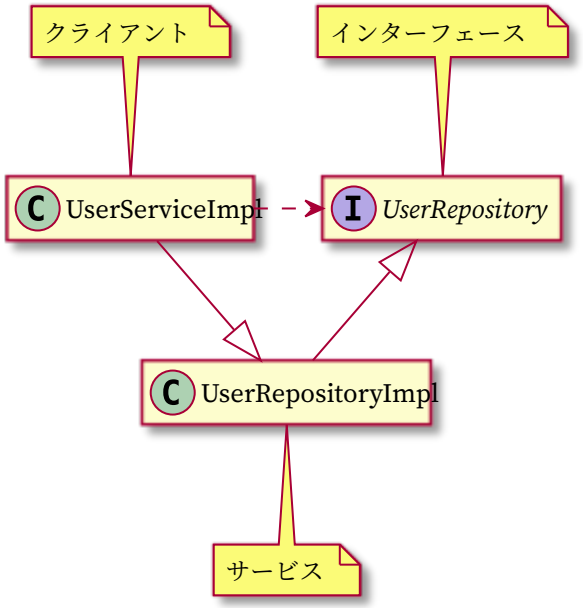
まずは一般的な「依存性の注入（Dependency Injection、DI）」の定義を確認しましょう。
依存性の注入について Wikipedia の Dependency injection の項目を見てみますと、

- Dependency とは実際にサービスなどで使われるオブジェクトである
- Injection とは Dependency を使うオブジェクトに渡すことである

とあります。さらに DI には以下の 4 つの役割が登場するとあります。

- 使われる対象の「サービス」
- サービスを使う（依存する）「クライアント」
- クライアントがどうサービスを使うかを定めた「インタフェース」
- サービスを構築し、クライアントに渡す「インジェクタ」

これらの役割について、今回の例の `UserRepository`、`UserRepositoryImpl`、`UserServiceImpl` で考えてみます。Wikipedia 中の「サービス」という用語と、これまでの例の中で登場するサービスという言葉は別の意味なので注意してください。
まずは DI を使っていない状態のクラス図を見てみましょう。

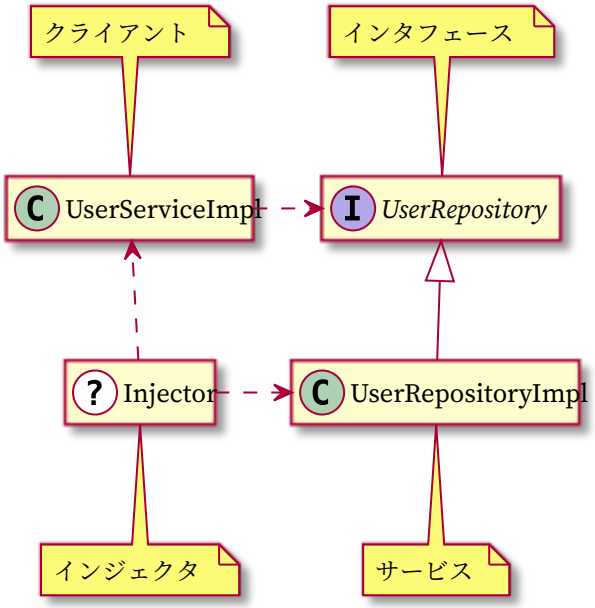


このクラス図の役割を表にしてみます。

DI の役割	コード上の名前	説明
インタフェース	UserRepository	抽象的なインタフェース
サービス	UserRepositoryImpl	具体的な実装
クライアント	UserServiceImpl	UserRepository の利用者

DI を使わない状態では `UserRepository` というインタフェースが定義されているのにもかかわらず、`UserServiceImpl` は `UserRepositoryImpl` を継承することで実装も参照していました。これではせっかくインタフェースを分離した意味がありません。`UserServiceImpl` が `UserRepository` インタフェースだけを参照（依存）するようにすれば、具体的な実装である `UserRepositoryImpl` の変更に影響されることはありません。この問題を解決するのが DI の目的です。

それでは DI のインジェクタを加えて、上記のクラス図を修正しましょう。



謎のインジェクタの登場により `UserServiceImpl` から `UserRepositoryImpl` への参照がなくなりました。おそらくインジェクタは何らかの手段でサービスである `UserRepositoryImpl`（Dependency）をクライアントである `UserServiceImpl` に渡しています（Injection）。このインジェクタの動作を指して「Dependency Injection」と呼ぶわけです。そして、このインジェクタをどうやって実現するか、それが DI 技術の核心に当たります。

22.4.2　依存性の注入の利点

では、この依存性の注入を使うとどのような利点があるのでしょうか。

1 つはクライアントがインタフェースだけを参照することにより、具体的な実装への参照が少なくなり**コンポーネント同士が疎結合になる**という点が挙げられます。たとえば `UserRepositoryImpl` の

クラス名やパッケージ名が変更されても `UserServiceImpl` には何の影響もなくなります。

次に挙げられる点は具体的な実装を差し替えることにより**クライアントの動作がカスタマイズ可能**になるという点です。たとえば今回の例では `UserRepositoryImpl` は ScalikeJDBC の実装でしたが、MongoDB に保存する `MongoUserRepositoryImpl` を新しく作って `UserServiceImpl` に渡せばクライアントを MongoDB に保存するように変更することができます。

また DI は設計レベルでも意味があります。DI を使うと**依存関係逆転の原則を実現できます**。通常の手続き型プログラミングでは、上位のモジュールから下位の詳細な実装のモジュールを呼ぶということがしばしばあります。しかし、この場合、上位のモジュールが下位のモジュールに依存することになります。つまり、下位の実装の変更が上位のモジュールにまで影響することになってしまうわけです。

依存関係逆転の原則というのは、上位のモジュールの側に下位のモジュールが実装すべき抽象を定義し、下位のモジュールはその抽象に対して実装を提供すべきという考え方です。たとえば `UserService` が上位のモジュールだとすると、`UserRepositoryImpl` は下位のモジュールになります。依存関係逆転をしない場合、`UserService` から直接 `UserRepositoryImpl` を呼ばざるをえません。このままだと先ほど述べたような問題が生じてしまうので、依存関係逆転の原則に従って、上位のモジュールに抽象的な `UserRepository` を用意し、`UserService` は `UserRepository` を使うようにします。そして、依存性の注入によって、下位のモジュールの `UserRepositoryImpl` を `UserService` に渡すことにより、このような問題を解決できるわけです。

また見落されがちな点ですが、DI では**クライアントに特別な実装を要求しない**という点も重要です。これは Java 界隈で DI が登場した背景に関連するのですが、Spring Framework などの DI を実現する DI コンテナは複雑な Enterprise JavaBeans(EJB) に対するアンチテーゼとして誕生しました。複雑な EJB に対し、何も特別でないただの Java オブジェクトである Plain Old Java Object(POJO) という概念が提唱され、わかりやすさや、言語そのものの機能による自由な記述が重視されました。DI の登場にはそのような背景があり、クライアントに対して純粋な言語機能以外求められないことが一般的です。

最後に、先ほども触れましたが**依存オブジェクトのモック化によるユニットテストが可能になる**という点です。たとえば Web アプリケーションについて考えると、Web アプリケーションは様々な外部システムを使います。Web アプリケーションは MySQL や Redis などのストレージを使い、Twitter や Facebook などの外部サービスにアクセスすることもあるでしょう。また刻一刻と変化する時間や天候などの情報を使うかもしれません。このような外部システムが関係するモジュールはユニットテストすることが困難です。DI を使えば外部システムの実装を分離できるので、モックに置き換えて、楽にテストできるようになります。

以上、DI の利点を見てきました。実装オブジェクト（Dependency）を取得し、サービスに渡す（Injection）という役割をするだけのインジェクタの登場により様々なメリットが生まれることが理解できたと思います。DI は特に大規模システムの構築に欠かせない技術であると言っても過言ではないと思います。

第 23 章

付録：様々な型クラスの紹介

本章では 16 章で説明した型クラスの具体例を紹介します。本章で紹介する型クラスは、必ずしも Scala でのプログラミングに必要というわけではありません。しかし、世の中に存在する Scala で実装されたライブラリやアプリケーションのいくつかでは、本章で紹介する型クラスなどを多用している場合があります。そのようなライブラリやアプリケーションに出会った際にも臆さずコードリーディングができるよう、最低限の知識をつけることが本章の目的です。

本章で紹介する型クラスを絡めた Scala でのプログラミングについて詳しく知りたい場合は Scala 関数型デザイン＆プログラミングを読みましょう。

23.1 Functor

前章に登場した `List` や `Option` には、`map` という関数が共通して定義されていました。この `map` 関数がある規則を満たす場合は Functor 型クラスとして抽象化できます[*1]。

```
1 trait Functor[F[_]] {
2   def map[A, B](fa: F[A])(f: A => B): F[B]
3 }
```

この型クラスが満たすべき規則は 2 つです。

```
1 def identityLaw[F[_], A](fa: F[A])(implicit F: Functor[F]): Boolean =
2   F.map(fa)(identity) == fa
3
4 def compositeLaw[F[_], A, B, C](fa: F[A], f1: A => B, f2: B => C)(implicit F: Functor[F
      ]): Boolean =
5   F.map(fa)(f2 compose f1) == F.map(F.map(fa)(f1))(f2)
```

[*1] ここで出現する `F` は、通常の型ではなく、「何かの型を受け取って、型を返すもの」で、型構築子、型コンストラクタなどと呼びます。`List` や `Option` は型構築子の一種です。詳細については、型システム入門プログラミング言語と型の理論の第 VI 部「高階の型システム」を参照してください。

なお、identity は次のように定義されます。

```
def identity[A](a: A): A = a
```

例として、Option 型で Functor 型クラスのインスタンスを定義し、前述の規則を満たすかどうか調べてみましょう。

```
trait Functor[F[_]] {
  def map[A, B](fa: F[A])(f: A => B): F[B]
}

def identityLaw[F[_], A](fa: F[A])(implicit F: Functor[F]): Boolean =
  F.map(fa)(identity) == fa

def compositeLaw[F[_], A, B, C](fa: F[A], f1: A => B, f2: B => C)(implicit F: Functor[F
    ]): Boolean =
  F.map(fa)(f2 compose f1) == F.map(F.map(fa)(f1))(f2)

implicit object OptionFunctor extends Functor[Option] {
  def map[A, B](fa: Option[A])(f: A => B): Option[B] = fa.map(f)
}

val n: Option[Int] = Some(2)
// n: Option[Int] = Some(2)
identityLaw(n)
// res0: Boolean = true
compositeLaw(n, (i: Int) => i * i, (i: Int) => i.toString)
// res1: Boolean = true
```

23.2　Applicative Functor

複数の値が登場する場合には Functor では力不足です。そこで、複数の引数を持つ関数と値を組み合わせて 1 つの値を作りだせる機能を提供する Applicative Functor が登場します。

```
trait Applicative[F[_]] {
  def point[A](a: A): F[A]
  def ap[A, B](fa: F[A])(f: F[A => B]): F[B]
}
```

Applicative Functor は Functor を特殊化したものなので、Applicative Functor が持つ関数から map 関数を定義できます。

```
def map[F[_], A, B](fa: F[A])(f: A => B)(implicit F: Applicative[F]): F[B] =
  F.ap(fa)(F.point(f))
```

Applicative Functor が満たすべき規則は以下の通りです。

```
def identityLaw[F[_], A](fa: F[A])(implicit F: Applicative[F]): Boolean =
  F.ap(fa)(F.point((a: A) => a)) == fa
```

```
3
4  def homomorphismLaw[F[_], A, B](f: A => B, a: A)(implicit F: Applicative[F]): Boolean =
5    F.ap(F.point(a))(F.point(f)) == F.point(f(a))
6
7  def interchangeLaw[F[_], A, B](f: F[A => B], a: A)(implicit F: Applicative[F]): Boolean
       =
8    F.ap(F.point(a))(f) == F.ap(f)(F.point((g: A => B) => g(a)))
```

また、ap と point を使って定義した map 関数が Functor のものと同じ振る舞いになることを確認する必要があります。

例として、Option 型で Applicative Functor を定義してみましょう。

```
1  trait Applicative[F[_]] {
2    def point[A](a: A): F[A]
3    def ap[A, B](fa: F[A])(f: F[A => B]): F[B]
4    def map[A, B](fa: F[A])(f: A => B): F[B] = ap(fa)(point(f))
5  }
6
7  def identityLaw[F[_], A](fa: F[A])(implicit F: Applicative[F]): Boolean =
8    F.ap(fa)(F.point((a: A) => a)) == fa
9
10 def homomorphismLaw[F[_], A, B](f: A => B, a: A)(implicit F: Applicative[F]): Boolean =
11   F.ap(F.point(a))(F.point(f)) == F.point(f(a))
12
13 def interchangeLaw[F[_], A, B](f: F[A => B], a: A)(implicit F: Applicative[F]): Boolean
       =
14   F.ap(F.point(a))(f) == F.ap(f)(F.point((g: A => B) => g(a)))
15
16 implicit object OptionApplicative extends Applicative[Option] {
17   def point[A](a: A): Option[A] = Some(a)
18   def ap[A, B](fa: Option[A])(f: Option[A => B]): Option[B] = f match {
19     case Some(g) => fa match {
20       case Some(a) => Some(g(a))
21       case None => None
22     }
23     case None => None
24   }
25 }
26
27 val a: Option[Int] = Some(1)
28 // a: Option[Int] = Some(1)
29 val f: Int => String = { i => i.toString }
30 // f: Function1[Int, String] = repl.MdocSession$MdocApp$$Lambda$14689/0
       x0000743a8e14e830@44063750
31 val af: Option[Int => String] = Some(f)
32 // af: Option[Function1[Int, String]] = Some(
33 //   repl.MdocSession$MdocApp$$Lambda$14689/0x0000743a8e14e830@44063750
34 // )
35 identityLaw(a)
36 // res2: Boolean = true
37 homomorphismLaw(f, 1)
38 // res3: Boolean = true
```

```
interchangeLaw(af, 1)
// res4: Boolean = true
OptionApplicative.map(a)(_ + 1) == OptionFunctor.map(a)(_ + 1)
// res5: Boolean = true
```

23.3　Monad

ある値を受け取りその値を包んだ型を返す関数を Applicative Functor で扱おうとすると、型がネストしてしまい平坦化できません。このネストする問題を解決するために Monad と呼ばれる型クラスを用います。

```
trait Monad[F[_]] {
  def point[A](a: A): F[A]
  def bind[A, B](fa: F[A])(f: A => F[B]): F[B]
}
```

bind は Option や List で登場した flatMap を抽象化したものです。

Monad は以下の規則を満たす必要があります。

```
def rightIdentityLaw[F[_], A](a: F[A])(implicit F: Monad[F]): Boolean =
  F.bind(a)(F.point(_)) == a

def leftIdentityLaw[F[_], A, B](a: A, f: A => F[B])(implicit F: Monad[F]): Boolean =
  F.bind(F.point(a))(f) == f(a)

def associativeLaw[F[_], A, B, C](fa: F[A], f: A => F[B], g: B => F[C])(implicit F:
    Monad[F]): Boolean =
  F.bind(F.bind(fa)(f))(g) == F.bind(fa)((a: A) => F.bind(f(a))(g))
```

Monad は Applicative Functor を特殊化したものなので、Monad が持つ関数から point 関数と ap 関数を定義できます。point に関しては同じシグネチャなので自明でしょう。

```
def ap[F[_], A, B](fa: F[A])(f: F[A => B])(implicit F: Monad[F]): F[B] =
  F.bind(f)((g: A => B) => F.bind(fa)((a: A) => F.point(g(a))))
```

それでは、Option 型が前述の規則をみたすかどうか確認してみましょう。

```
trait Monad[F[_]] {
  def point[A](a: A): F[A]
  def bind[A, B](fa: F[A])(f: A => F[B]): F[B]
}

def rightIdentityLaw[F[_], A](a: F[A])(implicit F: Monad[F]): Boolean =
  F.bind(a)(F.point(_)) == a

def leftIdentityLaw[F[_], A, B](a: A, f: A => F[B])(implicit F: Monad[F]): Boolean =
  F.bind(F.point(a))(f) == f(a)

```

```
def associativeLaw[F[_], A, B, C](fa: F[A], f: A => F[B], g: B => F[C])(implicit F:
    Monad[F]): Boolean =
  F.bind(F.bind(fa)(f))(g) == F.bind(fa)((a: A) => F.bind(f(a))(g))

implicit object OptionMonad extends Monad[Option] {
  def point[A](a: A): Option[A] = Some(a)
  def bind[A, B](fa: Option[A])(f: A => Option[B]): Option[B] = fa match {
    case Some(a) => f(a)
    case None => None
  }
}

val fa: Option[Int] = Some(1)
// fa: Option[Int] = Some(1)
val f: Int => Option[Int] = { n => Some(n + 1) }
// f: Function1[Int, Option[Int]] = repl.MdocSession$MdocApp$$Lambda$14694/0
    x0000743a8e150e40@58081046
val g: Int => Option[Int] = { n => Some(n * n) }
// g: Function1[Int, Option[Int]] = repl.MdocSession$MdocApp$$Lambda$14695/0
    x0000743a8e151430@19a8e6ab
rightIdentityLaw(fa)
// res6: Boolean = true
leftIdentityLaw(1, f)
// res7: Boolean = true
associativeLaw(fa, f, g)
// res8: Boolean = true
```

23.4　Monoid

2 つの同じ型を結合する機能を持ち、更にゼロ値を知る型クラスは Monoid と呼ばれています。

```
trait Monoid[F] {
  def append(a: F, b: F): F
  def zero: F
}
```

前章で定義した Additive 型とよく似ていますが、Monoid は次の規則を満たす必要があります。

```
def leftIdentityLaw[F](a: F)(implicit F: Monoid[F]): Boolean = a == F.append(F.zero, a)
def rightIdentityLaw[F](a: F)(implicit F: Monoid[F]): Boolean = a == F.append(a, F.zero
    )
def associativeLaw[F](a: F, b: F, c: F)(implicit F: Monoid[F]): Boolean = {
  F.append(F.append(a, b), c) == F.append(a, F.append(b, c))
}
```

Option[Int] 型で Monoid インスタンスを定義してみましょう。

```
trait Monoid[F] {
  def append(a: F, b: F): F
  def zero: F
```

```
}

def leftIdentityLaw[F](a: F)(implicit F: Monoid[F]): Boolean = a == F.append(F.zero, a)
def rightIdentityLaw[F](a: F)(implicit F: Monoid[F]): Boolean = a == F.append(a, F.zero
    )
def associativeLaw[F](a: F, b: F, c: F)(implicit F: Monoid[F]): Boolean = {
  F.append(F.append(a, b), c) == F.append(a, F.append(b, c))
}

implicit object OptionIntMonoid extends Monoid[Option[Int]] {
  def append(a: Option[Int], b: Option[Int]): Option[Int] = (a, b) match {
    case (None, None) => None
    case (Some(v), None) => Some(v)
    case (None, Some(v)) => Some(v)
    case (Some(v1), Some(v2)) => Some(v1 + v2)
  }
  def zero: Option[Int] = None
}

val n: Option[Int] = Some(1)
// n: Option[Int] = Some(1)
val m: Option[Int] = Some(2)
// m: Option[Int] = Some(2)
val o: Option[Int] = Some(3)
// o: Option[Int] = Some(3)
leftIdentityLaw(n)
// res9: Boolean = true
rightIdentityLaw(n)
// res10: Boolean = true
associativeLaw(n, m, o)
// res11: Boolean = true
```

型によっては結合方法が複数存在する場合があります。その際は複数の Monoid インスタンスを定義しておき、状況に応じて使いたい Monoid インスタンスを選択できるようにしておきましょう。